

For ASTU use prepared by the request of ENNCoMat project.

Advanced Numerical Methods (MATH5133)

Prepared by: Mesfin Mekuria Woldaregay¹,
email: msfnmkr02@gmail.com,

¹Department of Applied Mathematics, Adama Science and Technology
University, Adama, Ethiopia

Revised December 22, 2025

Contents

1	Introduction to Differential Equations	3
1.1	What is an Initial Value Problem?	4
1.1.1	Physical Motivation: Why IVPs Matter?	5
1.1.2	Why Initial Conditions are Essential?	7
1.1.3	Computational Examples with Matlab	8
1.1.4	Elementary Theory of Initial-Value Problems	13
1.1.5	Review Excercise	17
1.2	Introduction to BVPs	18
1.2.1	Existence and Uniqueness of Solutions of BVPs	19
1.3	Introduction to PDEs	22
1.3.1	Classification of Second-Order PDEs	24
1.3.2	Examples of Classification	24
2	Numerical Methods for IVPs	27
2.1	Numerical Methods for First Order IVPs	27
2.1.1	Euler's Method	27
2.1.2	Taylor's Methods	36
2.1.3	Runge Kutta Methods	42
2.1.4	Runge-Kutta Methods of Order Two	43
2.1.5	Midpoint Method	45
2.1.6	Higher-Order Runge-Kutta Methods	50
2.1.7	Computational Comparisons	56
2.1.8	Stability analysis of single step explicit methods	57
2.2	Multistep Methods	63
2.2.1	Common Examples of Multistep Methods	64
2.2.2	Derivation via Polynomial Interpolation	65

2.2.3	Local Truncation Error (LTE)	66
2.2.4	LTE of the Adams–Moulton Two-Step (AM2) Method	68
2.2.5	Common Adams Methods	69
2.2.6	Predictor–Corrector Methods	75
2.3	Systems of Differential Equations	79
2.3.1	Euler’s Method for Systems	80
2.3.2	Fourth-Order Runge–Kutta Method for Systems	80
2.3.3	Higher-Order Equations and Systems of IVPs	85
2.3.4	System of Higher-Order Differential Equations	90
3	Numerical Methods for Boundary Value Problems	95
3.1	Shooting Method	95
3.1.1	Linear Shooting Method	96
3.1.2	Nonlinear Shooting Method	104
3.2	Finite Difference Method for BVPs	113
3.2.1	Finite Difference Approximations	113
3.2.2	Finite Difference Method for Linear BVPs	122
3.2.3	Finite Difference Method for Nonlinear BVPs	133
3.2.4	Stability and Convergence Analysis	136
3.2.5	General Nonlinear BVP: $u'' = f(x, u, u')$	142
4	Explicit Numerical Method for PDEs	153
4.1	Explicit Method for Parabolic PDE	156
4.1.1	Stability Analysis (Von Neumann Method)	158
4.1.2	Consistency	159
4.1.3	Convergence	159
4.2	Finite Difference Methods for First-Order Hyperbolic PDEs	163
4.2.1	Forward-Time Forward-Space (FTFS) Scheme	163
4.2.2	Forward-Time Central-Space (FTCS) Scheme	164
4.2.3	Lax–Friedrichs Scheme	164
4.2.4	Leap-Frog Scheme	165
4.2.5	Lax–Wendroff Scheme	165
4.2.6	Summary of Explicit Schemes	166
4.3	Finite Difference Methods for Second-Order Hyperbolic PDEs	172
4.4	Elliptic Partial Differential Equations	174

4.4.1	Gravitational and Electrostatic Potentials	175
4.4.2	Steady-State Heat Conduction	175
4.4.3	Dirichlet Problem for Laplace's Equation	175
4.4.4	Finite Difference Approximation	176

List of Tables

2.1	Comparison of Euler method approximation and exact solution	31
2.2	Comparison of Actual Errors and Error Bounds for Euler's Method	35
2.3	Solution using Euler, Taylor-2, and Taylor-4 methods ($N = 20$, $h = 0.1$). . .	40
2.4	Verification of order conditions for Kutta's RK3.	52
2.5	Runge-Kutta method of order four and errors	55
2.6	Best Possible Local Truncation Error vs. Function Evaluations per Step . . .	56
2.7	Computational comparison with fair function evaluation.	57
2.8	Comparison of Adams-Bashforth methods (step size $h = 0.2$).	70
2.9	AB4-AM4 Predictor-Corrector method.	78
2.10	Comparison of Euler and RK4 Methods for the Given System	82
2.11	Maximum Absolute Errors on $t \in [0, 1]$ with Step Size $h = 0.05$	84
3.1	Numerical results for Example 3.1 ($N = 10$).	104
3.2	Solution of nonlinear BVP using Shooting method with Newton iteration with $Tol = 10^{-5}$, $N = 10$, $m = 10$	111
3.3	Finite difference approximations for $u'(x)$: formulas, stencils, leading error terms, and orders.	116
3.4	Absolute errors in derivative approximations for $u_1(x) = \sin(10x)$ at $x = 1$. .	117
3.5	Errors in derivative approximations for $u_2(x)$ at $x = 1$	118
3.6	Errors in derivative approximations for $u_3(x)$ at $x = 1$	119
3.7	Convergence analysis: errors and experimental order of convergence	128
3.8	Summary of boundary condition treatments in finite difference discretization.	131
3.9	Newton iteration performance for the nonlinear BVP $-u'' + 1 + e^u + u^4 = x + x^2$, $u(0) = u(1) = 0$	141
3.10	Convergence results for the nonlinear BVP.	148

4.1	Absolute errors for the Explicit method at selected (x, t) using 20 equally spaced spatial points on $[0, \pi]$ and $M = 256$ time steps.	162
4.2	Explicit finite difference schemes for the advection equation	166

List of Figures

1.1	Growth at 3% year	9
1.2	Coffee Cooling	9
1.3	Exponential Growth ($k = 2$)'	11
1.4	Radioactive decay.	12
1.5	Comparison of a convex set R and a non-convex set A . The red dashed line segment connects two points in the set. In A , the segment passes through the hole (where $x^2 + y^2 < 1$), so it is not contained in the set. Hence, A is not convex.	14
2.1	Geometric interpretation of Euler's method: the slope at each point is used to step forward.	29
2.2	Plot of the Euler's method approximation and exact solution for $h = 0.2$	31
2.3	Plot of Euler's method solution and exact solution for different h values.	31
2.4	Taylor methods comparison.	40
2.5	Stability regions for Runge–Kutta methods of orders 1 to 4.	61
2.6	Predictor corrector method.	79
2.7	Comparison of Euler method and RK4 method.	84
3.1	Graphical representation of the linear shooting method.	97
3.2	Top: Approximate and exact solutions. Bottom: Absolute error.	103
3.3	The procedure of shooting to determine the initial elevation.	106
3.4	Shooting method for different level.	106
3.5	Plot of errors vs. h for $u_1(x) = \sin(10x)$	117
3.6	Plot of errors vs. h for $u_2(x)$	119
3.7	Plot of errors vs. h for $u_3(x)$	120
3.8	Numerical solution and residual for $N = 64$	142

3.9	Comparison of Exact solution with FDM in left side, Absolute error on right side.	149
4.1	Representation of 2D grid on (x, t) coordinate.	154
4.2	Surface plot of the solution	161
4.3	Comparison of the explicit methods.	167
4.4	Five-point stencil for Laplace's equation.	176

Chapter 1

Introduction to Differential Equations

A differential equation (DE) is an equation that relates a function with one or more of its derivatives. In other words, it describes how a quantity changes in relation to other quantities. Differential equations are fundamental tools in mathematics, physics, engineering, biology, economics, and many other fields because they model dynamic systems that evolve over time or space.

Why are Differential Equations Important?

Differential equations (DEs) are among the most powerful and widely used mathematical tools for modeling dynamic processes in the natural and engineered world. Their significance arises from their unique ability to encode the *rate of change* of a system as a function of its current state and external influences. This makes them indispensable across scientific and engineering disciplines for the following reasons:

I. Modeling of Real-World Phenomena: Differential equations naturally describe systems that evolve continuously over time or space. Examples include:

- *Population dynamics:* The logistic or exponential growth models, $\frac{dP}{dt} = kP$, capture how populations change under resource constraints or ideal conditions.
- *Radioactive decay:* Governed by $\frac{dN}{dt} = -\lambda N$, where the decay rate is proportional to the number of undecayed nuclei.
- *Heat conduction:* Described by the heat equation $\frac{\partial u}{\partial t} = \alpha \nabla^2 u$, a partial differential equation modeling temperature diffusion in solids.

- *Fluid flow*: The Navier–Stokes equations model velocity, pressure, and density fields in fluids.
- *Electrical circuits*: Kirchhoff’s laws lead to ODEs such as $L\frac{dI}{dt} + RI = V(t)$ for an RL circuit.
- *Celestial mechanics*: Newton’s law of gravitation yields second-order ODEs that predict planetary orbits.

II. Predictive Capability: Given an initial (or boundary) condition, solving a differential equation provides the state of the system at any future (or past) time. This predictive power is foundational in simulation, control theory, and forecasting from climate models to epidemic spread (e.g., SIR models).

III. Fundamental Laws of Physics are Expressed as DEs: The core principles of classical and modern physics are inherently differential:

- *Newton’s second law*: $F = m\frac{d^2x}{dt^2}$ is a second-order ODE relating force to acceleration.
- *Maxwell’s equations*: A system of PDEs governing electric and magnetic fields, forming the foundation of electromagnetism.
- *Schrödinger’s equation*: $i\hbar\frac{\partial\psi}{\partial t} = \hat{H}\psi$, a PDE central to quantum mechanics.
- *Einstein’s field equations*: Nonlinear PDEs that describe gravity in general relativity. Thus, differential equations are not merely mathematical abstractions, they are the *language of physical law*.

Consequently, the study, analysis, and numerical solution of differential equations particularly through robust, efficient, and stable algorithms remains a central theme in applied mathematics and computational science.

1.1 What is an Initial Value Problem?

An **Initial Value Problem (IVP)** consists of a differential equation together with a specified value of the unknown function at a particular point usually at the “start” of the process.

For a first-order ordinary differential equation (ODE), the general form of an IVP is:

$$\begin{cases} \frac{dy}{dt} = f(t, y), & a \leq t \leq b, \\ y(a) = \alpha, \end{cases} \quad (1.1)$$

where: $y(t)$ is the unknown function (e.g., position, temperature, population), $f(t, y)$ describes the rate of change of y , $y(a) = \alpha$ is the **initial condition**, representing the state of the system at time $t = a$.

The goal is to find a function $y(t)$ that satisfies both the differential equation and the initial condition.

Remark 1.1. *Unlike boundary value problems (BVPs), where conditions are imposed at multiple points, IVPs model evolution over time, starting from a known state.*

1.1.1 Physical Motivation: Why IVPs Matter?

Many natural processes evolve according to **laws of change**. These are often expressed as differential equations. The initial condition reflects our knowledge of the system at a specific moment like “where it starts” and allows us to predict its future behavior. We now explore several classical examples.

Example 1: Population Growth

Suppose a population of bacteria grows at a rate proportional to its current size. Let $P(t)$ be the population at time t . Then:

$$\frac{dP}{dt} = kP,$$

where $k > 0$ is the growth rate. If we know the initial population $P(0) = P_0$, the IVP becomes:

$$\begin{cases} \frac{dP}{dt} = kP, \\ P(0) = P_0. \end{cases}$$

This models **exponential growth**. The solution is:

$$P(t) = P_0 e^{kt}.$$

Without the initial condition $P(0) = P_0$, we cannot determine the constant of integration. The initial condition **calibrates the model** to real data.

Remark 1.2. *In more realistic models, growth may slow due to limited resources (logistic model), but the IVP structure remains central.*

Example 2: Radioactive Decay

Radioactive substances decay at a rate proportional to the amount present. Let $A(t)$ be the amount of substance at time t . Then:

$$\frac{dA}{dt} = -kA, \quad k > 0.$$

Given an initial amount $A(0) = A_0$, the IVP is:

$$\begin{cases} \frac{dA}{dt} = -kA, \\ A(0) = A_0. \end{cases}$$

The solution is: $A(t) = A_0 e^{-kt}$.

This describes **exponential decay**. The half-life $T_{1/2} = \frac{\ln 2}{k}$ can be computed from k , but the actual amount at any time depends on A_0 .

Remark 1.3. *In experiments, A_0 is measured at $t = 0$; without it, we cannot predict how much material remains after, say, 10 years.*

Example 3: Newton's Law of Cooling

A hot object cools in a surrounding medium at a rate proportional to the temperature difference. Let $T(t)$ be the object's temperature, and T_m the constant ambient temperature. Then:

$$\frac{dT}{dt} = -k(T - T_m), \quad k > 0.$$

If the initial temperature is $T(0) = T_0$, the IVP is:

$$\begin{cases} \frac{dT}{dt} = -k(T - T_m), \\ T(0) = T_0. \end{cases}$$

The solution is: $T(t) = T_m + (T_0 - T_m)e^{-kt}$. This shows how the object's temperature approaches T_m over time. The initial condition T_0 determines the starting point of the cooling curve.

Example 4: Motion of a Falling Object (with Air Resistance)

Consider a mass m falling under gravity with air resistance proportional to velocity v . Newton's second law gives:

$$m \frac{dv}{dt} = mg - \gamma v,$$

where $\gamma > 0$ is the drag coefficient. Divide by m :

$$\frac{dv}{dt} = g - \frac{\gamma}{m}v.$$

with initial velocity $v(0) = v_0$, this forms the IVP:

$$\begin{cases} \frac{dv}{dt} = g - \frac{\gamma}{m}v, \\ v(0) = v_0. \end{cases}$$

The solution approaches a **terminal velocity** $v_t = \frac{mg}{\gamma}$. The initial condition determines how quickly this limit is approached.

1.1.2 Why Initial Conditions are Essential?

The differential equation describes **how** a system changes, but not **where** it starts. Without an initial condition:

- We get a **family of solutions** (general solution),
- But no way to pick the one that matches reality.

For example, the general solution to $y' = y$ is $y(t) = Ce^t$. Only when we impose $y(0) = 2$ do we get $y(t) = 2e^t$ a specific, predictive model.

Thus, the initial condition **anchors the mathematical model to physical observation**.

Summary

- An IVP combines a differential equation with an initial condition: $y(a) = \alpha$.
- IVPs model **time-evolving systems** in physics, biology, and engineering.
- Physical examples include:

■ Population growth: $P' = kP$

- Radioactive decay: $A' = -kA$
- Newton's Law of Cooling: $T' = -k(T - T_m)$
- Motion with drag: $v' = g - cv$
- The initial condition ensures the solution is **unique and physically meaningful**.
- Without it, we cannot make quantitative predictions.

IVPs are the foundation for understanding dynamic behavior in the natural world.

1.1.3 Computational Examples with Matlab

Example 1.1. Write the IVP for a population growing at 3% per year, starting from 1000 individuals.

Solution 1.1. Population growing at 3% per year, starting from 1000 individuals. The growth rate is 3% per year, so the proportionality constant is $k = 0.03$. The initial population is $P(0) = 1000$. Thus, the initial value problem (IVP) is:

$$\begin{cases} \frac{dP}{dt} = 0.03 P, \\ P(0) = 1000. \end{cases}$$

The solution is: $P(t) = 1000 e^{0.03t}$.

```
clear; clc; close all;
%% Exercise 1: Population Growth (3% per year, P0 = 1000)
k1 = 0.03;
P0 = 1000;
t1 = linspace(0, 20, 200); % 20 years
P1 = P0 * exp(k1 * t1);
figure('Position', [100, 100, 500, 400]);
plot(t1, P1, 'b-', 'LineWidth', 2);
xlabel('Time (years)');
ylabel('Population P(t)');
title('Population Growth at 3%/year');
grid on;
fprintf('P(t) = %d * exp(%.2ft)\n', P0, k1);
```




Figure 1.1: Growth at 3% year

Example 1.2. A cup of coffee at 90°C cools in a room at 20°C with $k = 0.1$ per minute. Write the IVP and find $T(5)$.

Solution 1.2. Using Newton's Law of Cooling, $\frac{dT}{dt} = -k(T - T_m)$, where $T_m = 20^\circ\text{C}$ and $k = 0.1 \text{ min}^{-1}$. The initial condition is $T(0) = 90$. The IVP is:

$$\begin{cases} \frac{dT}{dt} = -0.1(T - 20), \\ T(0) = 90. \end{cases}$$

The solution is: $T(t) = 20 + (90 - 20)e^{-0.1t} = 20 + 70e^{-0.1t}$.

At $t = 5$ minutes: $T(5) = 20 + 70e^{-0.5} \approx 20 + 70(0.6065) \approx 62.46^\circ\text{C}$.

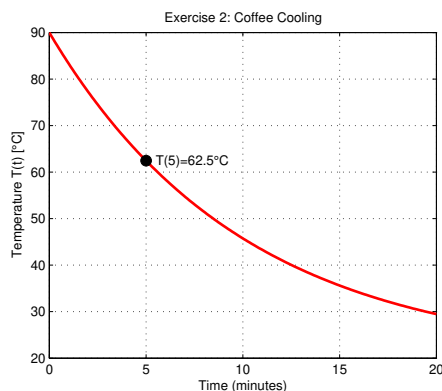


Figure 1.2: Coffee Cooling

```

%% Exercise 2: Newton's Law of Cooling
k2 = 0.1;           % per minute
Tm = 20;           % ambient temp (°C)
T0 = 90;           % initial coffee temp
t2 = linspace(0, 20, 200); % 20 minutes
T2 = Tm + (T0 - Tm) * exp(-k2 * t2);
% Compute T(5)
T_at_5 = Tm + (T0 - Tm) * exp(-k2 * 5);
fprintf('\nExercise 2:\n');
fprintf('IVP: dT/dt = -.1f*(T - %d), T(0) = %d\n', k2, Tm, T0);
fprintf('T(5) = %.2f°C\n', T_at_5);
figure('Position', [200, 100, 500, 400]);
plot(t2, T2, 'r-', 'LineWidth', 2);
hold on;
%yline(Tm, 'k--', 'Ambient (20°C)');
plot(5, T_at_5, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k');
text(5 + 0.5, T_at_5, sprintf('T(5)=%.1f°C', T_at_5), 'FontSize', 10);
xlabel('Time (minutes)');
ylabel('Temperature T(t) [°C]');
title('Exercise 2: Coffee Cooling');
grid on;

```

Example 1.3. Solve $y' = 2y$, $y(0) = 3$. Interpret as a population model.

Solution 1.3. This is a separable differential equation. The general solution is $y(t) = Ce^{2t}$. Applying the initial condition $y(0) = 3$: $3 = Ce^0 \implies C = 3$. Hence, the solution is: $y(t) = 3e^{2t}$.

Interpretation: This models a population that starts with 3 individuals and grows at a continuous rate of 200% per unit time. The population doubles approximately every $\frac{\ln 2}{2} \approx 0.347$ time units.

```

%% Example 3: Solve y'' = 2y, y(0) = 3
k3 = 2;
y0 = 3;
t3 = linspace(0, 2, 200); % short time due to fast growth
y3 = y0*exp(k3*t3);

figure('Position', [300, 100, 500, 400]);
plot(t3, y3, 'g-', 'LineWidth', 2);
xlabel('Time (units)');
ylabel('Population y(t)');

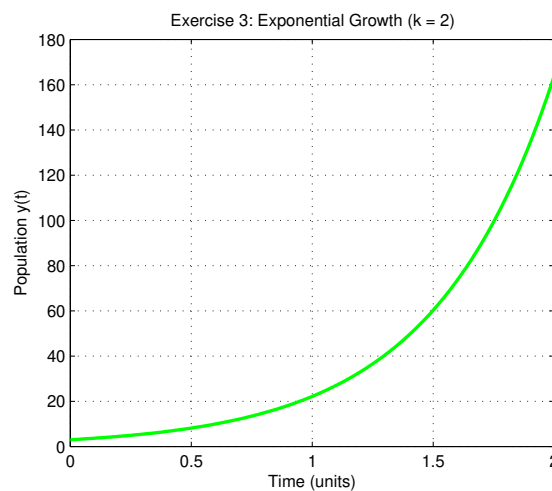
```

```

title('Exercise 3: Exponential Growth (k = 2)');
grid on;
fprintf('\nExercise 3: y(t) = %d * exp(%.1ft)\n', y0, k3);
fprintf('Interpretation: Population starts at %d and grows at 200%% per time
      unit.\n', y0);

% Optional: Show doubling time
T_double = log(2)/k3;
fprintf('Doubling time   %.3f time units\n', T_double);

```

Figure 1.3: Exponential Growth ($k = 2$)

Example 1.4. Radioactive substances decay at a rate $k = 0.1$. Let $A(t)$ be the amount of substance at time t with initial amount of $A_0 = 100$ grams. Find the amount of the substance after 50 years.

```

% Radioactive Decay Example
clear; clc; close all;
% Parameters
A0 = 100;          % Initial amount (e.g., grams)
k  = 0.1;          % Decay constant (1/time unit)
t_final = 50;      % Final time to simulate
% Time vector for analytical solution
t = linspace(0, t_final, 500);
% Analytical solution
A_analytical = A0 * exp(-k * t);

```

```

% Numerical solution using ode45
dAdt = @(t, A) -k * A;
[t_num, A_num] = ode45(dAdt, [0 t_final], A0);
% Compute half-life
T_half = log(2) / k;
% Plot results
figure;
plot(t, A_analytical, 'b-', 'LineWidth', 2); hold on;
xlabel('Time t');
ylabel('Amount A(t)');
title('Radioactive Decay Solution');
grid on;
hold on
% Display half-life
fprintf('Decay constant k = %.3f\n', k);
fprintf('Initial amount A_0 = %.1f\n', A0);
fprintf('Half-life T_{1/2} = %.3f time units\n', T_half);
% Mark half-life on plot
plot(T_half, A0/2, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k');
legend('Decay Solution', 'Half Life T_{1/2}');

```

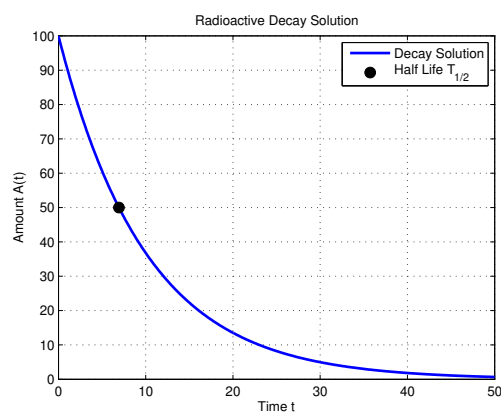


Figure 1.4: Radioactive decay.

1.1.4 Elementary Theory of Initial-Value Problems

In this section, we study the existence and uniqueness of the solution of the initial value problem (IVP) for ordinary differential equations. We begin with the IVP for a first-order ODE:

$$\begin{cases} \frac{dy}{dt} = f(t, y), & a \leq t \leq b, \\ y(a) = \alpha, \end{cases} \quad (1.2)$$

where α is a given constant.

Definition 1.1. A function $f(t, y)$ is said to satisfy a **Lipschitz condition** in the variable y on a set $D \subset \mathbb{R}^2$ if there exists a constant $L > 0$ such that

$$|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2|, \quad (1.3)$$

whenever (t, y_1) and (t, y_2) are in D . The constant L is called a **Lipschitz constant** for f .

Example 1.5. Show that $f(t, y) = 3ty$ satisfies the Lipschitz condition on the region

$$D = \{(t, y) \mid 1 \leq t \leq 2, -3 \leq y \leq 4\}.$$

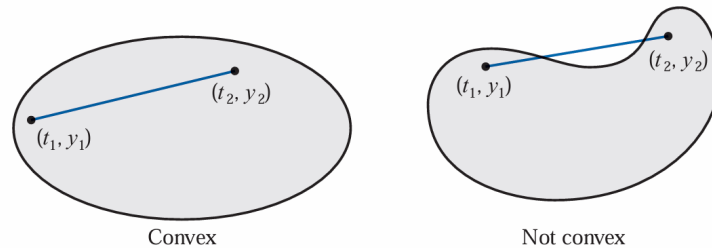
Solution: For any points (t, y_1) and (t, y_2) in D , we have

$$|f(t, y_1) - f(t, y_2)| = |3ty_1 - 3ty_2| = 3t|y_1 - y_2| \leq 3 \cdot 2 \cdot |y_1 - y_2| = 6|y_1 - y_2| \quad (\text{since } t \leq 2).$$

Hence, f satisfies the Lipschitz condition on D with Lipschitz constant $L = 6$.

Definition 1.2 (Convex Set). A set $D \subset \mathbb{R}^2$ is said to be **convex** if, whenever (t_1, y_1) and (t_2, y_2) belong to D , the entire line segment joining them lies in D . That is,

$$(1 - \lambda)(t_1, y_1) + \lambda(t_2, y_2) = ((1 - \lambda)t_1 + \lambda t_2, (1 - \lambda)y_1 + \lambda y_2) \in D, \quad \text{for all } \lambda \in [0, 1].$$



This means that if you pick any two points inside the set, the straight line connecting them stays entirely within the set.

Example 1.6. The rectangle defined by the region:

$$R = \{(x, y) \in \mathbb{R}^2 \mid 0 \leq x \leq 2, 0 \leq y \leq 2\}$$

is a convex set. Now consider a ring-shaped region (an annulus):

$$A = \{(x, y) \in \mathbb{R}^2 \mid 1 \leq x^2 + y^2 \leq 4\}.$$

This set is **not convex**.

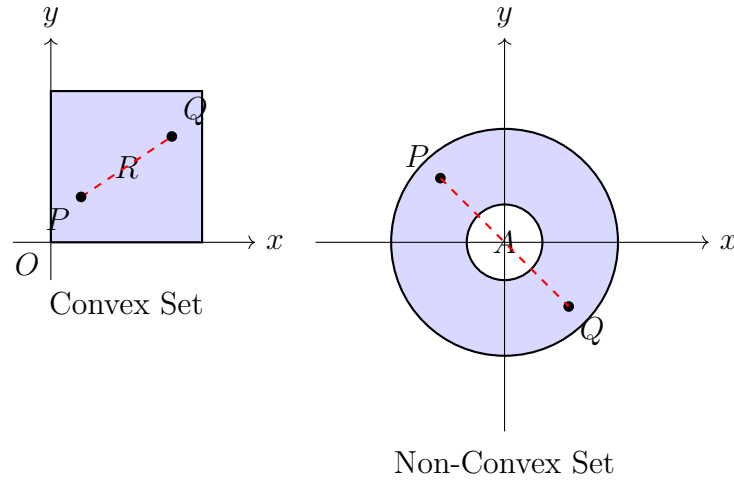


Figure 1.5: Comparison of a convex set R and a non-convex set A . The red dashed line segment connects two points in the set. In A , the segment passes through the hole (where $x^2 + y^2 < 1$), so it is not contained in the set. Hence, A is not convex.

Theorem 1.1.1 (Sufficient Condition for Lipschitz Condition). Suppose $f(t, y)$ is defined on a convex set $D \subset \mathbb{R}^2$. If there exists a constant L such that

$$\left| \frac{\partial f(t, y)}{\partial y} \right| \leq L \quad \text{for all } (t, y) \in D,$$

then f satisfies a Lipschitz condition in the variable y on D with Lipschitz constant L .

Existence and Uniqueness of Solutions

Theorem 1.1.2 (Existence and Uniqueness of Solution of First-Order IVPs). Suppose $D = \{(t, y) \mid a \leq t \leq b, -\infty < y < \infty\}$ and that $f(t, y)$ is continuous on D . If f satisfies a

Lipschitz condition in the variable y on D , then the initial value problem

$$\begin{cases} \frac{dy}{dt} = f(t, y), & a \leq t \leq b, \\ y(a) = \alpha, \end{cases} \quad (1.4)$$

has a unique solution $y(t)$ for all $t \in [a, b]$.

Example 1.7. Show that there exists a unique solution to the IVP

$$\begin{cases} \frac{dy}{dt} = 1 + t \sin(ty), & 0 \leq t \leq 2, \\ y(0) = 2. \end{cases} \quad (1.5)$$

Solution: In this example, $f(t, y) = 1 + t \sin(ty)$. We compute the partial derivative with respect to y :

$$\left| \frac{\partial f(t, y)}{\partial y} \right| = |t \cdot t \cos(ty)| = |t^2 \cos(ty)| \leq t^2 \cdot |\cos(ty)| \leq 2^2 \cdot 1 = 4, \quad (1.6)$$

since $t \in [0, 2]$, so $t^2 \leq 4$, and $|\cos(ty)| \leq 1$. Thus, $\left| \frac{\partial f}{\partial y} \right| \leq 4$ on the domain $[0, 2] \times \mathbb{R}$, which implies that f satisfies a Lipschitz condition in y with Lipschitz constant $L = 4$. By Theorem 1.1.2, the IVP (1.5) has a unique solution on $[0, 2]$.

Well-Posed Problems

Well-posedness: A mathematical problem is said to be **well-posed** if it satisfies the following three properties:

1. A solution exists,
2. The solution is unique,
3. The solution depends continuously on the data (e.g., initial conditions and forcing terms), meaning that small changes in the input lead to small changes in the solution.

Definition 1.3. The initial-value problem

$$\frac{dy}{dt} = f(t, y), \quad a \leq t \leq b, \quad y(a) = \alpha, \quad (1.7)$$

is said to be a **well-posed problem** if:

1. A unique solution $y(t)$ exists on $[a, b]$,
2. There exist constants $\varepsilon_0 > 0$ and $k > 0$ such that for any ε with $0 < \varepsilon < \varepsilon_0$, whenever $\delta(t)$ is continuous with $|\delta(t)| < \varepsilon$ for all $t \in [a, b]$, and $|\delta_0| < \varepsilon$, the perturbed initial-value problem

$$\frac{dz}{dt} = f(t, z) + \delta(t), \quad a \leq t \leq b, \quad z(a) = \alpha + \delta_0, \quad (1.8)$$

has a unique solution $z(t)$ satisfying

$$|z(t) - y(t)| < k\varepsilon \quad \text{for all } t \in [a, b].$$

The problem (1.8) is called the *perturbed problem* associated with the original problem (1.7). It models the effect of small errors in both the differential equation (via $\delta(t)$) and the initial condition (via δ_0).

Numerical methods will always be concerned with solving a perturbed problem because any round-off error introduced in the representation perturbs the original problem. Unless the original problem is well-posed, there is little reason to expect that the numerical solution to a perturbed problem will accurately approximate the solution to the original problem.

- Now that we know when an IVP has a solution and when it is unique, we can move to the third question: *Does the solution depend continuously on the data?*
- The well-posedness is important because IVPs obtained from physical observations generally only approximate the true situation, so we need to know whether small errors in the problem statement introduce correspondingly small changes in the solution.
- Another reason is round-off error when numerical methods are used.

Theorem 1.1.3 (Well-Posedness Theorem). *Suppose $D = \{(t, y) \mid a \leq t \leq b, -\infty < y < \infty\}$ and that $f(t, y)$ is continuous and satisfies a Lipschitz condition in y on D . Then the initial value problem*

$$\begin{cases} \frac{dy}{dt} = f(t, y), & a \leq t \leq b, \\ y(a) = \alpha, \end{cases}$$

is well-posed.

Example 1.8. *Show that the initial value problem $y' = y - t^2 + 1$, $0 \leq t \leq 2$, $y(0) = 0.5$ is well-posed on $D = \{(t, y) \mid 0 \leq t \leq 2, -\infty < y < \infty\}$.*

Solution: Here, $f(t, y) = y - t^2 + 1$. This function is continuous on D , and

$$\left| \frac{\partial f}{\partial y} \right| = \left| \frac{\partial}{\partial y}(y - t^2 + 1) \right| = 1.$$

Since the partial derivative is bounded, f satisfies a Lipschitz condition in y with Lipschitz constant $L = 1$. By Theorem 1.1.3, the IVP is well-posed.

As an illustration, consider the perturbed problem:

$$\frac{dz}{dt} = z - t^2 + 1 + \delta(t), \quad 0 \leq t \leq 2, \quad z(0) = 0.5 + \delta_0, \quad (1.9)$$

where $\delta(t)$ is a continuous function and δ_0 is a small constant. Suppose $|\delta(t)| < \varepsilon$ and $|\delta_0| < \varepsilon$ for some $\varepsilon > 0$. The exact solution to the original IVP is: $y(t) = (t + 1)^2 - 0.5e^t$.

The solution to the perturbed problem (1.9) (with constant $\delta(t) = \delta$) is:

$$z(t) = (t + 1)^2 + (\delta + \delta_0 - 0.5)e^t - \delta.$$

Then, for $t \in [0, 2]$,

$$\begin{aligned} |y(t) - z(t)| &= \left| [(t + 1)^2 - 0.5e^t] - [(t + 1)^2 + (\delta + \delta_0 - 0.5)e^t - \delta] \right| \\ &= \left| -0.5e^t - (\delta + \delta_0 - 0.5)e^t + \delta \right| \\ &= \left| -(\delta + \delta_0)e^t + \delta \right| \\ &\leq |\delta + \delta_0|e^t + |\delta| \\ &\leq (|\delta| + |\delta_0|)e^2 + |\delta| < (\varepsilon + \varepsilon)e^2 + \varepsilon = (2e^2 + 1)\varepsilon. \end{aligned}$$

Thus, $|z(t) - y(t)| < k\varepsilon$ with $k = 2e^2 + 1$, independent of t . This confirms that the problem is well-posed.

1.1.5 Review Exercise

1. Use Theorem 1.1.2 to show that each of the following initial-value problems has a unique solution, and find the solution.

(a) $y' = y \cos t, \quad 0 \leq t \leq 1, \quad y(0) = 1$

(b) $y' = \frac{2}{t}y + t^2e^t, \quad 1 \leq t \leq 2, \quad y(1) = 0$

(c) $y' = -\frac{2}{t}y + t^2e^t, \quad 1 \leq t \leq 2, \quad y(1) = \sqrt{2}e$

$$(d) \ y' = \frac{4t^3}{1+t^4}y, \quad 0 \leq t \leq 1, \quad y(0) = 1$$

2. Show that each of the following initial-value problems has a unique solution and find the solution. Can Theorem 1.1.2 be applied in each case?

$$(a) \ y' = e^{t-y}, \quad 0 \leq t \leq 1, \quad y(0) = 1$$

$$(b) \ y' = \frac{t-2(\sin 2t-2ty)}{3}, \quad 1 \leq t \leq 2, \quad y(1) = 2$$

$$(c) \ y' = -y + ty^{1/2}, \quad 2 \leq t \leq 3, \quad y(2) = 2$$

$$(d) \ y' = \frac{ty+y}{ty+t}, \quad 2 \leq t \leq 4, \quad y(2) = 4$$

3. For each choice of $f(t, y)$ given in parts (a)–(d):

$$(a) \text{ Does } f \text{ satisfy a Lipschitz condition on } D = \{(t, y) \mid 0 \leq t \leq 1, -\infty < y < \infty\}?$$

$$(b) \text{ Can Theorem 1.1.3 be used to show that the IVP } y' = f(t, y), \quad 0 \leq t \leq 1, \quad y(0) = 1 \text{ is well-posed?}$$

$$(a) \ f(t, y) = t^2y + 1$$

$$(c) \ f(t, y) = 1 - y$$

$$(b) \ f(t, y) = ty^3$$

$$(d) \ f(t, y) = -ty + \frac{4t}{y}$$

4. Suppose the perturbation $\delta(t)$ is proportional to t , that is, $\delta(t) = \delta t$ for some constant δ . Show directly that the following initial-value problems are well-posed.

$$(a) \ y' = 1 - y, \quad 0 \leq t \leq 2, \quad y(0) = 0$$

$$(b) \ y' = t + y, \quad 0 \leq t \leq 2, \quad y(0) = -1$$

$$(c) \ y' = \frac{2}{t}y + t^2e^t, \quad 1 \leq t \leq 2, \quad y(1) = 0$$

$$(d) \ y' = -\frac{2}{t}y + t^2e^t, \quad 1 \leq t \leq 2, \quad y(1) = \sqrt{2}e$$

1.2 Introduction to BVPs

While initial value problems (IVPs) model time evolution with data specified at a single point, **boundary value problems (BVPs)** describe equilibrium states where conditions are imposed at two distinct points typically the endpoints of an interval $[a, b]$.

A typical second-order BVP has the form:

$$y''(t) = f(t, y(t), y'(t)), \quad a < t < b, \tag{1.10}$$

subject to boundary conditions such as:

- Dirichlet: $y(a) = \alpha$, $y(b) = \beta$
- Neumann: $y'(a) = \gamma$, $y'(b) = \delta$
- Robin (mixed): $\alpha_1 y(a) + \alpha_2 y'(a) = \gamma$, $\beta_1 y(b) + \beta_2 y'(b) = \delta$

Unlike IVPs, where existence and uniqueness are typically guaranteed under mild conditions, BVPs exhibit more complex behavior: solutions may **not exist**, may be **non-unique**, or may depend sensitively on boundary data.

Physical problems that are position-dependent rather than time-dependent are often described by differential equations with conditions imposed at more than one point.

The two-point boundary-value problems in this course involve a second-order differential equation of the form:

$$y'' = f(x, y, y'), \quad a < x < b \quad (1.11)$$

together with the boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta. \quad (1.12)$$

The following theorem gives general conditions that ensure the existence and uniqueness of the solution to a second-order boundary value problem. The proof can be found in [?].

1.2.1 Existence and Uniqueness of Solutions of BVPs

Theorem 1.2.1 (Existence and uniqueness of solution). *Suppose the function f in the BVP (3.1)–(3.2) is continuous on the domain $D = \{(x, y, y') \mid a < x < b, -\infty < y < \infty, -\infty < y' < \infty\}$, and that the partial derivatives f_y and $f_{y'}$ are also continuous on D . If:*

1. $f_y(x, y, y') > 0$ for all $(x, y, y') \in D$,
2. *there exists a constant M such that $|f_{y'}(x, y, y')| \leq M$ for all $(x, y, y') \in D$,*

then the boundary-value problem has a unique solution.

Example 1.9. *Use Theorem 1.2.1 to show that the boundary-value problem*

$$y'' + e^{-xy} + \sin(y') = 0, \quad 1 < x < 2, \quad y(1) = y(2) = 0$$

has a unique solution.

Solution. Rewrite the differential equation in the standard form

$$y'' = f(x, y, y'),$$

where $f(x, y, y') = -e^{-xy} - \sin(y')$.

To apply Theorem 1.2.1, we verify its two key hypotheses on the domain $[1, 2] \times \mathbb{R} \times \mathbb{R}$:

1. **Monotonicity in y :** The partial derivative of f with respect to y is

$$f_y(x, y, y') = \frac{\partial}{\partial y}(-e^{-xy} - \sin(y')) = xe^{-xy}.$$

Since $x \in [1, 2]$ implies $x > 0$, and the exponential function is always positive, we have

$$f_y(x, y, y') = xe^{-xy} > 0 \quad \text{for all } (x, y, y').$$

2. **Boundedness of the derivative with respect to y' :** The partial derivative of f with respect to y' is

$$f_{y'}(x, y, y') = \frac{\partial}{\partial y'}(-e^{-xy} - \sin(y')) = -\cos(y').$$

Hence, $|f_{y'}(x, y, y')| = |\cos(y')| \leq 1$ for all (x, y, y') .

Both conditions of Theorem 1.2.1 are satisfied on the domain of interest. Therefore, the given boundary-value problem admits a unique solution on $[1, 2]$. $\square$

For Linear Boundary-Value Problems: The second-order differential equation

$$y'' = f(x, y, y')$$

is said to be *linear* if the function f can be expressed in the form

$$f(x, y, y') = p(x)y' + q(x)y + r(x),$$

where $p(x)$, $q(x)$, and $r(x)$ are given functions depending only on x . Linear boundary-value problems of this type arise frequently in applications. In this special case, the general uniqueness result (Theorem 1.2.1) reduces to the following simpler criterion.

Theorem 1.2.2. *Consider the linear boundary-value problem*

$$y'' = p(x) y' + q(x) y + r(x), \quad a < x < b, \quad (1.13)$$

subject to the boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta. \quad (1.14)$$

Assume that:

1. *The coefficient functions $p(x)$, $q(x)$, and $r(x)$ are continuous on the closed interval $[a, b]$;*
2. *The coefficient $q(x)$ is strictly positive on $[a, b]$, i.e., $q(x) > 0$ for all $x \in [a, b]$.*

Then the boundary-value problem (3.3)–(3.4) has a unique solution on $[a, b]$.

Example 1.10. *The boundary value problem $y'' = 2y' + 3y + \sin x$, $0 < x < \pi$, with boundary conditions $y(0) = 0$, $y(\pi) = 1$. satisfies the Theorem.*

Here, $p(x) = 2$, $q(x) = 3$, and $r(x) = \sin x$. All are continuous on $[0, \pi]$, and $q(x) = 3 > 0$ on $[0, \pi]$. Hence, the hypotheses of Theorem 1.2.2 are satisfied, and a unique solution exists.

Example 1.11. *The boundary value problem $y'' = x y' + (1 + x^2) y + e^x$, $-1 < x < 1$, with $y(-1) = 2$, $y(1) = -1$.*

We have $p(x) = x$, $q(x) = 1 + x^2$, and $r(x) = e^x$, all continuous on $[-1, 1]$. Moreover, $q(x) \geq 1 > 0$ for all $x \in [-1, 1]$. Thus, the BVP satisfies the conditions of Theorem 1.2.2 and admits a unique solution.

Example 1.12. *The boundary value problem $y'' = \cos(x) y' + (2 + \sin x) y + x^2$, $0 < x < \frac{\pi}{2}$, with $y(0) = 1$, $y(\frac{\pi}{2}) = 0$.*

The functions $p(x) = \cos x$, $q(x) = 2 + \sin x$, and $r(x) = x^2$ are continuous on $[0, \frac{\pi}{2}]$, and $q(x) \geq 2 > 0$ on this interval. Therefore, the problem satisfies the theorem and has a unique solution.

Example 1.13 (Example that does not satisfy the Theorem). *Consider the boundary value problem $y'' = -\pi^2 y$, $0 < x < 1$, with $y(0) = 0$, $y(1) = 0$. Here, $p(x) = 0$, $q(x) = -\pi^2 < 0$, and $r(x) = 0$. While all coefficient functions are continuous on $[0, 1]$, the condition $q(x) > 0$ is violated. Indeed, this BVP has infinitely many solutions of the form $y(x) = C \sin(\pi x)$ for any constant C , so the solution is not unique. This shows that the positivity condition on $q(x)$ in Theorem 1.2.2 is essential.*

1.3 Introduction to PDEs

A **partial differential equation (PDE)** is a differential equation that involves partial derivatives of one or more dependent variables with respect to two or more independent variables. PDEs are fundamental in modeling physical phenomena such as heat conduction, wave propagation, fluid dynamics, and electrostatics.

Consider the following second-order PDEs in two independent variables x and y , with dependent variable $u = u(x, y)$:

$$u_{xx} + u_{yy} = 0 \quad (1.15)$$

$$u_{xx} + u_{yy} = f(x, y) \quad (1.16)$$

$$u_{xx} - u_{yy} = 0 \quad (1.17)$$

$$a^2 u_{xx} - u_{tt} = 0 \quad (1.18)$$

$$u_{xx} + (1 + u_x^2)u_{yy} = 0 \quad (1.19)$$

$$xu_{xx} + 2xyuu_{xy} + yu_{yy} = 0 \quad (1.20)$$

Order of a PDE

The **order** of a PDE is defined as the order of the highest-order partial derivative present in the equation.

All equations (1.15)–(1.20) involve second derivatives (e.g., u_{xx} , u_{yy} , u_{xy}), so they are all **second-order PDEs**.

Classification by Linearity

A second-order PDE can be classified more precisely based on how the unknown function u and its derivatives appear. The general form is:

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = G, \quad (1.21)$$

where the coefficients A, B, C, D, E, F, G may depend on x, y, u, u_x , and u_y . The nature of this dependence determines the type of nonlinearity.

1. **Linear:** The PDE is **linear** if A, B, C, D, E, F, G depend *only on the independent variables* x and y , and u and its derivatives appear linearly.

Examples: (1.15)–(1.18) are linear. Equation (1.15) is the Laplace equation; (1.16) is Poisson’s equation; (1.17) and (1.18) are wave-type equations.

2. **Semi-linear:** The PDE is **semi-linear** if the coefficients of the highest-order derivatives (A, B, C) depend only on x and y , but the lower-order terms may be nonlinear functions of u , u_x , or u_y .

Example: $u_{xx} + u_{yy} = \sin(u)$ is semi-linear (not in the list above, but illustrative).

3. **Quasi-linear:** The PDE is **quasi-linear** if the highest-order derivatives appear linearly, but their coefficients may depend on x , y , u , u_x , or u_y .

Example: Equation (1.19),

$$u_{xx} + (1 + u_x^2)u_{yy} = 0, \quad (1.22)$$

is quasi-linear because u_{xx} and u_{yy} appear linearly, but the coefficient of u_{yy} depends on the first derivative u_x .

4. **Fully nonlinear:** The PDE is **fully nonlinear** if the highest-order derivatives appear in a nonlinear way (e.g., products, powers, or other nonlinear combinations).

Example: The Monge–Ampère equation $u_{xx}u_{yy} - (u_{xy})^2 = f(x, y)$ is fully nonlinear. Note that equation (1.20),

$$xu_{xx} + 2xyuu_{xy} + yu_{yy} = 0, \quad (1.23)$$

is **quasi-linear**, not fully nonlinear, because each second-order derivative appears linearly (even though the coefficient of u_{xy} depends on u).

Thus, the hierarchy of PDE types is:

$$\text{Linear} \subset \text{Semi-linear} \subset \text{Quasi-linear} \subset \text{Fully nonlinear}.$$

Homogeneous vs Non-homogeneous PDEs

A PDE is **homogeneous** if every term in the equation contains the dependent variable u or one of its derivatives. Otherwise, it is **non-homogeneous**.

From the list: Equations (1.15), (1.17), (1.18), (1.19), (1.20) are **homogeneous**. Equation (1.16) ($u_{xx} + u_{yy} = f(x, y)$) is **non-homogeneous** due to the source term $f(x, y)$.

Notation

It is common to use subscript notation for partial derivatives:

$$u_x = \frac{\partial u}{\partial x}, \quad u_{xx} = \frac{\partial^2 u}{\partial x^2}, \quad u_{xy} = \frac{\partial^2 u}{\partial x \partial y}, \text{ etc.}$$

For example, the Laplace equation in three dimensions can be written as:

$$u_{xx} + u_{yy} + u_{zz} = 0 \quad (1.24)$$

The Poisson equation is:

$$u_{xx} + u_{yy} + u_{zz} = f(x, y, z) \quad (1.25)$$

1.3.1 Classification of Second-Order PDEs

Consider a general second-order PDE in two independent variables x and y :

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = G \quad (1.26)$$

where A, B, C, D, E, F, G are functions of x and y only (making the equation at most *quasi-linear*). The nature of the PDE is determined by the discriminant:

$$\Delta = B^2 - 4AC$$

We classify the PDE as follows:

- **Elliptic** if $\Delta < 0$: e.g., Laplace and Poisson equations,
- **Parabolic** if $\Delta = 0$: e.g., Heat equation,
- **Hyperbolic** if $\Delta > 0$: e.g., Wave equation.

1.3.2 Examples of Classification

1. Laplace Equation (Elliptic)

The Laplace equation for steady-state temperature distribution in a plane:

$$u_{xx} + u_{yy} = 0$$

Comparing with (1.26): $A = 1$, $B = 0$, $C = 1$. Then:

$$\Delta = 0^2 - 4(1)(1) = -4 < 0$$

Hence, the equation is **elliptic**.

2. Heat Conduction Equation (Parabolic)

The one-dimensional heat equation:

$$u_t = ku_{xx} \quad \Rightarrow \quad ku_{xx} - u_t = 0$$

In variables x and t , rewrite as:

$$ku_{xx} + 0 \cdot u_{xt} + 0 \cdot u_{tt} - u_t = 0$$

So $A = k$, $B = 0$, $C = 0$. Then:

$$\Delta = 0^2 - 4(k)(0) = 0$$

Hence, the equation is **parabolic**.

3. Wave Equation (Hyperbolic)

The one-dimensional wave equation:

$$u_{tt} = c^2 u_{xx} \quad \Rightarrow \quad c^2 u_{xx} - u_{tt} = 0$$

So $A = c^2$, $B = 0$, $C = -1$. Then:

$$\Delta = 0^2 - 4(c^2)(-1) = 4c^2 > 0 \quad (\text{for } c \neq 0)$$

Hence, the equation is **hyperbolic**.

Example 1.14. Consider the PDE: $xu_{xx} + 2xyu_{xy} + yu_{yy} = 0$. For what values of x and y is this PDE: Elliptic?, Parabolic?, Hyperbolic?

Solution 1.4. : Compare with the general form (1.26): $A = x$, $B = 2xy$, $C = y$. Discrimi-

nant:

$$\Delta = B^2 - 4AC = (2xy)^2 - 4(x)(y) = 4x^2y^2 - 4xy = 4xy(xy - 1)$$

So:

Elliptic when $\Delta < 0$:

$$4xy(xy - 1) < 0 \Rightarrow xy(xy - 1) < 0$$

This occurs when $0 < xy < 1$.

Parabolic when $\Delta = 0$:

$$4xy(xy - 1) = 0 \Rightarrow xy = 0 \quad \text{or} \quad xy = 1$$

Hyperbolic when $\Delta > 0$:

$$4xy(xy - 1) > 0 \Rightarrow xy < 0 \quad \text{or} \quad xy > 1$$

Summary:

Type	Region in xy-plane
<i>Elliptic</i>	$0 < xy < 1$
<i>Parabolic</i>	$xy = 0$ or $xy = 1$
<i>Hyperbolic</i>	$xy < 0$ or $xy > 1$

Exercise 1.1. In which part of the x - y plane is the following PDE elliptic?

$$u_{xx} + 2xu_{xy} + (1 - y^2)u_{yy} = 0$$

Chapter 2

Numerical Methods for IVPs

2.1 Numerical Methods for First Order IVPs

Several numerical methods exist to approximate the solution of an IVP. Some common methods include: Euler Method, Runge-Kutta Methods, and so on.

2.1.1 Euler's Method

In this sub-section, we introduce **Euler's method**, the most elementary numerical technique for solving initial-value problems (IVPs). Although it is seldom used in practice due to its low accuracy, its simplicity makes it an excellent tool for illustrating the principles behind more advanced methods.

The objective is to approximate the solution to the well-posed initial-value problem:

$$\begin{cases} \frac{dy}{dt} = f(t, y), & a \leq t \leq b, \\ y(a) = \alpha. \end{cases} \quad (2.1)$$

We do not obtain a continuous approximation to $y(t)$; instead, we compute approximations at discrete points $t_0, t_1, \dots, t_N$ in the interval $[a, b]$, called **grid points**.

Divide $[a, b]$ into N equal subintervals with step size: $h = \frac{b-a}{N}$.

The grid points are:

$$t_i = a + ih, \quad i = 0, 1, 2, \dots, N. \quad (2.2)$$

Assume the true solution $y(t)$ has two continuous derivatives on $[a, b]$. By Taylor's The-

orem, for each $i = 0, 1, \dots, N - 1$, there exists $\zeta_i \in (t_i, t_{i+1})$ such that:

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\zeta_i).$$

Since $y'(t_i) = f(t_i, y(t_i))$, we have:

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + T_i,$$

where $T_i = \frac{h^2}{2}y''(\zeta_i)$ is the **local truncation error**, which is $O(h^2)$.

Euler's method constructs an approximation $w_i \approx y(t_i)$ by discarding the error term. The recurrence is:

$$\begin{cases} w_0 = \alpha, \\ w_{i+1} = w_i + hf(t_i, w_i), \quad i = 0, 1, \dots, N - 1. \end{cases} \quad (2.3)$$

Algorithm 1 Explicit Euler Method

Require: Function $f(t, y)$, interval endpoints a and b , number of steps N , initial con. α .

Ensure: Approximate values $w \approx y(t)$ at $N + 1$ equally spaced points in $[a, b]$.

1: Set $h = \frac{b - a}{N}$;

2: Set $t = a$;

3: Set $w = \alpha$;

4: OUTPUT (t_i, w_i)

5: **for** $i = 1, 2, \dots, N$ **do**

6: Compute:

$$w_{i+1} \leftarrow w_i + h \cdot f(t_i, w_i)$$

 {Euler update: approximate $y(t_i + h)$ }

7: Set $t_{i+1} \leftarrow a + i \cdot h$ {Update current time to t_i }

8: OUTPUT (t_i, w_i)

9: **end for**

10: **STOP**

Example 2.1. Use Euler's method with $h = 0.2$ to solve the IVP:

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5.$$

The exact solution is: $y(t) = (t + 1)^2 - 0.5e^t$.

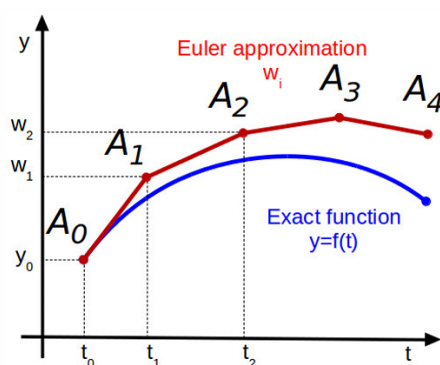


Figure 2.1: Geometric interpretation of Euler's method: the slope at each point is used to step forward.

Solution: Here, $f(t, y) = y - t^2 + 1$, $a = 0$, $b = 2$, $N = \frac{2-0}{0.2} = 10$.

We compute w_{i+1} iteratively from $w_{i+1} = w_i + hf(t_i, w_i)$ as follows:

$$\begin{aligned}
 t_0 = 0.0 : \quad w_0 &= 0.5 \\
 t_1 = 0.2 : \quad w_1 &= 0.5 + 0.2(0.5 - 0^2 + 1) = 0.5 + 0.2(1.5) = 0.8 \\
 t_2 = 0.4 : \quad w_2 &= 0.8 + 0.2(0.8 - 0.04 + 1) = 0.8 + 0.2(1.76) = 1.152 \\
 t_3 = 0.6 : \quad w_3 &= 1.152 + 0.2(1.152 - 0.16 + 1) = 1.5504 \\
 t_4 = 0.8 : \quad w_4 &= 1.5504 + 0.2(1.5504 - 0.36 + 1) = 1.98848 \\
 t_5 = 1.0 : \quad w_5 &= 1.98848 + 0.2(1.98848 - 0.64 + 1) = 2.458176 \\
 t_6 = 1.2 : \quad w_6 &= 2.458176 + 0.2(2.458176 - 1.00 + 1) = 2.9498112 \\
 t_7 = 1.4 : \quad w_7 &= 2.9498112 + 0.2(2.9498112 - 1.44 + 1) = 3.45177344 \\
 t_8 = 1.6 : \quad w_8 &= 3.45177344 + 0.2(3.45177344 - 1.96 + 1) = 4.069128128 \\
 t_9 = 1.8 : \quad w_9 &= 4.069128128 + 0.2(4.069128128 - 2.56 + 1) = 4.7629537536 \\
 t_{10} = 2.0 : \quad w_{10} &= 4.7629537536 + 0.2(4.7629537536 - 3.24 + 1) = \boxed{5.26754450432}
 \end{aligned}$$

So, the Euler approximation at $t = 2$ is: $w_{10} \approx 5.2675$. The exact value is:

$$y(2) = (2 + 1)^2 - 0.5e^2 = 9 - 0.5 \cdot 7.389 \approx 9 - 3.6945 = 5.3055$$

Thus, the absolute error is: $|y(2) - w_{10}| \approx |5.3055 - 5.2675| = 0.038$.

Euler's method has: **Local truncation error:** $O(h^2)$, **Global truncation error:** $O(h)$ to

be proved later.

This means that halving h approximately halves the error. While simple, Euler's method is only first-order accurate and is often replaced by higher-order methods like Runge-Kutta.

The MATLAB code for the Euler method is given below:

Euler Method Matlab Implementation

```
function [t,y] = eul(fun,a,b,y0,N)
% Euler's method for solving the IVP:  y' = f(t,y) in [a,b],  y(a) = y0
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
t(i+1) = t(i) + h;
y(i+1) = y(i) + h*fun(t(i),y(i));
end
```

Using the example above, we apply Euler's method. The driver code are given below.

```
%% ex_5_1_1 from Numerical Analysis, Richard Burden
clc
fun = @(t,y) y-t^2+1; % function f(t,y)
y0 = 0.5; % initial conditino
a = 0; % starting time
b = 2; % ending time
N = 10; % number of subintervals, h = (b-a)/N
[t,y] = eul(fun,a,b,y0,N);
%% Display Solution
exactY = @(t) (t+1).^2-0.5*exp(t);
Y = exactY(t);
disp('-----');
disp(['          Euler Method    h = ', num2str((b-a)/N)]);
disp('-----');
disp('t_i          y_i          y(t_i)      |y_i-y(t_i)|')
disp('-----')
formatSpec = '%.4f      % .4f      % .4f      % .4f\n';
fprintf(formatSpec,[t';y';Y';(abs(y-Y))'])
%% Plot Solution
plot(t, Y, 'r*- ', t, y, 'b*- ', 'LineWidth', 2); hold off
legend('Exact Solution','Euler Solution','Location','northwest'); grid on
```

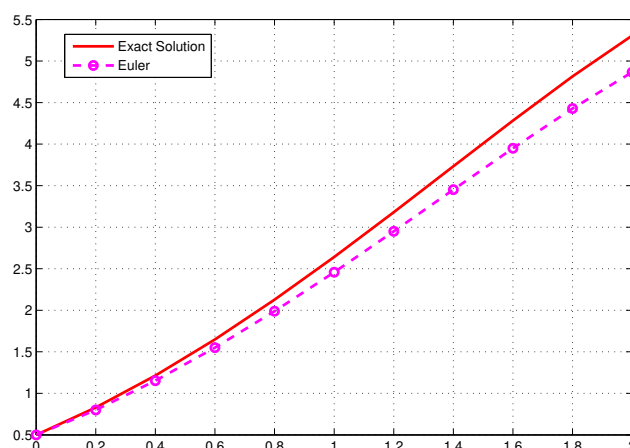
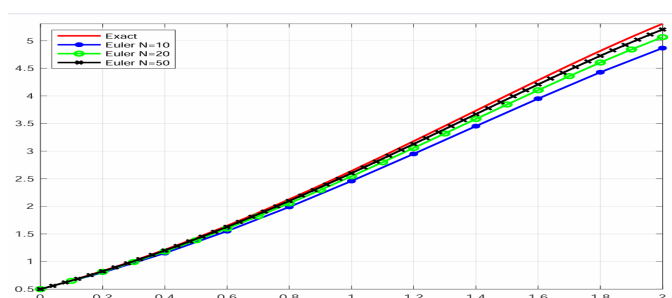
Figure 2.2: Plot of the Euler's method approximation and exact solution for $h = 0.2$.

Table 2.1: Comparison of Euler method approximation and exact solution

t_i	$y(t_i)$	Euler (y_i)	Error
0.0	0.50000	0.50000	0.00000e+00
0.2	0.82930	0.80000	2.92986e-02
0.4	1.21409	1.15200	6.20877e-02
0.6	1.64894	1.55040	9.85406e-02
0.8	2.12723	1.98848	1.38750e-01
1.0	2.64086	2.45818	1.82683e-01
1.2	3.17994	2.94981	2.30130e-01
1.4	3.73240	3.45177	2.80627e-01
1.6	4.28348	3.95013	3.33356e-01
1.8	4.81518	4.42815	3.87023e-01
2.0	5.30547	4.86578	4.39687e-01

Figure 2.3: Plot of Euler's method solution and exact solution for different h values.

The approximations become more accurate as the step size h gets smaller.

Error Bounds for Euler's Method

Although Euler's method is not accurate enough for practical use, it is sufficiently elementary to allow a clear analysis of the error it produces. The more accurate methods discussed later follow similar principles but involve greater complexity.

To derive an error bound for Euler's method, we first establish two computational lemmas.

Lemma 2.1. *For all $x \geq -1$ and any positive integer m , $0 \leq (1+x)^m \leq e^{mx}$.*

Proof. By Taylor's Theorem applied to $f(x) = e^x$ with $x_0 = 0$ and $n = 1$,

$$e^x = 1 + x + \frac{1}{2}x^2e^\xi,$$

where ξ lies between 0 and x . Since $\frac{1}{2}x^2e^\xi \geq 0$, we have $1 + x \leq e^x$.

Given $x \geq -1$, we also have $1 + x \geq 0$. Raising both sides to the power m preserves the inequality:

$$(1+x)^m \leq (e^x)^m = e^{mx}.$$

The left inequality $(1+x)^m \geq 0$ follows from $1+x \geq 0$. This completes the proof. $\square$

Lemma 2.2. *Let $s > 0$, $t > 0$, and let $\{a_i\}_{i=0}^k$ be a sequence such that $a_0 \geq -\frac{t}{s}$, and $a_{i+1} \leq (1+s)a_i + t$, for each $i = 0, 1, \dots, k-1$. Then*

$$a_{i+1} \leq e^{(i+1)s} \left(a_0 + \frac{t}{s} \right) - \frac{t}{s}, \text{ for } i = 0, 1, \dots, k-1.$$

Proof. We proceed by unwinding the recurrence:

$$\begin{aligned} a_{i+1} &\leq (1+s)a_i + t \\ &\leq (1+s)[(1+s)a_{i-1} + t] + t = (1+s)^2a_{i-1} + [1 + (1+s)]t \\ &\leq (1+s)^3a_{i-2} + [1 + (1+s) + (1+s)^2]t \\ &\vdots \\ &\leq (1+s)^{i+1}a_0 + \left[\sum_{j=0}^i (1+s)^j \right] t. \end{aligned}$$

The sum is a geometric series: $\sum_{j=0}^i (1+s)^j = \frac{(1+s)^{i+1} - 1}{s}$.

Thus,

$$a_{i+1} \leq (1+s)^{i+1}a_0 + \frac{(1+s)^{i+1} - 1}{s}t = (1+s)^{i+1} \left(a_0 + \frac{t}{s} \right) - \frac{t}{s}.$$

Applying Lemma 2.1 with $x = s$, we get $(1 + s)^{i+1} \leq e^{(i+1)s}$. Therefore,

$$a_{i+1} \leq e^{(i+1)s} \left(a_0 + \frac{t}{s} \right) - \frac{t}{s}.$$

This completes the proof. $\square$

Theorem 2.1.1 (Global Error Bound for Euler's Method). *Suppose f is continuous and satisfies a Lipschitz condition in y with constant L on the strip $D = \{(t, y) \mid a \leq t \leq b, -\infty < y < \infty\}$. Let $y(t)$ be the unique solution to the IVP $y' = f(t, y)$, $y(a) = \alpha$, $a \leq t \leq b$. Assume $y \in C^2[a, b]$, and suppose there exists a constant M such that*

$$|y''(t)| \leq M, \quad \text{for all } t \in [a, b].$$

. Let $w_0, w_1, \dots, w_N$ be the approximations generated by Euler's method with step size $h = \frac{b-a}{N}$. Then, for each $i = 0, 1, \dots, N$,

$$|y(t_i) - w_i| \leq \frac{hM}{2L} (e^{L(t_i-a)} - 1).$$

Proof. For $i = 0$, $t_0 = a$, and $w_0 = \alpha = y(a)$, so $|y(t_0) - w_0| = 0$, and the bound holds trivially.

From Taylor's Theorem, for each $i = 0, 1, \dots, N-1$, there exists $\xi_i \in (t_i, t_{i+1})$ such that

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2} y''(\xi_i).$$

Euler's method gives: $w_{i+1} = w_i + hf(t_i, w_i)$. Let $e_i = y(t_i) - w_i$ denote the global error. Subtracting the two equations:

$$e_{i+1} = e_i + h[f(t_i, y(t_i)) - f(t_i, w_i)] + \frac{h^2}{2} y''(\xi_i).$$

Taking absolute values and using the Lipschitz condition $|f(t, y_i) - f(t, w_i)| \leq L|e_i|$, and $|y''(\xi_i)| \leq M$, we get:

$$|e_{i+1}| \leq |e_i| + hL|e_i| + \frac{h^2M}{2} = (1 + hL)|e_i| + \frac{h^2M}{2}.$$

This recurrence fits the form in Lemma 2.2 with: $s = hL$, $t = \frac{h^2M}{2}$, $a_i = |e_i|$.

Note that $a_0 = |e_0| = 0 \geq -\frac{t}{s} = -\frac{h^2 M/2}{hL} = -\frac{hM}{2L}$, so the initial condition is satisfied.

Applying Lemma 2.2:

$$|e_{i+1}| \leq e^{(i+1)hL} \left(0 + \frac{h^2 M/2}{hL} \right) - \frac{h^2 M/2}{hL} = \frac{hM}{2L} (e^{(i+1)hL} - 1).$$

Since $t_{i+1} = a + (i+1)h$, we have $(i+1)h = t_{i+1} - a$. Thus,

$$|y(t_{i+1}) - w_{i+1}| \leq \frac{hM}{2L} (e^{L(t_{i+1}-a)} - 1).$$

This holds for $i = 0, 1, \dots, N-1$. Reindexing, we conclude that for all $i = 0, 1, \dots, N$,

$$|y(t_i) - w_i| \leq \frac{hM}{2L} (e^{L(t_i-a)} - 1).$$

This completes the proof. □

The weakness of Theorem 2.1.1 lies in the requirement that a bound M on $|y''(t)|$ be known. In practice, this is often not available since $y(t)$ is unknown. However, if $f \in C^1(D)$, then by the chain rule:

$$y''(t) = \frac{d}{dt}f(t, y(t)) = \frac{\partial f}{\partial t}(t, y(t)) + \frac{\partial f}{\partial y}(t, y(t)) \cdot f(t, y(t)).$$

Thus, we can sometimes bound $|y''(t)|$ using only f and its partial derivatives, without explicitly knowing $y(t)$. For example, if f , f_t , and f_y are bounded on D , then $y''(t)$ is bounded, and an error estimate can be derived.

This global error bound confirms that Euler's method is **first-order accurate**: the error is $O(h)$ as $h \rightarrow 0$. This motivates the development of higher-order methods, such as the Taylor's methods, the Runge–Kutta methods, which achieve better accuracy with comparable computational effort.

Example 2.2. *The solution to the initial-value problem*

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5,$$

was approximated in Example 2.1 using Euler's method with $h = 0.2$. Use the inequality in Theorem 2.1.1 to find bounds for the approximation errors and compare these to the actual errors.

Solution 2.1. Because $f(t, y) = y - t^2 + 1$, we have $\frac{\partial f}{\partial y} = 1$ for all y , so the Lipschitz constant is $L = 1$. For this problem, the exact solution is $y(t) = (t + 1)^2 - 0.5e^t$, so $y'(t) = 2(t + 1) - 0.5e^t$, and

$$|y''(t)| \leq M = 0.5e^2 - 2, \quad \text{for all } t \in [0, 2].$$

Using the error bound from Theorem 2.1.1 with $h = 0.2$, $L = 1$, and $M = 0.5e^2 - 2$, we obtain:

$$|y(t_i) - w_i| \leq \frac{hM}{2L} (e^{L(t_i-a)} - 1) = 0.1(0.5e^2 - 2)(e^{t_i} - 1).$$

Thus,

$$|y(0.2) - w_1| \leq 0.1(0.5e^2 - 2)(e^{0.2} - 1) = 0.03752,$$

$$|y(0.4) - w_2| \leq 0.1(0.5e^2 - 2)(e^{0.4} - 1) = 0.08334,$$

and so on.

Table 2.2 lists the actual error found in Example 2.2 together with this error bound. Note that even though the true bound for the second derivative of the solution was used, the error bound is considerably larger than the actual error, especially for increasing values of t .

Table 2.2: Comparison of Actual Errors and Error Bounds for Euler's Method

t_i	Actual Error	Error Bound
0.0	0.00000	0.00000
0.2	0.02930	0.03752
0.4	0.06209	0.08334
0.6	0.09854	0.13931
0.8	0.13875	0.20767
1.0	0.18268	0.29117
1.2	0.23013	0.39315
1.4	0.28063	0.51771
1.6	0.33336	0.66985
1.8	0.38702	0.85568
2.0	0.43969	1.08264

The principal importance of the error-bound formula given in Theorem 2.1.1 is that the bound depends linearly on the step size h . Consequently, diminishing the step size should give correspondingly greater accuracy to the approximations.

Exercise 2.1 (Lab work). Use Euler's method to approximate the solutions for each of the following IVPs. Modify the above codes for each cases.

a. $y' = \frac{2-2ty}{t^2+1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \frac{2t+1}{t^2+1}$

b. $y' = \frac{y^2}{1+t}$, $1 \leq t \leq 2$, $y(1) = -(\ln 2)^{-1}$, with $h = 0.1$. the exact solution is $y(t) = -\frac{1}{\ln(1+t)}$

c. $y' = \frac{y^2+y}{t}$, $1 \leq t \leq 3$, $y(1) = -2$, with $h = 0.2$. the exact solution is $y(t) = \frac{2t}{1-2t}$

d. $y' = -ty + 4ty^{-1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \sqrt{4 - 3e^{-t^2}}$

2.1.2 Taylor's Methods

Consider the initial-value problem (IVP)

$$\begin{cases} y'(t) = f(t, y(t)), & a \leq t \leq b, \\ y(a) = y_0, \end{cases} \quad (2.4)$$

where $f : [a, b] \times \mathbb{R} \rightarrow \mathbb{R}$ is assumed sufficiently smooth (e.g., $f \in C^{n+1}$).

Let $y(t)$ be the exact solution of (2.4). By Taylor's theorem with Lagrange remainder, for any $t_i \in [a, b]$ and step size $h > 0$ such that $t_{i+1} = t_i + h \in [a, b]$, we have

$$y(t_{i+1}) = \sum_{k=0}^n \frac{h^k}{k!} y^{(k)}(t_i) + R_{n+1}(t_i), \quad (2.5)$$

where the remainder term is

$$R_{n+1}(t_i) = \frac{h^{n+1}}{(n+1)!} y^{(n+1)}(\xi_i), \quad \text{for some } \xi_i \in (t_i, t_{i+1}). \quad (2.6)$$

To construct a numerical method, we must express the derivatives $y^{(k)}(t_i)$ in terms of f and its partial derivatives. From (2.4), the first derivative is

$$y'(t) = f(t, y(t)). \quad (2.7)$$

Differentiating with respect to t and applying the chain rule yields

$$\begin{aligned} y''(t) &= \frac{d}{dt}f(t, y(t)) = \frac{\partial f}{\partial t}(t, y(t)) + \frac{\partial f}{\partial y}(t, y(t))y'(t) \\ &= f_t(t, y(t)) + f_y(t, y(t))f(t, y(t)). \end{aligned} \quad (2.8)$$

This motivates the definition of the *total derivative operator* (or *Lie derivative* along the flow):

Definition 2.1 (Total Derivative Operator). *Define the operator $\mathcal{D}$ acting on sufficiently smooth functions $g(t, y)$ by*

$$\mathcal{D}g := \frac{\partial g}{\partial t} + y' \frac{\partial g}{\partial y}. \quad (2.9)$$

Using $\mathcal{D}$, higher-order time derivatives of y can be written compactly:

Theorem 2.1.2. *For $k \geq 1$, the k -th derivative of the solution satisfies*

$$y^{(k)}(t) = \mathcal{D}^{k-1}f(t, y(t)). \quad (2.10)$$

Proof. By induction. The case $k = 1$ holds since $y' = f = \mathcal{D}^0 f$. Assume (2.10) holds for some k . Then

$$y^{(k+1)}(t) = \frac{d}{dt} \left[\mathcal{D}^{k-1}f(t, y(t)) \right] = \mathcal{D}(\mathcal{D}^{k-1}f)(t, y(t)) = \mathcal{D}^k f(t, y(t)).$$

as required. □

Define successive total derivatives of f :

Definition 2.2. *Let $f^{[k]}(t, y)$ denote the k -th total derivative of f :*

$$f^{[0]}(t, y) := f(t, y), \quad (2.11)$$

$$f^{[k]}(t, y) := \mathcal{D}^k f(t, y), \quad k = 1, 2, \dots \quad (2.12)$$

Then $y^{(k+1)}(t) = f^{[k]}(t, y(t))$.

Substituting (2.10) into (2.5) gives

$$y(t_{i+1}) = y(t_i) + \sum_{k=1}^n \frac{h^k}{k!} f^{[k-1]}(t_i, y(t_i)) + \frac{h^{n+1}}{(n+1)!} f^{[n]}(\xi_i, y(\xi_i)). \quad (2.13)$$

A numerical method is obtained by:

- Approximating $y(t_i)$ by a computable value y_i ,
- Dropping the remainder term.

Definition 2.3 (Taylor's Method of Order n). *Given $t_0 = a$, y_0 , and step size $h = (b-a)/N$, compute for $i = 0, 1, \dots, N-1$:*

$$y_{i+1} = y_i + \sum_{k=1}^n \frac{h^k}{k!} f^{[k-1]}(t_i, y_i) \quad (2.14)$$

with $t_{i+1} = t_i + h$.

Explicitly, the first few orders are:

Order 1 (Euler's method):

$$y_{i+1} = y_i + h f(t_i, y_i).$$

Order 2:

$$y_{i+1} = y_i + h f(t_i, y_i) + \frac{h^2}{2} [f_t(t_i, y_i) + f_y(t_i, y_i) f(t_i, y_i)].$$

Order 3:

$$y_{i+1} = y_i + h f + \frac{h^2}{2} (f_t + f_y f) + \frac{h^3}{6} [f_{tt} + 2f_{ty} f + f_{yy} f^2 + f_y (f_t + f_y f)],$$

Order 4:

$$\begin{aligned} y_{i+1} = y_i + h f + \frac{h^2}{2} (f_t + f_y f) + \frac{h^3}{6} (f_{tt} + 2f_{ty} f + f_{yy} f^2 + f_y f_t + f_y^2 f) \\ + \frac{h^4}{24} [f_{ttt} + 3f_{tty} f + 3f_{tyy} f^2 + f_{yyy} f^3 \\ + 3f_{tt} f_y + 3f_{ty} (2f_t + 3f_y f) + f_{yy} (3f_t f + 3f_y f^2) \\ + f_t (f_{ty} + f_{yy} f + f_y^2) + f_y^2 (f_t + f_y f) + f_y (f_{tt} + 2f_{ty} f + f_{yy} f^2)], \end{aligned}$$

where all derivatives are evaluated at (t_i, y_i) .

Theorem 2.1.3 (Local Truncation Error). *The local truncation error (LTE) of Taylor's method of order n satisfies*

$$\text{LTE}_{i+1} := y(t_{i+1}) - \left(y(t_i) + \sum_{k=1}^n \frac{h^k}{k!} f^{[k-1]}(t_i, y(t_i)) \right) = \frac{h^{n+1}}{(n+1)!} f^{[n]}(\xi_i, y(\xi_i)) = \mathcal{O}(h^{n+1}), \quad (2.15)$$

provided $f^{[n]}$ is bounded on a neighborhood of the solution.

Theorem 2.1.4 (Global Truncation Error). Assume f is Lipschitz continuous in y and $f^{[k]}$ are bounded for $k = 0, \dots, n-1$. Then the global error satisfies

$$\max_{0 \leq i \leq N} |y(t_i) - y_i| = \mathcal{O}(h^n), \quad (2.16)$$

i.e., Taylor's method of order n is convergent of order n .

Remark 2.1. Euler's method corresponds to $n = 1$: $LTE = \mathcal{O}(h^2)$, $GTE = \mathcal{O}(h)$.

Example 2.3. Apply Taylor's methods of order 1, 2 and 4 to solve the IVP:

$$y' = f(t, y) = y - t^2 + 1, \quad y(0) = \frac{1}{2}, \quad 0 \leq t \leq 2. \quad (2.17)$$

Solution: Compute required derivatives. Let $f(t, y) = y - t^2 + 1$. Using the total derivative operator $D = \partial_t + f \partial_y$, we obtain:

$$\begin{aligned} y' &= f = y - t^2 + 1, \\ y'' &= Df = (-2t) + (1)(y - t^2 + 1) = y - t^2 - 2t + 1, \\ y''' &= D(y'') = (-2t - 2) + (1)(y - t^2 - 2t + 1) = y - t^2 - 4t - 1, \\ y^{(4)} &= D(y''') = (-2t - 4) + (1)(y - t^2 - 4t - 1) = y - t^2 - 6t - 5. \end{aligned}$$

For step size h , the approximations are:

Order 1 (Euler):

$$y_{n+1} = y_n + h(y_n - t_n^2 + 1).$$

Order 2:

$$y_{n+1} = y_n + h y'_n + \frac{h^2}{2} y''_n = y_n + h(y_n - t_n^2 + 1) + \frac{h^2}{2}(y_n - t_n^2 - 2t_n + 1).$$

Order 4:

$$\begin{aligned} y_{n+1} &= y_n + h y'_n + \frac{h^2}{2} y''_n + \frac{h^3}{6} y'''_n + \frac{h^4}{24} y^{(4)}_n \\ &= y_n + h(y_n - t_n^2 + 1) + \frac{h^2}{2}(y_n - t_n^2 - 2t_n + 1) + \frac{h^3}{6}(y_n - t_n^2 - 4t_n - 1) \\ &\quad + \frac{h^4}{24}(y_n - t_n^2 - 6t_n - 5). \end{aligned}$$

Exact solution (for computing error e_i) is $y(t) = (t+1)^2 - \frac{1}{2}e^t$.

Table 2.3: Solution using Euler, Taylor-2, and Taylor-4 methods ($N = 20$, $h = 0.1$).

Exact			Euler		Taylor 2		Taylor 4	
t_i	$y(t_i)$	y_i	$ e_i $	y_i	$ e_i $	y_i	$ e_i $	
0.0	0.50000	0.50000	0.00000e+00	0.50000	0.00000e+00	0.50000	0.00000e+00	
0.2	0.82930	0.80000	2.92986e-02	0.83000	7.01379e-04	0.82930	1.37908e-06	
0.4	1.21409	1.15200	6.20877e-02	1.21580	1.71235e-03	1.21409	3.36882e-06	
0.6	1.64894	1.55040	9.85406e-02	1.65208	3.13540e-03	1.64895	6.17202e-06	
0.8	2.12723	1.98848	1.38750e-01	2.13233	5.10318e-03	2.12724	1.00514e-05	
1.0	2.64086	2.45818	1.82683e-01	2.64865	7.78683e-03	2.64087	1.53459e-05	
1.2	3.17994	2.94981	2.30130e-01	3.19135	1.14065e-02	3.17996	2.24922e-05	
1.4	3.73240	3.45177	2.80627e-01	3.74864	1.62446e-02	3.73243	3.20507e-05	
1.6	4.28348	3.95013	3.33356e-01	4.30615	2.26626e-02	4.28353	4.47392e-05	
1.8	4.81518	4.42815	3.87023e-01	4.84630	3.11223e-02	4.81524	6.14751e-05	
2.0	5.30547	4.86578	4.39687e-01	5.34768	4.22123e-02	5.30556	8.34286e-05	

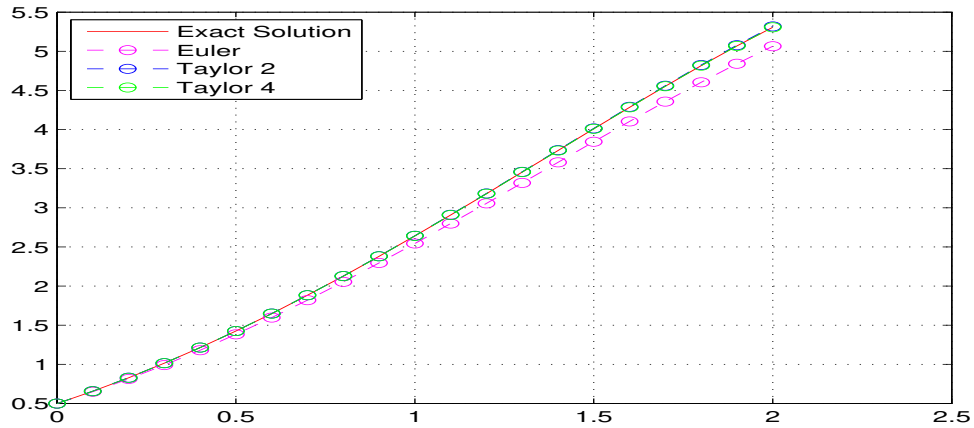


Figure 2.4: Taylor methods comparison.

Taylor Methods Matlab Implementation

```
function [t,y] = taylor2(fun,Dfun,a,b,y0,N)
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
t(i+1) = t(i) + h;
y(i+1) = y(i) + h*fun(t(i),y(i)) + h^2/2*Dfun(t(i),y(i));
```



```

end

function [t,y] = taylor4(fun,Dfun,DDfun,DDDfun,a,b,y0,N)
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
t(i+1) = t(i) + h;
y(i+1) = y(i) + h*fun(t(i),y(i)) + h^2/2*Dfun(t(i),y(i))+...
            h^3/6*DDfun(t(i),y(i)) + h^4/24*DDDfun(t(i),y(i));
end

```

The driver file are given below

```

%% ex_5_3_1
clc
close all
fun = @(t,y) y-t.^2+1;
Dfun = @(t,y) y-t.^2+1 -2*t;
DDfun = @(t,y) y-t.^2 -2*t - 1;
DDDfun = @(t,y) y-t.^2 -2*t -1;
a = 0;
b = 2;
y0 = 0.5;
N = 20;
[t1,y1] = eul(fun,a,b,y0,N);
[t2,y2] = taylor2(fun,Dfun,a,b,y0,N);
[t4,y4] = taylor4(fun,Dfun,DDfun,DDDfun,a,b,y0,N);
%% Display Solution
exactY = @(t) (t+1).^2-0.5*exp(t); Y = exactY(t1);
disp('-----');
disp('          Exact          Euler          Taylor(2)          Taylor(4) ');
disp('-----');
disp('t_i      y(t_i)      y_i      |e_i|      y_i      |e_i|      y_i      |e_i|');
disp('-----');
formatSpec = '%.1f % .5f % .5f % .5e % .5f % .5e % .5f % .5e\n';
fprintf(formatSpec,[t1';Y';y1';(abs(y1-Y))';y2';(abs(y2-Y))';y4';(abs(y4-Y))
    '])
%% Plot Solution
plot(t1, Y, 'r',t1,y1,'mo--',t2,y2,'bo--',t4,y4,'go--')
hold off; grid on

```

```
legend('Exact Solution','Euler','Taylor 2','Taylor 4','Location','northwest')
```

Remark 2.2. For this linear problem, higher total derivatives stabilize: $f^{[k]} = f^{[2]}$ for all $k \geq 2$, enabling highly accurate high-order Taylor approximations even with moderate h .

Remark 2.3. A major drawback of Taylor series methods is that they require the explicit computation of higher-order derivatives of $f(t, y)$, i.e., expressions such as

$$\frac{d^k}{dt^k} f(t, y(t)) = \mathcal{D}^k f(t, y), \quad k = 1, 2, \dots, p-1,$$

where $\mathcal{D} = \partial_t + f(t, y) \partial_y$ is the total derivative operator. These derivatives become increasingly complex and cumbersome to derive and implement even for modest orders p and are often infeasible for nonlinear or high-dimensional systems.

Exercise 2.2 (Lab work). Use Taylor's method of order 2, 3 and 4 to approximate the solutions for each of the following IVPs. Modify the above Taylor's method codes for each case.

a. $y' = \frac{2-2ty}{t^2+1}, \quad 0 \leq t \leq 1, \quad y(0) = 1, \quad \text{with } h = 0.1. \text{ the exact solution is } y(t) = \frac{2t+1}{t^2+1}$

b. $y' = \frac{y^2}{1+t}, \quad 1 \leq t \leq 2, \quad y(1) = -(\ln 2)^{-1}, \quad \text{with } h = 0.1. \text{ the exact solution is } y(t) = -\frac{1}{\ln(1+t)}$

c. $y' = \frac{y^2+y}{t}, \quad 1 \leq t \leq 3, \quad y(1) = -2, \quad \text{with } h = 0.2. \text{ the exact solution is } y(t) = \frac{2t}{1-2t}$

d. $y' = -ty + 4ty^{-1}, \quad 0 \leq t \leq 1, \quad y(0) = 1, \quad \text{with } h = 0.1. \text{ the exact solution is } y(t) = \sqrt{4-3e^{-t^2}}$

2.1.3 Runge Kutta Methods

The Taylor methods introduced in the previous section possess the desirable property of high-order local truncation error. However, their practical applicability is limited by a significant drawback: they require the explicit computation and evaluation of higher-order partial derivatives of $f(t, y)$ (e.g., f_t , f_y , f_{tt} , f_{ty} , etc.). For most nonlinear or moderately complex problems, deriving and implementing these derivatives is algebraically cumbersome, error-prone, and computationally inefficient. Consequently, Taylor methods are rarely used in practice beyond low-order implementations.

To overcome this limitation, we now introduce the *Runge-Kutta methods*. These methods attain the same high order of local truncation error as their Taylor counterparts, yet avoid the need to compute analytical derivatives of f . Instead, they achieve high-order accuracy by evaluating f multiple times per step at strategically chosen intermediate points.

Theorem 2.1.5. *Suppose that $f(t, y)$ and all its partial derivatives of order less than or equal to $n + 1$ are continuous on $D = \{(t, y) \mid a \leq t \leq b, c \leq y \leq d\}$, and let $(t_0, y_0) \in D$. For every $(t, y) \in D$, there exist ξ between t and t_0 , and μ between y and y_0 , such that*

$$f(t, y) = P_n(t, y) + R_n(t, y),$$

where

$$\begin{aligned} P_n(t, y) = & f(t_0, y_0) + (t - t_0) \frac{\partial f}{\partial t}(t_0, y_0) + (y - y_0) \frac{\partial f}{\partial y}(t_0, y_0) \\ & + \frac{(t - t_0)^2}{2} \frac{\partial^2 f}{\partial t^2}(t_0, y_0) + (t - t_0)(y - y_0) \frac{\partial^2 f}{\partial t \partial y}(t_0, y_0) + \frac{(y - y_0)^2}{2} \frac{\partial^2 f}{\partial y^2}(t_0, y_0) + \cdots \\ & + \left[\frac{1}{n!} \sum_{j=0}^n \binom{n}{j} (t - t_0)^{n-j} (y - y_0)^j \frac{\partial^n f}{\partial t^{n-j} \partial y^j}(t_0, y_0) \right], \end{aligned}$$

and

$$R_n(t, y) = \frac{1}{(n+1)!} \sum_{j=0}^{n+1} \binom{n+1}{j} (t - t_0)^{n+1-j} (y - y_0)^j \frac{\partial^{n+1} f}{\partial t^{n+1-j} \partial y^j}(\xi, \mu).$$

The function $P_n(t, y)$ is called the n th Taylor polynomial in two variables for the function f about (t_0, y_0) , and $R_n(t, y)$ is the remainder term associated with $P_n(t, y)$.

2.1.4 Runge-Kutta Methods of Order Two

The first step in deriving a Runge-Kutta method is to determine values for a_1 , α_1 , and β_1 with the property that

$$a_1 f(t + \alpha_1, y + \beta_1)$$

approximates

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} f'(t, y),$$

with error no greater than $O(h^2)$, which is the same as the order of the local truncation error for the Taylor method of order two.

Since

$$f'(t, y) = \frac{d}{dt}f(t, y) = \frac{\partial f}{\partial t}(t, y) + \frac{\partial f}{\partial y}(t, y) \cdot y'(t),$$

and $y'(t) = f(t, y)$,

$$f'(t, y) = \frac{\partial f}{\partial t}(t, y) + \frac{\partial f}{\partial y}(t, y) \cdot f(t, y).$$

We have

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} \frac{\partial f}{\partial t}(t, y) + \frac{h}{2} \frac{\partial f}{\partial y}(t, y) \cdot f(t, y).$$

Expanding $f(t + \alpha_1, y + \beta_1)$ in its Taylor polynomial of degree one about (t, y) gives

$$a_1 f(t + \alpha_1, y + \beta_1) = a_1 f(t, y) + a_1 \alpha_1 \frac{\partial f}{\partial t}(t, y) + a_1 \beta_1 \frac{\partial f}{\partial y}(t, y) + a_1 \cdot R_1(t + \alpha_1, y + \beta_1), \quad (2.18)$$

where

$$R_1(t + \alpha_1, y + \beta_1) = \frac{\alpha_1^2}{2} \frac{\partial^2 f}{\partial t^2}(\xi, \mu) + \alpha_1 \beta_1 \frac{\partial^2 f}{\partial t \partial y}(\xi, \mu) + \frac{\beta_1^2}{2} \frac{\partial^2 f}{\partial y^2}(\xi, \mu),$$

for some ξ between t and $t + \alpha_1$, and μ between y and $y + \beta_1$.

Matching the coefficients of f and its derivatives gives the three equations:

$$\begin{aligned} f(t, y) : \quad & a_1 = 1; \\ \frac{\partial f}{\partial t}(t, y) : \quad & a_1 \alpha_1 = \frac{h}{2}; \\ \frac{\partial f}{\partial y}(t, y) : \quad & a_1 \beta_1 = \frac{h}{2} f(t, y). \end{aligned}$$

Thus, the parameters are:

$$a_1 = 1, \quad \alpha_1 = \frac{h}{2}, \quad \beta_1 = \frac{h}{2} f(t, y).$$

Therefore,

$$T^{(2)}(t, y) = f\left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y)\right) - R_1\left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y)\right),$$

and,

$$R_1\left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y)\right) = \frac{h^2}{8} \frac{\partial^2 f}{\partial t^2}(\xi, \mu) + \frac{h^2}{4} f(t, y) \frac{\partial^2 f}{\partial t \partial y}(\xi, \mu) + \frac{h^2}{8} (f(t, y))^2 \frac{\partial^2 f}{\partial y^2}(\xi, \mu).$$

If all the second-order partial derivatives of f are bounded, then

$$R_1 \left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y) \right) = O(h^2)$$

As a consequence: The order of error for this new method is the same as that of the Taylor method of order two.

The difference-equation method resulting from replacing $T^{(2)}(t, y)$ in Taylor's method of order two by

$$f \left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y) \right)$$

is a specific Runge-Kutta method known as the **Midpoint method**.

2.1.5 Midpoint Method

The Midpoint Method is defined as follows:

$$\begin{cases} w_0 = \alpha, \\ w_{i+1} = w_i + hf \left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i) \right), \quad \text{for } i = 0, 1, \dots, N-1. \end{cases} \quad (2.19)$$

Only three parameters are present in the expression $a_1 f(t + \alpha_1, y + \beta_1)$, and all are needed to match $T^{(2)}$. Therefore, a more complicated form is required to satisfy the conditions for any higher-order Taylor methods.

The most appropriate four-parameter form for approximating

$$T^{(3)}(t, y) = f(t, y) + \frac{h}{2} f'(t, y) + \frac{h^2}{6} f''(t, y)$$

is

$$a_1 f(t, y) + a_2 f(t + \alpha_2, y + \delta_2 f(t, y));$$

however, even with this form, there is insufficient flexibility to match the term

$$\frac{h^2}{6} \left(2 \frac{\partial f}{\partial y}(t, y) \right) f(t, y), \quad (2.20)$$

which arises from the expansion of $\frac{h^2}{6} f''(t, y)$. Consequently, the best that can be obtained using (2.20) are methods with $O(h^2)$ local truncation error.

The fact that (2.20) has four parameters, however, gives flexibility in their choice, so a number of $O(h^2)$ methods can be derived.

One of the most important is the **Modified Euler method**, which corresponds to choosing

$$a_1 = a_2 = \frac{1}{2}, \quad \alpha_2 = \delta_2 = h.$$

It has the following difference-equation form.

Modified Euler Method

$$\begin{cases} w_0 = \alpha, \\ w_{i+1} = w_i + \frac{h}{2} [f(t_i, w_i) + f(t_{i+1}, w_i + hf(t_i, w_i))], \quad \text{for } i = 0, 1, \dots, N-1. \end{cases} \quad (2.21)$$

Example 2.4. Use the **Midpoint method** and the **Modified Euler method** with $N = 10$, $h = 0.2$ to approximate the solution to the initial value problem:

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5$$

Using Midpoint Method

Given: $f(t, y) = y - t^2 + 1$, $h = 0.2$. Initial condition: $t_0 = 0$, $y_0 = 0.5$

Step 1: For $i = 0$:

$$\begin{aligned} k_1 &= h \cdot f(t_0, y_0) = 0.2 \cdot (0.5 - 0^2 + 1) = 0.2 \cdot 1.5 = 0.3 \\ k_2 &= h \cdot f\left(t_0 + \frac{1}{2}h, y_0 + \frac{1}{2}k_1\right) \\ &= 0.2 \cdot f(0.1, 0.65) = 0.2 \cdot (0.65 - 0.01 + 1) = 0.2 \cdot 1.64 = 0.328 \\ y_1 &= y_0 + k_2 = 0.5 + 0.328 = 0.828 \end{aligned}$$

Step 2: For $i = 1$:

$$t_1 = t_0 + h = 0 + 0.2 = 0.2, \quad y_1 = 0.828$$

$$\begin{aligned}
k_1 &= h \cdot f(t_1, y_1) = 0.2 \cdot (0.828 - 0.04 + 1) = 0.2 \cdot 1.788 = 0.3576 \\
k_2 &= h \cdot f\left(t_1 + \frac{1}{2}h, y_1 + \frac{1}{2}k_1\right) \\
&= 0.2 \cdot f(0.3, 1.0068) = 0.2 \cdot (1.0068 - 0.09 + 1) = 0.2 \cdot 1.9168 = 0.38336 \\
y_2 &= y_1 + k_2 = 0.828 + 0.38336 = 1.21136
\end{aligned}$$

Using Modified Euler's Method

We are approximating the solution to the initial value problem:

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5$$

using the Modified Euler's Method with step size $h = 0.2$ and $N = 10$.

The method uses the following formulas:

$$\begin{cases} k_1 = h f(t_i, y_i), \\ k_2 = h f(t_i + h, y_i + k_1), \\ y_{i+1} = y_i + \frac{1}{2}(k_1 + k_2). \end{cases} \quad (2.22)$$

Given:

$$f(t, y) = y - t^2 + 1$$

Step 1: For $i = 0, t_0 = 0, \quad y_0 = 0.5$

Compute:

$$k_1 = h \cdot f(t_0, y_0) = 0.2 \cdot (0.5 - 0^2 + 1) = 0.2 \cdot 1.5 = 0.3$$

$$k_2 = h \cdot f(t_0 + h, y_0 + k_1) = 0.2 \cdot f(0.2, 0.8) = 0.2 \cdot (0.8 - 0.2^2 + 1) = 0.2 \cdot (0.8 - 0.04 + 1) = 0.2 \cdot 1.76 = 0.352$$

Update:

$$y_1 = y_0 + \frac{1}{2}(k_1 + k_2) = 0.5 + \frac{1}{2}(0.3 + 0.352) = 0.5 + 0.326 = 0.826$$

Step 2: For $i = 1$

$$t_1 = 0.2, \quad y_1 = 0.826$$

Compute:

$$k_1 = h \cdot f(t_1, y_1) = 0.2 \cdot (0.826 - 0.2^2 + 1) = 0.2 \cdot (0.826 - 0.04 + 1) = 0.2 \cdot 1.786 = 0.3572$$

$$k_2 = h \cdot f(t_1 + h, y_1 + k_1) = 0.2 \cdot f(0.4, 1.1832) = 0.2 \cdot (1.1832 - 0.4^2 + 1) = 0.40464$$

Update:

$$y_2 = y_1 + \frac{1}{2}(k_1 + k_2) = 0.826 + \frac{1}{2}(0.3572 + 0.40464) = 0.826 + 0.38092 = 1.20692$$

The remaining steps can be computed similarly or implemented in a program to complete the full approximation over the interval $[0, 2]$.

Modified Euler's Method Matlab Implementation

```
function [t,y] = eulmod(fun,a,b,y0,N)
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
    t(i+1) = t(i) + h;
    k1 = h*fun(t(i),y(i));
    k2 = h*fun(t(i)+h,y(i)+k1);
    y(i+1) = y(i) + 1/2*(k1 + k2);
end
```

Midpoint Method

```
function [t,y] = midpoint(fun,a,b,y0,N)
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
    t(i+1) = t(i) + h;
    k1 = h*fun(t(i),y(i));
    k2 = h*fun(t(i)+h/2,y(i)+k1/2);
    y(i+1) = y(i) + k2;
end
```

MATLAB Driver File: `ex_5_4_2.m`


```

%% ex_5_4_2
clc
fun = @(t,y) y - t.^2 + 1;
y0 = 0.5;
a = 0;
b = 2;
N = 10;
[t1,y1] = eul(fun,a,b,y0,N);          % Euler's Method
[t2,y2] = eulmod(fun,a,b,y0,N);       % Modified Euler Method
[t3,y3] = midpoint(fun,a,b,y0,N);     % Midpoint Method
%% Display Solution
exactY = @(t) (t+1).^2 - 0.5*exp(t);
Y = exactY(t1);
disp('-----');
disp('t_i    y(t_i)    Euler    Error    ModEuler    Error    Midpoint
      Error');
disp('-----');
formatSpec = '%.1f % .5f % .5f % .5f % .5f % .5f % .5f % .5f\n';
fprintf(formatSpec,[t1'; Y'; y1'; abs(y1 - Y)'; y2'; abs(y2 - Y)'; y3'; abs(y3
- Y)']);

```

The following table compares the exact solution to the approximations obtained using: Classical Euler Method (1st order), Modified Euler Method (2nd order), Midpoint Method (2nd order).

t_i	Exact $y(t_i)$	Euler	Error	Mod. Euler	Error	Midpoint	Error
0.0	0.50000	0.50000	0.00000	0.50000	0.00000	0.50000	0.00000
0.2	0.82930	0.80000	0.02930	0.82600	0.00330	0.82800	0.00130
0.4	1.21409	1.15200	0.06209	1.20692	0.00717	1.21136	0.00273
0.6	1.64894	1.55040	0.09854	1.63724	0.01170	1.64466	0.00428
0.8	2.12723	1.98848	0.13875	2.11024	0.01699	2.12128	0.00595
1.0	2.64086	2.45818	0.18268	2.61769	0.02317	2.63317	0.00769
1.2	3.17994	2.94981	0.23013	3.14958	0.03036	3.17046	0.00948
1.4	3.73240	3.45177	0.28063	3.69369	0.03871	3.72117	0.01123
1.6	4.28348	3.95013	0.33336	4.23510	0.04839	4.27062	0.01286
1.8	4.81518	4.42815	0.38702	4.75562	0.05956	4.80096	0.01422
2.0	5.30547	4.86578	0.43969	5.23305	0.07242	5.29037	0.01510

- As expected, both the Modified Euler method and the Midpoint method significantly outperform the classical Euler method. This is because the former are second-order accurate methods, while the latter are only first-order accurate.

- Among the two second-order methods, the Midpoint method produces more accurate results than the Modified Euler method throughout the interval. This suggests that for this particular initial value problem, the Midpoint method provides the best approximation when using a fixed step size $h = 0.2$.
- This behavior aligns with general expectations from numerical analysis: higher-order methods yield smaller global truncation errors over the same integration interval, especially for smooth functions.

2.1.6 Higher-Order Runge-Kutta Methods

Runge-Kutta Method of Third Order

We derive **Kutta's third-order Runge-Kutta method** (RK3) by enforcing third-order accuracy conditions on a general 3-stage explicit Runge-Kutta scheme.

A 3-stage explicit Runge-Kutta method has the form:

$$\begin{cases} k_1 = f(t_i, y_i), \\ k_2 = f(t_i + c_2h, y_i + h a_{21}k_1), \\ k_3 = f(t_i + c_3h, y_i + h (a_{31}k_1 + a_{32}k_2)), \\ y_{i+1} = y_i + h (b_1k_1 + b_2k_2 + b_3k_3). \end{cases} \quad (2.23)$$

where the coefficients satisfy the explicitness constraints:

$$a_{21} = c_2, \quad c_3 = a_{31} + a_{32}.$$

To achieve local truncation error $\mathcal{O}(h^4)$ (i.e., global error $\mathcal{O}(h^3)$), the method must satisfy the following four classical order conditions:

$$(C1) \quad b_1 + b_2 + b_3 = 1, \quad (2.24)$$

$$(C2) \quad b_2c_2 + b_3c_3 = \frac{1}{2}, \quad (2.25)$$

$$(C3) \quad b_2c_2^2 + b_3c_3^2 = \frac{1}{3}, \quad (2.26)$$

$$(C4) \quad b_3a_{32}c_2 = \frac{1}{6}. \quad (2.27)$$

These arise from matching the Taylor expansion of the exact solution up to terms of order h^3 .

Kutta (1901) chose stage abscissae corresponding to natural points in the interval $[t_i, t_{i+1}]$:

$$c_2 = \frac{1}{2}, \quad c_3 = 1.$$

This aligns stages with t_i , $t_i + h/2$, and $t_i + h$, analogous to Simpson's quadrature rule.

Step 1: Solve for b_2 , b_3 , b_1

Substitute $c_2 = \frac{1}{2}$, $c_3 = 1$ into (C2) and (C3):

From (2.25):

$$\frac{1}{2}b_2 + b_3 = \frac{1}{2} \quad \Rightarrow \quad b_2 = 1 - 2b_3. \quad (1)$$

From (2.26):

$$\frac{1}{4}b_2 + b_3 = \frac{1}{3}. \quad (2)$$

Insert (1) into (2):

$$\begin{aligned} \frac{1}{4}(1 - 2b_3) + b_3 &= \frac{1}{3} \\ \frac{1}{4} - \frac{1}{2}b_3 + b_3 &= \frac{1}{3} \\ \frac{1}{4} + \frac{1}{2}b_3 &= \frac{1}{3} \\ \frac{1}{2}b_3 &= \frac{1}{3} - \frac{1}{4} = \frac{1}{12} \\ \Rightarrow b_3 &= \frac{1}{6}. \end{aligned}$$

Then from (1):

$$b_2 = 1 - 2 \cdot \frac{1}{6} = \frac{2}{3},$$

and from (C1):

$$b_1 = 1 - b_2 - b_3 = 1 - \frac{2}{3} - \frac{1}{6} = \frac{1}{6}.$$

Step 2: Determine a_{32} and a_{31}

Use condition (C4), (2.27):

$$b_3 a_{32} c_2 = \frac{1}{6} \quad \Rightarrow \quad \frac{1}{6} \cdot a_{32} \cdot \frac{1}{2} = \frac{1}{6} \quad \Rightarrow \quad a_{32} = 2.$$

Now apply the consistency condition $c_3 = a_{31} + a_{32}$:

$$1 = a_{31} + 2 \quad \Rightarrow \quad a_{31} = -1.$$

Substituting the coefficients yields **Kutta's third-order method**:

$$\begin{cases} k_1 = f(t_i, y_i), \\ k_2 = f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right), \\ k_3 = f(t_i + h, y_i - hk_1 + 2hk_2), \\ y_{i+1} = y_i + \frac{h}{6}(k_1 + 4k_2 + k_3). \end{cases} \quad (2.28)$$

Its Butcher tableau is:

$$\begin{array}{c|ccc} 0 & & & \\ \frac{1}{2} & \frac{1}{2} & & \\ 1 & -1 & 2 & \\ \hline & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \end{array}$$

Condition	Left-hand side	Required value
(C1) $b_1 + b_2 + b_3$	$\frac{1}{6} + \frac{2}{3} + \frac{1}{6} = 1$	1
(C2) $b_2c_2 + b_3c_3$	$\frac{2}{3} \cdot \frac{1}{2} + \frac{1}{6} \cdot 1 = \frac{1}{2}$	$\frac{1}{2}$
(C3) $b_2c_2^2 + b_3c_3^2$	$\frac{2}{3} \cdot \frac{1}{4} + \frac{1}{6} \cdot 1 = \frac{1}{3}$	$\frac{1}{3}$
(C4) $b_3a_{32}c_2$	$\frac{1}{6} \cdot 2 \cdot \frac{1}{2} = \frac{1}{6}$	$\frac{1}{6}$

Table 2.4: Verification of order conditions for Kutta's RK3.

All conditions are satisfied; thus the method has local truncation error $\mathcal{O}(h^4)$ and global error $\mathcal{O}(h^3)$.

Runge-Kutta Method of Order Four

$$\left\{ \begin{array}{l} w_0 = \alpha, \\ k_1 = hf(t_i, w_i), \\ k_2 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right), \\ k_3 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right), \\ k_4 = hf(t_i + h, w_i + k_3), \\ w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4), \end{array} \right. \quad (2.29)$$

for each $i = 0, 1, \dots, N - 1$.

This method has local truncation error $O(h^4)$, provided the solution $y(t)$ has five continuous derivatives. We introduce the notation k_1, k_2, k_3, k_4 into the method to eliminate the need for successive nesting in the second variable of $f(t, y)$.

Example 2.5. Use the Runge-Kutta method of order four with $h = 0.2$, $N = 10$, and $t_i = 0.2i$ to obtain approximations to the solution of the initial-value problem

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5.$$

Solution 2.2. The approximation to $y(0.2)$ is obtained as follows:

$$w_0 = 0.5$$

$$K_1 = 0.2 \cdot f(0, 0.5) = 0.2 \cdot (0.5 - 0 + 1) = 0.2 \cdot 1.5 = 0.3$$

$$K_2 = 0.2 \cdot f(0.1, 0.65) = 0.2 \cdot (0.65 - 0.01 + 1) = 0.2 \cdot 1.64 = 0.328$$

$$K_3 = 0.2 \cdot f(0.1, 0.664) = 0.2 \cdot (0.664 - 0.01 + 1) = 0.2 \cdot 1.654 = 0.3308$$

$$K_4 = 0.2 \cdot f(0.2, 0.8308) = 0.2 \cdot (0.8308 - 0.04 + 1) = 0.2 \cdot 1.7908 = 0.35816$$

$$\begin{aligned} w_1 &= w_0 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ &= 0.5 + \frac{1.97576}{6} = 0.5 + 0.3292933 = 0.8292933. \end{aligned}$$

The remaining results and their errors are listed in Table 2.5.

Algorithm 2 Classical Fourth-Order Runge-Kutta Method

Require: Function $f(t, y)$, interval endpoints a and b , number of steps N , initial condition α .

Ensure: Approximate values $w \approx y(t)$ at $N + 1$ equally spaced points in $[a, b]$.

- 1: Set $h = \frac{b-a}{N}$;
- 2: Set $t = a$;
- 3: Set $w_0 = \alpha$;
 OUTPUT (t_i, w_i)
- 4: **for** $i = 1, 2, \dots, N$ **do**
- 5: Compute:

$$k_1 \leftarrow h \cdot f(t, w) \tag{2.30}$$

$$k_2 \leftarrow h \cdot f\left(t + \frac{h}{2}, w_i + \frac{k_1}{2}\right) \tag{2.31}$$

$$k_3 \leftarrow h \cdot f\left(t + \frac{h}{2}, w_i + \frac{k_2}{2}\right) \tag{2.32}$$

$$k_4 \leftarrow h \cdot f(t + h, w_i + k_3) \tag{2.33}$$

- 6: Update:

$$w_{i+1} \leftarrow w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

- 7: Set $t \leftarrow a + i \cdot h$ {Update current time t_i } OUTPUT (t_i, w_i)
 - 8: **end for**
 - 9: **STOP**
-

Table 2.5: Runge–Kutta method of order four and errors

t_i	Exact	Approximation w_i	$ y(t_i) - w_i $
0.0	0.5000000	0.5000000	0.0000000e0
0.2	0.8292986	0.8292933	5.3000e-6
0.4	1.2140877	1.2140762	1.1400e-5
0.6	1.6489406	1.6489220	1.8600e-5
0.8	2.1272295	2.1272027	2.6900e-5
1.0	2.6408591	2.6408227	3.6400e-5
1.2	3.1799415	3.1798942	4.7400e-5
1.4	3.7324000	3.7323401	5.9900e-5
1.6	4.2834838	4.2834095	7.4300e-5
1.8	4.8151763	4.8150857	9.0600e-5
2.0	5.3054720	5.3053630	1.0890e-4

```
function [t,y] = rk4(fun,a,b,y0,N)
h = (b-a)/N;
t = zeros(N+1,1);
y = zeros(N+1,1);
t(1) = a;
y(1) = y0;
for i = 1:N
    t(i+1) = t(i) + h;
    k1 = h*fun(t(i),y(i));
    k2 = h*fun(t(i)+h/2,y(i)+k1/2);
    k3 = h*fun(t(i)+h/2,y(i)+k2/2);
    k4 = h*fun(t(i)+h,y(i)+k3);
    y(i+1) = y(i) + 1/6*(k1 + 2*k2 + 2*k3 + k4);
end
```

```
clc
fun = @(t,y) y - t.^2 + 1;
y0 = 0.5; a = 0; b = 2; N = 10;
[t,y] = rk4(fun,a,b,y0,N);
%% Display Solution
exactY = @(t) (t+1).^2 - 0.5*exp(t);
Y = exactY(t);
disp('-----');
disp('t_i    y(t_i)    RK4    Error')
disp('-----');
formatSpec = '%.1f % .6f % .6f % .6f\n';
fprintf(formatSpec,[t'; Y'; y'; (abs(y - Y))'])
```

2.1.7 Computational Comparisons

The main computational effort in applying the Runge-Kutta methods is the evaluation of $f(t, y)$. In second-order methods, the local truncation error is $O(h^2)$, and the cost is two function evaluations per step.

The Runge-Kutta method of order four requires four evaluations per step, and the local truncation error is $O(h^4)$. Butcher has established the relationship between the number of function evaluations per step and the best possible order of the local truncation error, as shown in Table 2.6.

This table indicates why methods of order less than five with smaller step sizes are often preferred over higher-order methods using larger step sizes.

Table 2.6: Best Possible Local Truncation Error vs. Function Evaluations per Step

Evaluations per step	2	3	4	n	
Best possible local truncation error	$O(h^2)$	$O(h^3)$	$O(h^4)$	$O(h^{n-1})$	$O(h^{n-2})$
Range of n	–	–	–	$5 \leq n \leq 7$	$8 \leq n \leq 9, 10 \leq n$

Example 2.6. *We consider the initial value problem:*

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5$$

To fairly compare the performance of different numerical methods, we set the total number of function evaluations to be the same across all methods:

Euler's Method (1 evaluation/step): $h = 0.05 \rightarrow 40$ steps.

Modified Euler Method (2 evaluations/step): $h = 0.1 \rightarrow 20$ steps.

RK4 Method (4 evaluations/step): $h = 0.2 \rightarrow 10$ steps.

Each method performs a total of **40 function evaluations**, enabling a fair comparison. The following table compares the exact solution with the numerical approximations at selected time points:

From this example, it is clear that the 4th-order Runge-Kutta method (RK4) significantly outperforms both Euler's and Modified Euler's methods in terms of accuracy, even though all three methods use the same number of function evaluations.

This demonstrates the advantage of using higher-order methods when high accuracy is required.

Table 2.7: Computational comparison with fair function evaluation.

t_i	Exact $y(t_i)$	Euler ($h = 0.05$)		Mod. Euler ($h = 0.1$)		RK4 ($h = 0.2$)	
		y_i	$ e_i $	y_i	$ e_i $	y_i	$ e_i $
0.0	0.5000000	0.5000000	0.00e+0	0.5000000	0.00e+0	0.5000000	0.00e+0
0.2	0.8292986	0.8214716	7.83e-3	0.8284350	8.64e-4	0.8292933	5.30e-6
0.4	1.2140877	1.1973995	1.67e-2	1.2122113	1.88e-3	1.2140762	1.15e-5
0.6	1.6489406	1.6222790	2.67e-2	1.6458789	3.06e-3	1.6489220	1.86e-5
0.8	2.1272295	2.0894190	3.78e-2	2.1227827	4.45e-3	2.1272027	2.68e-5
1.0	2.6408591	2.5906863	5.02e-2	2.6347973	6.06e-3	2.6408227	3.64e-5
1.2	3.1799415	3.1161950	6.37e-2	3.1720009	7.94e-3	3.1798942	4.73e-5
1.4	3.7324000	3.6539290	7.85e-2	3.7222788	1.01e-2	3.7323401	5.99e-5
1.6	4.2834838	4.1892822	9.42e-2	4.2708390	1.26e-2	4.2834095	7.43e-5
1.8	4.8151763	4.7045011	1.11e-1	4.7996197	1.56e-2	4.8150857	9.06e-5
2.0	5.3054720	5.1780062	1.27e-1	5.2865672	1.89e-2	5.3053630	1.09e-4

Exercise 2.3 (Lab work). Use Runge Kutta methods of order 2 (Modified Euler and Mid-point), 3 and 4 to approximate the solutions for each of the following IVPs. Modify the above codes for each case.

- a. $y' = \frac{2-2ty}{t^2+1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \frac{2t+1}{t^2+1}$
- b. $y' = \frac{y^2}{1+t}$, $1 \leq t \leq 2$, $y(1) = -(\ln 2)^{-1}$, with $h = 0.1$. the exact solution is $y(t) = -\frac{1}{\ln(1+t)}$
- c. $y' = \frac{y^2+y}{t}$, $1 \leq t \leq 3$, $y(1) = -2$, with $h = 0.2$. the exact solution is $y(t) = \frac{2t}{1-2t}$
- d. $y' = -ty + 4ty^{-1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \sqrt{4 - 3e^{-t^2}}$

2.1.8 Stability analysis of single step explicit methods

Numerical methods for solving initial value problems (IVPs) such as Euler's method and Runge-Kutta methods are essential tools in applied mathematics, physics, and engineering. While accuracy and convergence are important, **stability** is equally critical: it determines whether small errors grow uncontrollably over time.

This section analyzes the stability of several one-step methods by applying them to the **Linear test equation**:

$$y' = \lambda y, \quad y(0) = y_0,$$

where $\lambda \in \mathbb{C}$ (often with $\Re(\lambda) < 0$ to model decaying solutions). The exact solution is $y(t) = y_0 e^{\lambda t}$, which decays as $t \rightarrow \infty$.

A numerical method is considered **absolutely stable** if the numerical solution remains bounded (or decays) under iteration, provided the step size h is chosen appropriately.

Euler's Method

Euler's method for $y' = f(t, y)$ is:

$$y_{n+1} = y_n + hf(t_n, y_n). \quad (2.34)$$

Applied to the test equation $y' = \lambda y$, this becomes:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)y_n.$$

Iterating gives:

$$y_n = (1 + h\lambda)^n y_0.$$

For stability, we require $|y_n| \leq |y_0|$ as $n \rightarrow \infty$, so:

$$|1 + h\lambda| \leq 1.$$

This defines the **region of absolute stability** in the complex $z = h\lambda$ -plane.

Definition 2.4 (Stability Function). *The **stability function** $R(z)$ of a numerical method is defined by $y_{n+1} = R(z)y_n$ when applied to $y' = \lambda y$, with $z = h\lambda$.*

For Euler's method:

$$R(z) = 1 + z. \quad (2.35)$$

The method is stable when $|R(z)| \leq 1$, i.e.,

$$|1 + z| \leq 1. \quad (2.36)$$

This is a disk of radius 1 centered at $z = -1$ in the complex plane.

Remark 2.4. *For real $\lambda < 0$, stability requires $h < 2/|\lambda|$. This limits the usable step size, especially for stiff equations.*

Second-Order Runge–Kutta (Midpoint Method)

The second-order Runge–Kutta (RK2), or midpoint method, is:

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \\ y_{n+1} = y_n + h k_2. \end{cases} \quad (2.37)$$

Apply to $y' = \lambda y$, so $f(t, y) = \lambda y$:

$$\begin{cases} k_1 = \lambda y_n, \\ k_2 = \lambda\left(y_n + \frac{h}{2} \cdot \lambda y_n\right) = \lambda y_n\left(1 + \frac{h\lambda}{2}\right), \\ y_{n+1} = y_n + h \cdot \lambda y_n\left(1 + \frac{h\lambda}{2}\right) = y_n \left[1 + z + \frac{z^2}{2}\right], \end{cases} \quad (2.38)$$

where $z = h\lambda$. Thus, the stability function is:

$$R_{\text{RK2}}(z) = 1 + z + \frac{z^2}{2}.$$

The method is stable when $|R_{\text{RK2}}(z)| \leq 1$. This region is larger than Euler's, allowing larger step sizes for stable integration.

Third-Order Runge–Kutta Method

Consider the classical third-order Runge–Kutta method with Butcher tableau:

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 \\ 1 & -1 & 2 & 0 \\ \hline & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \end{array}$$

The RK3 method is:

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \\ k_3 = f\left(t_n + h, y_n - hk_1 + 2hk_2\right), \\ y_{n+1} = y_n + h\left(\frac{1}{6}k_1 + \frac{2}{3}k_2 + \frac{1}{6}k_3\right). \end{cases} \quad (2.39)$$

Apply to $y' = \lambda y$ ($f(t, y) = \lambda y$), with $z = h\lambda$:

$$\begin{aligned}
 k_1 &= \lambda y_n, \\
 k_2 &= \lambda \left(y_n + \frac{h}{2} k_1 \right) = \lambda y_n \left(1 + \frac{z}{2} \right), \\
 k_3 &= \lambda (y_n - h k_1 + 2h k_2) \\
 &= \lambda \left[y_n - z y_n + 2z y_n \left(1 + \frac{z}{2} \right) \right] \\
 &= \lambda y_n [1 - z + 2z + z^2] = \lambda y_n (1 + z + z^2).
 \end{aligned}$$

Now compute y_{n+1} :

$$\begin{aligned}
 y_{n+1} &= y_n + h \left(\frac{1}{6} k_1 + \frac{2}{3} k_2 + \frac{1}{6} k_3 \right) \\
 &= y_n + z y_n \left[\frac{1}{6} + \frac{2}{3} \left(1 + \frac{z}{2} \right) + \frac{1}{6} (1 + z + z^2) \right] \\
 &= y_n \left\{ 1 + z \left[\underbrace{\frac{1}{6} + \frac{2}{3} + \frac{1}{6}}_{=1} + z \underbrace{\left(\frac{2}{3} \cdot \frac{1}{2} + \frac{1}{6} \right)}_{=\frac{1}{2}} + z^2 \underbrace{\frac{1}{6}}_{=\frac{1}{6}} \right] \right\} \\
 &= y_n \left(1 + z + \frac{z^2}{2} + \frac{z^3}{6} \right).
 \end{aligned}$$

Thus, the stability function is:

$$R_{\text{RK3}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6}. \quad (2.40)$$

This matches the third-order Taylor expansion of e^z , as expected for a third-order method.

Stability Regions

The **region of absolute stability** is the set $\{z \in \mathbb{C} : |R(z)| \leq 1\}$.

- **Euler (RK1):** $R(z) = 1 + z$
Stability region: disk $|1 + z| \leq 1$ (center -1 , radius 1).
- **RK2 (midpoint):** $R(z) = 1 + z + \frac{z^2}{2}$
Region is a bounded, teardrop-shaped domain containing the interval $[-2, 0]$ on the real axis.
- **RK3 (classical):** $R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6}$
Region is larger than RK2's; contains $[-2.5, 0]$ approximately on the real axis.

All three are **explicit** methods, so their stability regions are bounded. Higher-order explicit RK methods have larger (but still bounded) stability regions, permitting larger stable step sizes for non-stiff problems. For stiff problems, implicit methods (e.g., backward Euler, Radau) are required.

Method	Stability Function $R(z)$
Euler (RK1)	$1 + z$
RK2 (Midpoint)	$1 + z + \frac{z^2}{2}$
RK3 (Classical)	$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6}$
RK4 (Classical)	$1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$

Note: The RK4 stability function comes from its Taylor expansion matching e^z to $O(z^5)$.

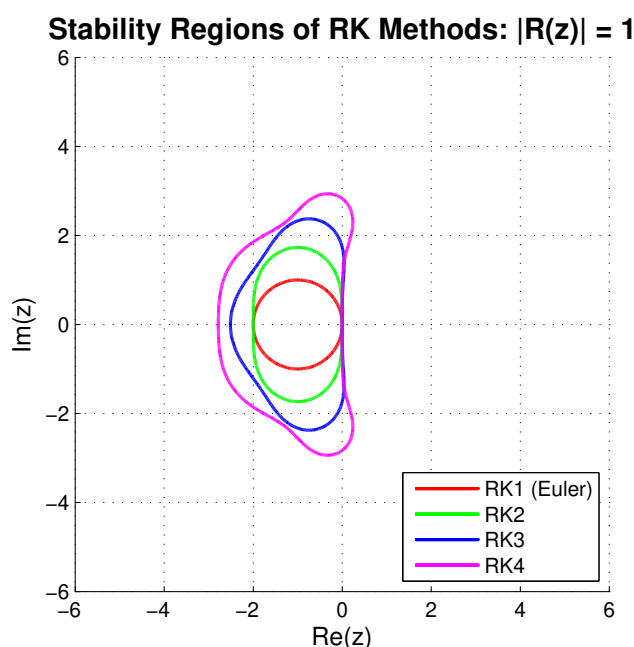


Figure 2.5: Stability regions for Runge–Kutta methods of orders 1 to 4.

Key Observations

- All explicit Runge–Kutta methods have polynomial stability functions.
- Their stability regions are bounded they cannot handle very stiff problems without small h .

- Higher-order methods have larger stability regions, but still limited compared to implicit methods.
- For real $\lambda < 0$, the maximum stable step size increases with method order.

Stability analysis via the test equation $y' = \lambda y$ provides crucial insight into the behavior of numerical methods. Euler's method has a small stability region, while higher-order Runge–Kutta methods offer improved stability and accuracy. However, for stiff systems, **implicit methods** (e.g., backward Euler, BDF) are often required.

The following MATLAB script computes and plots the stability regions.

```
%% Stability Regions for RK1 to RK4
clc; clear; close all;
% Define complex grid
x = linspace(-3, 2, 500);
y = linspace(-2, 2, 500); % Keep consistent with actual data
[X, Y] = meshgrid(x, y);
Z = X + 1i*Y;
% Stability functions (Taylor approximations up to order p)
R1 = 1 + Z; % RK1: Euler
R2 = 1 + Z + (Z.^2)/2; % RK2
R3 = 1 + Z + (Z.^2)/2 + (Z.^3)/6; % RK3 (corrected: /6, not /3)
R4 = 1 + Z + (Z.^2)/2 + (Z.^3)/6 + (Z.^4)/24; % RK4
% Magnitudes
modR1 = abs(R1);
modR2 = abs(R2);
modR3 = abs(R3);
modR4 = abs(R4);
% Create figure
figure;
hold on; grid on;
contour(X, Y, modR1, [1 1], 'Color', 'r', 'LineWidth', 1.5);
contour(X, Y, modR2, [1 1], 'Color', 'g', 'LineWidth', 1.5);
contour(X, Y, modR3, [1 1], 'Color', 'b', 'LineWidth', 1.5);
contour(X, Y, modR4, [1 1], 'Color', 'm', 'LineWidth', 1.5);
% Formatting
axis equal;
xlabel('Re(z)', 'FontSize', 12);
ylabel('Im(z)', 'FontSize', 12);
title('Stability Regions of RK Methods: |R(z)| = 1', 'FontSize', 14, 'FontWeight', 'bold');
```

```
axis([-3 2 -2 2]); % Match the grid
legend('RK1 (Euler)', 'RK2', 'RK3', 'RK4', 'Location', 'southeast');
hold off;
```

The stability regions grow with the order of the method, making higher-order Runge–Kutta methods more robust for non-stiff problems. However, for stiff systems, implicit methods are required due to the bounded nature of explicit stability regions.

Exercise 2.4. *Modify the code to overlay all four stability boundaries on a single plot for comparison to the linear test equation:*

$$y' = -\lambda y, \quad y(0) = y_0,$$

where $\lambda \in \mathbb{C}$ and $\Re(\lambda) > 0$ (ensuring the exact solution decays as $t \rightarrow \infty$). The exact solution is $y(t) = y_0 e^{-\lambda t}$, which satisfies $|y(t)| \rightarrow 0$ as $t \rightarrow \infty$ whenever $\Re(\lambda) > 0$.

2.2 Multistep Methods

The numerical methods discussed earlier such as Euler’s method and Runge–Kutta methods are called **one-step methods** because the approximation at t_{i+1} depends only on information from the single previous point t_i .

In contrast, **multistep methods** use approximations from *multiple* previous time steps to compute y_{i+1} . Since the solution has already been approximated at $t_0, t_1, \dots, t_i$, it is natural to exploit this historical data to construct higher-order, more efficient schemes.

Definition 2.5 (Multistep Method). *An m -step multistep method for solving the initial value problem*

$$y' = f(t, y), \quad y(a) = y_0, \quad a \leq t \leq b,$$

is defined by the recurrence relation

$$y_{i+1} = \sum_{j=0}^{m-1} a_j y_{i-j} + h \sum_{j=0}^m b_j f(t_{i-j+1}, y_{i-j+1}), \quad i = m-1, m, \dots, N-1,$$

where: $h = \frac{b-a}{N}$ is the uniform step size, a_j, b_j are fixed real constants (independent of h), $f_k = f(t_k, y_k)$ denotes the derivative at step k .

The method is classified as:

- **Explicit** if $b_m = 0$ (so f_{i+1} does not appear),
- **Implicit** if $b_m \neq 0$ (so y_{i+1} appears on both sides and requires solving an equation).

Remark 2.5. To start an m -step method, we need m initial values:

$$y_0, y_1, \dots, y_{m-1}.$$

Since only y_0 is given by the initial condition, the remaining $m - 1$ values must be computed using a one-step method (e.g., a fourth-order Runge–Kutta method) of comparable accuracy.

2.2.1 Common Examples of Multistep Methods

Multistep methods are often derived by integrating the ODE over an interval and approximating the integral using polynomial interpolation of f . Two important families are the Adams–Bashforth (explicit) and Adams–Moulton (implicit) methods.

Adams–Bashforth Methods (Explicit)

These are derived by interpolating f at previous points $t_i, t_{i-1}, \dots$ and integrating forward.

- **Two-step AB2:**

$$y_{i+1} = y_i + \frac{h}{2}(3f_i - f_{i-1})$$

- **Three-step AB3:**

$$y_{i+1} = y_i + \frac{h}{12}(23f_i - 16f_{i-1} + 5f_{i-2})$$

- **Four-step AB4:**

$$y_{i+1} = y_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

Adams–Moulton Methods (Implicit)

These use interpolation that includes the future point t_{i+1} , leading to implicit formulas with higher accuracy.

- **Two-step AM2 (also called the trapezoidal rule when $m = 1$):**

$$y_{i+1} = y_i + \frac{h}{12}(5f_{i+1} + 8f_i - f_{i-1})$$

- **Three-step AM3:**

$$y_{i+1} = y_i + \frac{h}{24}(9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$$

2.2.2 Derivation via Polynomial Interpolation

Multistep methods are derived by integrating the differential equation

$$y'(t) = f(t, y(t))$$

over a single step:

$$y(t_{i+1}) = y(t_i) + \int_{t_i}^{t_{i+1}} f(t, y(t)) dt.$$

Since the exact solution $y(t)$ is unknown, we approximate the integrand $f(t, y(t))$ by a polynomial $P(t)$ that interpolates known (or previously computed) values of f . The choice of interpolation points determines whether the resulting method is explicit or implicit.

Adams–Bashforth Two-Step Method (AB2)

To obtain an explicit method, we interpolate f at the two most recent *past* points: (t_{i-1}, f_{i-1}) and (t_i, f_i) , where $f_j = f(t_j, y_j)$.

The linear interpolating polynomial is

$$P_1(t) = f_{i-1} \frac{t_i - t}{h} + f_i \frac{t - t_{i-1}}{h}.$$

Integrating $P_1(t)$ from t_i to t_{i+1} gives:

$$\int_{t_i}^{t_{i+1}} P_1(t) dt = \frac{h}{2}(3f_i - f_{i-1}).$$

Substituting into the integral form yields the two-step Adams–Bashforth method:

$$y_{i+1} = y_i + \frac{h}{2}(3f_i - f_{i-1}).$$

This method is explicit (no f_{i+1} appears) and second-order accurate.

Algorithm 3 Adams–Bashforth 2-Step Method (AB2)

Require: Initial value y_0 , step size h , interval $[a, b]$, and function $f(t, y)$

Ensure: Approximations $y_1, y_2, \dots, y_N$

1: Compute y_1 using a *starting method* (e.g., RK2 or RK4) to ensure second-order accuracy.

2: **for** $i = 1$ to $N - 1$ **do**

3: Evaluate $f_i = f(t_i, y_i)$ and $f_{i-1} = f(t_{i-1}, y_{i-1})$

4: Update:

$$y_{i+1} = y_i + \frac{h}{2}(3f_i - f_{i-1})$$

5: **end for**

Adams–Moulton Two-Step Method (AM2)

For higher accuracy, we include the future point (t_{i+1}, f_{i+1}) in the interpolation. Using three points t_{i-1}, t_i, t_{i+1} we construct a quadratic interpolating polynomial $P_2(t)$.

Integrating $P_2(t)$ over $[t_i, t_{i+1}]$ results in:

$$\int_{t_i}^{t_{i+1}} P_2(t) dt = \frac{h}{12}(5f_{i+1} + 8f_i - f_{i-1}).$$

Thus, we obtain the two-step Adams–Moulton method:

$$y_{i+1} = y_i + \frac{h}{12}(5f_{i+1} + 8f_i - f_{i-1}).$$

This method is implicit (since $f_{i+1} = f(t_{i+1}, y_{i+1})$ depends on the unknown y_{i+1}) and is third-order accurate.

2.2.3 Local Truncation Error (LTE)

For a multistep method applied to the ODE $y' = f(t, y)$, the **local truncation error** τ_{i+1} is defined as the error made in one step, assuming all previous values are exact:

$$\tau_{i+1} = \frac{1}{h} [y(t_{i+1}) - (\text{numerical update using exact } y(t_i), y(t_{i-1}), \dots)].$$

The method is said to be of **order** p if $\tau_{i+1} = \mathcal{O}(h^p)$.

Algorithm 4 Adams–Moulton Three-Level Method

Require: y_{i-1}, y_i , step size h , function f **Ensure:** y_{i+1} 1: Predict $y_{i+1}^{(0)}$ (e.g., using AB2):

$$y_{i+1}^{(0)} = y_i + \frac{h}{2}(3f_i - f_{i-1})$$

2: **Corrector (fixed-point or Newton):**3: **repeat**4: $f_{i+1}^{(k)} = f(t_{i+1}, y_{i+1}^{(k)})$

5: Update:

$$y_{i+1}^{(k+1)} = y_i + \frac{h}{12}(5f_{i+1}^{(k)} + 8f_i - f_{i-1})$$

6: **until** convergence7: Set $y_{i+1} \leftarrow y_{i+1}^{(k+1)}$

LTE of the Adams–Bashforth Two-Step (AB2) Method

The AB2 method is:

$$y_{i+1} = y_i + \frac{h}{2}(3f_i - f_{i-1}).$$

Assume $y(t)$ is the exact solution. The LTE is:

$$\tau_{i+1}^{(\text{AB2})} = \frac{1}{h} \left[y(t_{i+1}) - y(t_i) - \frac{h}{2}(3y'(t_i) - y'(t_{i-1})) \right].$$

Expand $y(t_{i+1})$ and $y'(t_{i-1})$ in Taylor series about t_i :

$$\begin{aligned} y(t_{i+1}) &= y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(t_i) + \frac{h^3}{6}y'''(t_i) + \frac{h^4}{24}y^{(4)}(t_i) + \mathcal{O}(h^5), \\ y'(t_{i-1}) &= y'(t_i) - hy''(t_i) + \frac{h^2}{2}y'''(t_i) - \frac{h^3}{6}y^{(4)}(t_i) + \mathcal{O}(h^4). \end{aligned}$$

Substitute into the LTE expression:

$$\begin{aligned}
\tau_{i+1}^{(\text{AB2})} &= \frac{1}{h} \left[\left(hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} \right) - \frac{h}{2} \left(3y' - \left[y' - hy'' + \frac{h^2}{2}y''' - \frac{h^3}{6}y^{(4)} \right] \right) + \mathcal{O}(h^4) \right] \\
&= \frac{1}{h} \left[hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} - \frac{h}{2} \left(2y' + hy'' - \frac{h^2}{2}y''' + \frac{h^3}{6}y^{(4)} \right) + \mathcal{O}(h^4) \right] \\
&= \frac{1}{h} \left[hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} - hy' - \frac{h^2}{2}y'' + \frac{h^3}{4}y''' - \frac{h^4}{12}y^{(4)} + \mathcal{O}(h^4) \right] \\
&= \frac{1}{h} \left(\frac{5h^3}{12}y''' + \mathcal{O}(h^4) \right) = \frac{5}{12}h^2y'''(t_i) + \mathcal{O}(h^3).
\end{aligned}$$

Thus,

$$\tau_{i+1}^{(\text{AB2})} = \mathcal{O}(h^2),$$

so the AB2 method is second-order accurate (global error $\mathcal{O}(h^2)$), with LTE $\mathcal{O}(h^3)$.

2.2.4 LTE of the Adams–Moulton Two-Step (AM2) Method

The AM2 method is:

$$y_{i+1} = y_i + \frac{h}{12}(5f_{i+1} + 8f_i - f_{i-1}).$$

The LTE is:

$$\tau_{i+1}^{(\text{AM2})} = \frac{1}{h} \left[y(t_{i+1}) - y(t_i) - \frac{h}{12}(5y'(t_{i+1}) + 8y'(t_i) - y'(t_{i-1})) \right].$$

Use Taylor expansions about t_i :

$$\begin{aligned}
y(t_{i+1}) &= y + hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} + \mathcal{O}(h^5), \\
y'(t_{i+1}) &= y' + hy'' + \frac{h^2}{2}y''' + \frac{h^3}{6}y^{(4)} + \mathcal{O}(h^4), \\
y'(t_{i-1}) &= y' - hy'' + \frac{h^2}{2}y''' - \frac{h^3}{6}y^{(4)} + \mathcal{O}(h^4),
\end{aligned}$$

where all derivatives are evaluated at t_i . Now compute the bracketed term:

$$\begin{aligned}
& y(t_{i+1}) - y(t_i) - \frac{h}{12}(5y'(t_{i+1}) + 8y'(t_i) - y'(t_{i-1})) \\
&= \left(hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} \right) \\
&\quad - \frac{h}{12} \left[5 \left(y' + hy'' + \frac{h^2}{2}y''' + \frac{h^3}{6}y^{(4)} \right) + 8y' - \left(y' - hy'' + \frac{h^2}{2}y''' - \frac{h^3}{6}y^{(4)} \right) \right] + \mathcal{O}(h^5) \\
&= hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} \\
&\quad - \frac{h}{12} \left[(5y' + 8y' - y') + (5hy'' + hy'') + \left(\frac{5h^2}{2}y''' - \frac{h^2}{2}y''' \right) + \left(\frac{5h^3}{6}y^{(4)} + \frac{h^3}{6}y^{(4)} \right) \right] + \mathcal{O}(h^5) \\
&= hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} - \frac{h}{12} [12y' + 6hy'' + 2h^2y''' + h^3y^{(4)}] + \mathcal{O}(h^5) \\
&= hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} - hy' - \frac{h^2}{2}y'' - \frac{h^3}{6}y''' - \frac{h^4}{12}y^{(4)} + \mathcal{O}(h^5) \\
&= -\frac{h^4}{24}y^{(4)} + \mathcal{O}(h^5).
\end{aligned}$$

Therefore,

$$\tau_{i+1}^{(\text{AM2})} = \frac{1}{h} \left(-\frac{h^4}{24}y^{(4)} + \mathcal{O}(h^5) \right) = -\frac{1}{24}h^3y^{(4)}(t_i) + \mathcal{O}(h^4).$$

Hence,

$$\tau_{i+1}^{(\text{AM2})} = \mathcal{O}(h^3),$$

so the AM2 method is third-order accurate (global error $\mathcal{O}(h^3)$), with LTE $\mathcal{O}(h^4)$.

2.2.5 Common Adams Methods

The table below summarizes frequently used Adams–Bashforth (explicit) and Adams–Moulton (implicit) methods, along with their local truncation error (LTE) orders.

Method	Formula	LTE
AB2	$y_{i+1} = y_i + \frac{h}{2}(3f_i - f_{i-1})$	$\mathcal{O}(h^3)$
AB3	$y_{i+1} = y_i + \frac{h}{12}(23f_i - 16f_{i-1} + 5f_{i-2})$	$\mathcal{O}(h^4)$
AB4	$y_{i+1} = y_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$	$\mathcal{O}(h^5)$
AM2	$y_{i+1} = y_i + \frac{h}{12}(5f_{i+1} + 8f_i - f_{i-1})$	$\mathcal{O}(h^3)$
AM3	$y_{i+1} = y_i + \frac{h}{24}(9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$	$\mathcal{O}(h^4)$

Note:

- The order of accuracy is one higher than the LTE exponent (e.g., LTE $\mathcal{O}(h^3)$ global error $\mathcal{O}(h^2)$).

Example 2.7. Apply AB2, AB3 and AB4 methods to the IVP:

$$y' = y - t^2 + 1, \quad y(0) = 0.5, \quad t \in [0, 2], \quad h = 0.2$$

Exact solution: $y(t) = (t + 1)^2 - 0.5e^t$.

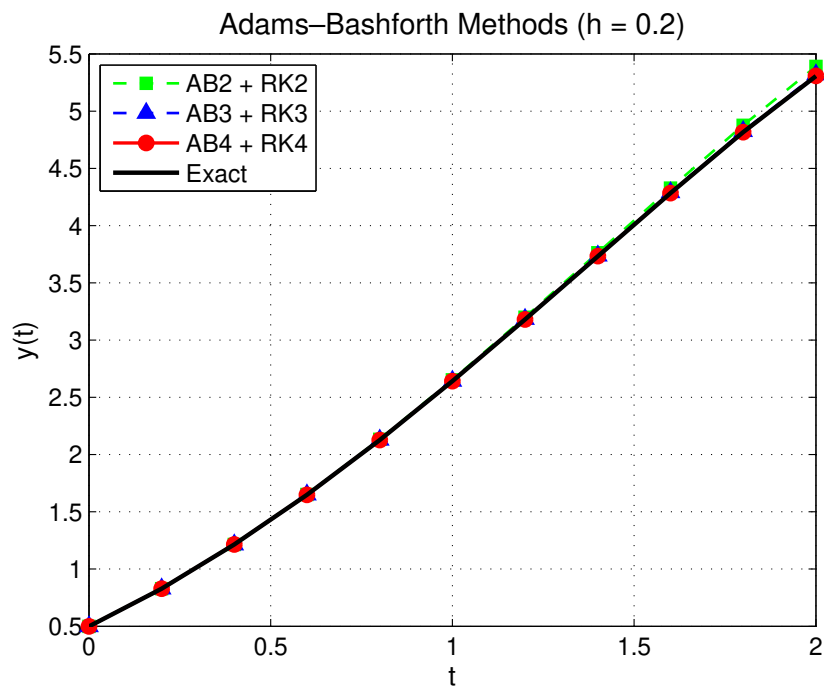
Table 2.8: Comparison of Adams–Bashforth methods (step size $h = 0.2$).

t	Exact	AB2	err _{AB2}	AB3	err _{AB3}	AB4	err _{AB4}
0.0	0.500000	0.500000	0.00e+00	0.500000	0.00e+00	0.500000	0.00e+00
0.2	0.829299	0.828000	1.30e-3	0.829200	9.86e-5	0.829293	5.29e-6
0.4	1.214088	1.214400	3.12e-4	1.213876	2.11e-4	1.214076	1.14e-5
0.6	1.648941	1.651920	2.98e-3	1.649076	1.35e-4	1.648922	1.86e-5
0.8	2.127230	2.134056	6.83e-3	2.127954	7.25e-4	2.127289	5.97e-5
1.0	2.640859	2.653081	1.22e-2	2.642406	1.55e-3	2.641053	1.94e-4
1.2	3.179942	3.199599	1.97e-2	3.182630	2.69e-3	3.180314	3.73e-4
1.4	3.732400	3.762171	2.98e-2	3.736660	4.26e-3	3.733019	6.19e-4
1.6	4.283484	4.326863	4.34e-2	4.289878	6.39e-3	4.284442	9.59e-4
1.8	4.815176	4.876704	6.15e-2	4.824442	9.27e-3	4.816596	1.42e-3
2.0	5.305472	5.391029	8.56e-2	5.318565	1.31e-2	5.307508	2.04e-3

Observation: Higher-order methods yield significantly smaller errors.

Matlab Codes of AB Methods

```
function [t, y] = ab2_with_rk2_start(fun, tspan, y0, h)
% AB2_WITH_RK2_START: Solve IVP using 2-step Adams-Bashforth
% with second-order Runge--Kutta (midpoint method) to compute y1.
% The starter is the explicit midpoint method:
%   k1 = f(t_i, y_i)
%   k2 = f(t_i + h/2, y_i + (h/2)*k1)
%   y_{i+1} = y_i + h * k2
% This is a genuine RK2 method, not cast in predictor-corrector form.
t0 = tspan(1); tf = tspan(2);
N = round((tf - t0) / h);
t = t0 + (0:N)*h;
```



```

y = zeros(1, N+1);
y(1) = y0;
% ===== Step 1: Compute y(2) using RK2 (midpoint method) =====
ti = t(1); yi = y(1);
k1 = fun(ti, yi);
k2 = fun(ti + h/2, yi + (h/2)*k1);
y(2) = yi + h * k2; % Note: no averaging - pure RK2
% ===== Step 2: Apply AB2 for i = 2 to N =====
for i = 2:N
    f_i = fun(t(i), y(i));
    f_im1 = fun(t(i-1), y(i-1));
    y(i+1) = y(i) + (h/2) * (3*f_i - f_im1);
end
end

```

```

function [t, y] = ab3_with_rk3_start(fun, tspan, y0, h)
% AB3_WITH_RK3_START Solve IVP using 3-step Adams-Bashforth (AB3)
% with third-order Runge--Kutta (Kutta's RK3) to generate y(2) and y(3).
% Inputs:
%   fun    - function handle f(t, y)
%   tspan  - [t0, tf]
%   y0     - initial condition y(t0) = y0
%   h      - step size

```

```

% Outputs:
%   t       - time vector: [t0, t1, ..., tf]
%   y       - numerical solution at times t
t0 = tspan(1); tf = tspan(2);
N = round((tf - t0) / h);
t = t0 + (0:N)*h;
y = zeros(1, N+1);
y(1) = y0;
% ===== Generate y(2) and y(3) using Kutta's RK3 (order 3) =====
for i = 1:2
    ti = t(i);
    yi = y(i);
    % Kutta's third-order Runge--Kutta method
    k1 = fun(ti, yi);
    k2 = fun(ti + h/2, yi + (h/2)*k1);
    k3 = fun(ti + h, yi - h*k1 + 2*h*k2);

    y(i+1) = yi + (h/6) * (k1 + 4*k2 + k3);
end
% ===== Apply AB3 from i = 3 to N (compute y(4) to y(N+1)) =====
for i = 3:N
    f_i = fun(t(i), y(i));
    f_im1 = fun(t(i-1), y(i-1));
    f_im2 = fun(t(i-2), y(i-2));

    y(i+1) = y(i) + (h/12) * (23*f_i - 16*f_im1 + 5*f_im2);
end
end

```

```

function [t, y] = ab4_with_rk4_start(fun, tspan, y0, h)
% AB4_WITH_RK4_START Solve IVP using 4-step Adams-Bashforth method
% with RK4 to generate starting values.
% Inputs:
%   fun      - function handle f(t, y)
%   tspan    - [t0, tf]
%   y0       - initial condition y(t0) = y0
%   h        - step size (must divide (tf - t0) evenly or be compatible)
% Outputs:
%   t        - time vector [t0, t1, ..., tf]
%   y        - numerical solution at times t
t0 = tspan(1); tf = tspan(2);
N = round((tf - t0) / h); % total number of steps

```



```

t = t0 + (0:N)*h;           % time grid: t0, t0+h, ..., tf
y = zeros(1, N+1);
y(1) = y0;
% Step 1: Use RK4 to compute y(2), y(3), y(4) (i.e., y1, y2, y3)
for i = 1:3
    ti = t(i);
    yi = y(i);
    % Classical RK4
    k1 = fun(ti, yi);
    k2 = fun(ti + h/2, yi + h*k1/2);
    k3 = fun(ti + h/2, yi + h*k2/2);
    k4 = fun(ti + h, yi + h*k3);
    y(i+1) = yi + h/6 * (k1 + 2*k2 + 2*k3 + k4);
end
% Step 2: Apply AB4 for steps 4 to N (compute y(5) to y(N+1))
for i = 4:N
    f_i   = fun(t(i), y(i));
    f_im1 = fun(t(i-1), y(i-1));
    f_im2 = fun(t(i-2), y(i-2));
    f_im3 = fun(t(i-3), y(i-3));
    y(i+1) = y(i) + h/24 * (55*f_i - 59*f_im1 + 37*f_im2 - 9*f_im3);
end
end

```

Driver for AB Methods

```

% Define the ODE
f = @(t, y) y - t^2 + 1;
% Exact solution
y_exact = @(t) (t + 1).^2 - 0.5 * exp(t);
% Parameters
tspan = [0, 2];
y0 = 0.5;
h = 0.2;
% Precompute time vector (needed before evaluating exact solution)
N = round((tspan(2) - tspan(1)) / h);
t = tspan(1) + (0:N)*h; % t = [0.0, 0.2, ..., 2.0]
% Evaluate exact solution on the grid
Y = y_exact(t); % Now t is defined!
% Solve using AB2 (RK2 start), AB3 (RK3 start), AB4 (RK4 start)
[~, y_ab2] = ab2_with_rk2_start(f, tspan, y0, h);

```

```

[~, y_ab3] = ab3_with_rk3_start(f, tspan, y0, h);
[~, y_ab4] = ab4_with_rk4_start(f, tspan, y0, h);
% Compute absolute errors
err_ab2 = abs(y_ab2 - Y);
err_ab3 = abs(y_ab3 - Y);
err_ab4 = abs(y_ab4 - Y);
% Display errors at all time points
fprintf(' t      |   Exact   |   AB2     |   err_AB2  |   AB3     |   err_AB3  |
        AB4     |   err_AB4  \n');
fprintf('-----|-----|-----|-----|-----|---\n');
for i = 1:length(t)
    fprintf('%4.1f | %9.6f | %9.6f | %9.2e | %9.6f | %9.2e | %9.6f | %9.2e\n',
        ...
t(i), Y(i), y_ab2(i), err_ab2(i), y_ab3(i), err_ab3(i), y_ab4(i), err_ab4(i));
end
% Plot all solutions
figure;
plot(t, y_ab2, 'gs--', 'LineWidth', 1.2, 'MarkerFaceColor', 'g', 'DisplayName',
    , 'AB2 + RK2');
hold on;
plot(t, y_ab3, 'b^--', 'LineWidth', 1.2, 'MarkerFaceColor', 'b', 'DisplayName',
    , 'AB3 + RK3');
plot(t, y_ab4, 'ro-', 'LineWidth', 1.5, 'MarkerFaceColor', 'r', 'DisplayName',
    , 'AB4 + RK4');
plot(t, Y, 'k-', 'LineWidth', 2.0, 'MarkerFaceColor', 'k', 'DisplayName',
    , 'Exact'); hold off;
xlabel('t', 'FontSize', 12); ylabel('y(t)', 'FontSize', 12);
title(sprintf('-AdamsBashforth Methods (h = %.1f)', h), 'FontSize', 13);
legend('Location', 'northwest'); grid on; set(gca, 'FontSize', 11);

```

Exercise 2.5 (Lab work). Use $AB2$, $AB3$, $AB4$ methods with starter Modified Euler, $RK3$ and $RK4$ to approximate the solutions for each of the following IVPs. Modify the above codes for each cases.

a. $y' = \frac{2-2ty}{t^2+1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \frac{2t+1}{t^2+1}$

b. $y' = \frac{y^2}{1+t}$, $1 \leq t \leq 2$, $y(1) = -(\ln 2)^{-1}$, with $h = 0.1$. the exact solution is $y(t) = -\frac{1}{\ln(1+t)}$

c. $y' = \frac{y^2+y}{t}$, $1 \leq t \leq 3$, $y(1) = -2$, with $h = 0.2$. the exact solution is $y(t) = \frac{2t}{1-2t}$

d. $y' = -ty + 4ty^{-1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \sqrt{4 - 3e^{-t^2}}$

2.2.6 Predictor–Corrector Methods

Implicit multistep methods (e.g., Adams–Moulton) offer high accuracy and favorable stability properties, yet their direct implementation requires solving nonlinear algebraic equations of the form

$$y_{i+1} = y_i + h \sum_{j=0}^k \beta_j f(t_{i+1-j}, y_{i+1-j}),$$

which generally necessitates iterative solvers (e.g., Newton’s method). For stiff or large-scale systems, this can be computationally expensive and sensitive to initial guesses.

To circumvent this difficulty while retaining high-order accuracy, **predictor–corrector methods** combine:

- an *explicit* method (the **predictor**) to furnish an initial estimate $y_{i+1}^{(p)}$,
- followed by a *single evaluation* of an **implicit** method (the **corrector**) using the predicted value to approximate the unknown $f(t_{i+1}, y_{i+1})$.

This strategy avoids iterative solution of nonlinear equations, at the cost of slightly reduced stability compared to fully implicit schemes.

AB4–AM4 Predictor–Corrector Pair

A widely used fourth-order predictor–corrector pair is the combination of the 4-step Adams–Bashforth (AB4) predictor and the 3-step Adams–Moulton (AM3) corrector. Despite the notation “AB4–AM4” sometimes appearing in literature, the standard classical implementation uses **AB4 for prediction and AM3 (3-step, order 4) for correction**, yielding an overall method of order 4.

Properties:

- *Order of accuracy*: Global error $\mathcal{O}(h^4)$, matching that of classical RK4.
- *Efficiency*: Requires only **two** new function evaluations per step (one for $f_{i+1}^{(p)}$, and optionally one for f_{i+1} if cached for future steps), compared to four for RK4.
- *Startup*: Requires four initial values y_0, y_1, y_2, y_3 , typically generated by a self-starting method (e.g., RK4).

Algorithm 5 Adams–Bashforth–Moulton Predictor–Corrector (AB4–AM3)

Require: $y_{i-3}, y_{i-2}, y_{i-1}, y_i$, step size h , and $f_j := f(t_j, y_j)$ for $j = i-3, \dots, i$

Ensure: Updated approximation y_{i+1}

1: **Predictor (AB4, explicit, order 4):**

$$y_{i+1}^{(p)} = y_i + \frac{h}{24} (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

2: Evaluate predicted slope:

$$f_{i+1}^{(p)} = f(t_{i+1}, y_{i+1}^{(p)})$$

3: **Corrector (AM3, implicit, order 4):**

$$y_{i+1} = y_i + \frac{h}{24} (9f_{i+1}^{(p)} + 19f_i - 5f_{i-1} + f_{i-2})$$

4: (Optional) Update $f_{i+1} \leftarrow f(t_{i+1}, y_{i+1})$ for next step.

- *Stability:* Region of absolute stability is larger than AB4 alone, but smaller than the fully implicit AM4; suitable for non-stiff or mildly stiff problems.

This approach exemplifies a pragmatic compromise: it achieves high-order accuracy with modest computational overhead, making it especially attractive for long-time integrations of non-stiff ODEs where function evaluations dominate the cost.

Summary

- Multistep methods use past values for higher efficiency.
- Adams–Bashforth: explicit; Adams–Moulton: implicit.
- Predictor-corrector methods combine both for accuracy and stability.
- Systems of ODEs are solved using vectorized versions of scalar methods.
- RK4 is significantly more accurate than Euler for systems.

Exercise 2.6. Implement the AB4-AM4 predictor-corrector method in MATLAB for the IVP:

$$y' = y - t^2 + 1, \quad y(0) = 0.5, \quad h = 0.2.$$

Use RK4 to generate starting values.

```
%% AB4-AM4 Predictor-Corrector Method
clear; clc; close all;
% Define the ODE: y' = f(t, y) = y - t^2 + 1
f = @(t, y) y - t.^2 + 1;
% Exact solution (for comparison): y(t) = (t + 1)^2 - 0.5*exp(t)
y_exact = @(t) (t + 1).^2 - 0.5 * exp(t);
% Parameters
t0 = 0; tf = 2;          % Interval [0, 2]
h = 0.2;                 % Step size
N = round((tf - t0)/h); % Number of steps
% Time vector
t = t0:h:tf;
if length(t) ~= N+1
    t = t0 + (0:N)*h; % Ensure exact length
end
% Preallocate solution vector
y = zeros(1, N+1);
y(1) = 0.5; % Initial condition y(0) = 0.5
%% Step 1: Generate starting values y(1) to y(4) using RK4
% We need y0, y1, y2, y3 (i.e., first 4 values) for AB4/AM4
for i = 1:3 % Compute y(2), y(3), y(4)
    ti = t(i);
    yi = y(i);

    k1 = f(ti, yi);
    k2 = f(ti + h/2, yi + h*k1/2);
    k3 = f(ti + h/2, yi + h*k2/2);
    k4 = f(ti + h, yi + h*k3);

    y(i+1) = yi + h*(k1 + 2*k2 + 2*k3 + k4)/6;
end
%% Step 2: AB4-AM4 Predictor-Corrector for i = 4 to N
for i = 4:N
    % --- Predictor: AB4 ---
    f_i = f(t(i), y(i));
    f_im1 = f(t(i-1), y(i-1));
    f_im2 = f(t(i-2), y(i-2));
    f_im3 = f(t(i-3), y(i-3));
    y_pred = y(i) + h/24 * (55*f_i - 59*f_im1 + 37*f_im2 - 9*f_im3);
    % --- Corrector: AM4 ---
```

```

    f_pred = f(t(i+1), y_pred); % f_{i+1} using predicted y
    y_corr = y(i) + h/24 * (9*f_pred + 19*f_i - 5*f_im1 + f_im2);
    % Store corrected value
    y(i+1) = y_corr;
end
%% Step 3: Compare with exact solution
y_true = y_exact(t);
error = abs(y - y_true);
%% Step 4: Display results in a table
fprintf(' t      | y_AB4-AM4      | y_exact      | |error|\n');
for i = 1:length(t)
    fprintf('%4.1f | %12.8f | %12.8f | %8.2e\n', ...
        t(i), y(i), y_true(i), error(i));
end

%% Step 5: Plot results
figure;
plot(t, y_true, 'k-', 'LineWidth', 2); hold on;
plot(t, y, 'bo--', 'MarkerFaceColor', 'b');
xlabel('t'); ylabel('y(t)');
title('AB4-AM4 Predictor-Corrector vs Exact Solution');
legend('Exact', 'AB4-AM4', 'Location', 'best');
grid on;

```

Table 2.9: AB4-AM4 Predictor-Corrector method.

t	$y_{AB4-AM4}$	y_{exact}	$ error $
0.0	0.500000000	0.500000000	0.00e+00
0.2	0.82929333	0.82929862	5.29e-06
0.4	1.21407621	1.21408765	1.14e-05
0.6	1.64892202	1.64894060	1.86e-05
0.8	2.12720563	2.12722954	2.39e-05
1.0	2.64082860	2.64085909	3.05e-05
1.2	3.17990264	3.17994154	3.89e-05
1.4	3.73235048	3.73240002	4.95e-05
1.6	4.28342082	4.28348379	6.30e-05
1.8	4.81509636	4.81517627	7.99e-05
2.0	5.30537067	5.30547195	1.01e-04

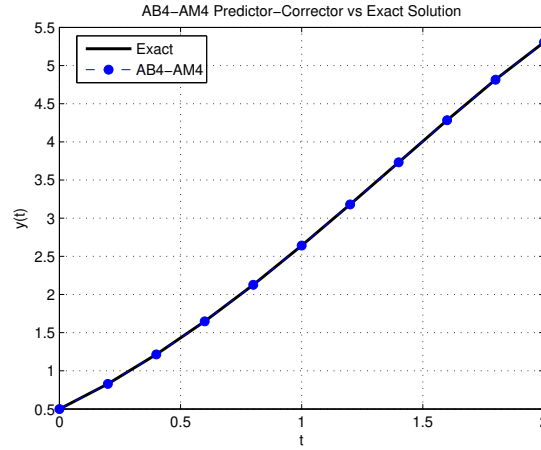


Figure 2.6: Predictor corrector method.

Exercise 2.7 (Lab work). Apply Predictor-corrector method to approximate the solutions of each of the following IVPs. Modify the above codes for each cases.

a. $y' = \frac{2-2ty}{t^2+1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \frac{2t+1}{t^2+1}$

b. $y' = \frac{y^2}{1+t}$, $1 \leq t \leq 2$, $y(1) = -(\ln 2)^{-1}$, with $h = 0.1$. the exact solution is $y(t) = -\frac{1}{\ln(1+t)}$

c. $y' = \frac{y^2+y}{t}$, $1 \leq t \leq 3$, $y(1) = -2$, with $h = 0.2$. the exact solution is $y(t) = \frac{2t}{1-2t}$

d. $y' = -ty + 4ty^{-1}$, $0 \leq t \leq 1$, $y(0) = 1$, with $h = 0.1$. the exact solution is $y(t) = \sqrt{4 - 3e^{-t^2}}$

2.3 Systems of Differential Equations

Many physical, biological, and engineering models involve multiple interdependent quantities. Such problems naturally lead to *systems* of ordinary differential equations (ODEs). A general system of m first-order ODEs has the form:

$$\begin{aligned} y_1'(t) &= f_1(t, y_1(t), \dots, y_m(t)), & y_1(a) &= \alpha_1, \\ y_2'(t) &= f_2(t, y_1(t), \dots, y_m(t)), & y_2(a) &= \alpha_2, \\ &\vdots & &\vdots \\ y_m'(t) &= f_m(t, y_1(t), \dots, y_m(t)), & y_m(a) &= \alpha_m. \end{aligned}$$

Introducing vector notation simplifies the representation. Define the solution vector and the right-hand side vector function as:

$$\mathbf{Y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_m(t) \end{bmatrix}, \quad \mathbf{F}(t, \mathbf{Y}) = \begin{bmatrix} f_1(t, \mathbf{Y}) \\ f_2(t, \mathbf{Y}) \\ \vdots \\ f_m(t, \mathbf{Y}) \end{bmatrix}.$$

Then the system becomes the compact initial-value problem:

$$\mathbf{Y}'(t) = \mathbf{F}(t, \mathbf{Y}(t)), \quad \mathbf{Y}(a) = \mathbf{Y}_0,$$

where $\mathbf{Y}_0 = [\alpha_1, \alpha_2, \dots, \alpha_m]^\top$.

2.3.1 Euler's Method for Systems

Given a step size $h > 0$ and grid points $t_i = a + ih$, Euler's method extends naturally to systems:

$$\mathbf{Y}_{i+1} = \mathbf{Y}_i + h \mathbf{F}(t_i, \mathbf{Y}_i).$$

This is applied component-wise and is simple to implement, though it is only first-order accurate.

2.3.2 Fourth-Order Runge–Kutta Method for Systems

The classical fourth-order Runge–Kutta (RK4) method provides significantly higher accuracy. The update formula is:

$$\mathbf{Y}_{i+1} = \mathbf{Y}_i + \frac{1}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4),$$

where the stage vectors are computed as:

$$\begin{aligned}
\mathbf{K}_1 &= h \mathbf{F}(t_i, \mathbf{Y}_i), \\
\mathbf{K}_2 &= h \mathbf{F}\left(t_i + \frac{h}{2}, \mathbf{Y}_i + \frac{\mathbf{K}_1}{2}\right), \\
\mathbf{K}_3 &= h \mathbf{F}\left(t_i + \frac{h}{2}, \mathbf{Y}_i + \frac{\mathbf{K}_2}{2}\right), \\
\mathbf{K}_4 &= h \mathbf{F}(t_i + h, \mathbf{Y}_i + \mathbf{K}_3).
\end{aligned}$$

This method is fourth-order accurate and widely used for non-stiff systems.

Example 2.8. Consider the linear system:

$$\begin{aligned}
x'(t) &= x(t) + 4y(t) - e^t, \quad x(0) = 4, \\
y'(t) &= x(t) + y(t) + 2e^t, \quad y(0) = 1.25.
\end{aligned}$$

This system admits the exact (analytical) solution:

$$\begin{aligned}
x(t) &= 4e^{3t} + 2e^{-t} - 2e^t, \\
y(t) &= 2e^{3t} - e^{-t} + \frac{1}{4}e^t.
\end{aligned}$$

This example is ideal for testing numerical methods (e.g., Euler or RK4), as the true solution is known and can be used to compute errors and verify convergence rates.

Define the state vector $\mathbf{Y}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$ and the right-hand side function

$$\mathbf{F}(t, \mathbf{Y}) = \begin{bmatrix} x + 4y - e^t \\ x + y + 2e^t \end{bmatrix}.$$

The following table compares Euler's method and the classical fourth-order Runge–Kutta (RK4) method for solving this system numerically.

Both methods are applied iteratively from $t_0 = 0$ to $t_f = 1$ with step size h , starting from $\mathbf{Y}_0 = [4, 1.25]^\top$. While Euler's method is intuitive and easy to code, RK4 is preferred in most applications due to its superior accuracy and stability for smooth problems.

Table 2.10: Comparison of Euler and RK4 Methods for the Given System

Feature	Euler's Method	Runge-Kutta (RK4)
Update Formula	$\mathbf{Y}_{i+1} = \mathbf{Y}_i + h \mathbf{F}(t_i, \mathbf{Y}_i)$	$\mathbf{Y}_{i+1} = \mathbf{Y}_i + \frac{1}{6} (\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4)$
Stage Computations	Single evaluation per step: $\mathbf{K}_1 = h \mathbf{F}(t_i, \mathbf{Y}_i)$	Four evaluations per step: $\mathbf{K}_1 = h \mathbf{F}(t_i, \mathbf{Y}_i),$ $\mathbf{K}_2 = h \mathbf{F}(t_i + \frac{h}{2}, \mathbf{Y}_i + \frac{\mathbf{K}_1}{2}),$ $\mathbf{K}_3 = h \mathbf{F}(t_i + \frac{h}{2}, \mathbf{Y}_i + \frac{\mathbf{K}_2}{2}),$ $\mathbf{K}_4 = h \mathbf{F}(t_i + h, \mathbf{Y}_i + \mathbf{K}_3).$
Order of Accuracy	First-order ($\mathcal{O}(h)$)	Fourth-order ($\mathcal{O}(h^4)$)
Computational Cost per Step	1 evaluation of $\mathbf{F}$	4 evaluations of $\mathbf{F}$
Error (for $h = 0.05$)	Larger global error	Very small global error
Simplicity	Very simple	Moderately more complex, but standard in practice

MATLAB Implementation

```

% Solve the system:
%   x' = x + 4*y - exp(t),    x(0) = 4
%   y' = x + y + 2*exp(t),    y(0) = 1.25
% Exact solution:
%   x(t) = 4*exp(3*t) + 2*exp(-t) - 2*exp(t)
%   y(t) = 2*exp(3*t) - exp(-t) + 0.25*exp(t)
clear; clc; close all;
% Interval and step size
t0 = 0; tf = 1;
h = 0.05; % Step size
N = round((tf - t0)/h);
t = linspace(t0, tf, N+1);
% Initial condition: [x; y]
Y0 = [4; 1.25];
% Preallocate solution arrays
Y_euler = zeros(2, N+1);
Y_rk4 = zeros(2, N+1);
Y_euler(:,1) = Y0;
Y_rk4(:,1) = Y0;

```

```

% Define the RHS function F(t, Y) = [x'; y']
F = @(t, Y) [ ...
    Y(1) + 4*Y(2) - exp(t); ...
    Y(1) + Y(2) + 2*exp(t) ...
];

% --- Euler's Method ---
for i = 1:N
    Y_euler(:,i+1) = Y_euler(:,i) + h * F(t(i), Y_euler(:,i));
end

% --- RK4 Method ---
for i = 1:N
    K1 = F(t(i), Y_rk4(:,i));
    K2 = F(t(i) + h/2, Y_rk4(:,i) + h*K1/2);
    K3 = F(t(i) + h/2, Y_rk4(:,i) + h*K2/2);
    K4 = F(t(i) + h, Y_rk4(:,i) + h*K3);
    Y_rk4(:,i+1) = Y_rk4(:,i) + h*(K1 + 2*K2 + 2*K3 + K4)/6;
end

% Exact solution
x_exact = 4*exp(3*t) + 2*exp(-t) - 2*exp(t);
y_exact = 2*exp(3*t) - exp(-t) + 0.25*exp(t);

% Plot x(t)
figure;
plot(t, x_exact, 'k-', 'LineWidth', 2); hold on;
plot(t, Y_euler(1,:), 'bo--', 'MarkerSize', 4);
plot(t, Y_rk4(1,:), 'rs--', 'MarkerSize', 4);
xlabel('t'); ylabel('x(t)');
legend('Exact', 'Euler', 'RK4', 'Location', 'northwest');
title('Solution for x(t)');
grid on;

% Plot y(t)
figure;
plot(t, y_exact, 'k-', 'LineWidth', 2); hold on;
plot(t, Y_euler(2,:), 'bo--', 'MarkerSize', 4);
plot(t, Y_rk4(2,:), 'rs--', 'MarkerSize', 4);
xlabel('t'); ylabel('y(t)');
legend('Exact', 'Euler', 'RK4', 'Location', 'northwest');
title('Solution for y(t)');
grid on;

% Compute max errors
err_euler_x = max(abs(Y_euler(1,:) - x_exact));
err_rk4_x = max(abs(Y_rk4(1,:) - x_exact));

```

```

err_euler_y = max(abs(Y_euler(2,:) - y_exact));
err_rk4_y    = max(abs(Y_rk4(2,:)   - y_exact));
fprintf('Max error in x (Euler): %.4e\n', err_euler_x);
fprintf('Max error in x (RK4):   %.4e\n', err_rk4_x);
fprintf('Max error in y (Euler): %.4e\n', err_euler_y);
fprintf('Max error in y (RK4):   %.4e\n', err_rk4_y);

```

Table 2.11: Maximum Absolute Errors on $t \in [0, 1]$ with Step Size $h = 0.05$

Method	Max Error in $x(t)$	Max Error in $y(t)$
Euler System	1.4726e+01	7.3812e+00
RK4 System	8.8820e-04	4.4399e-04

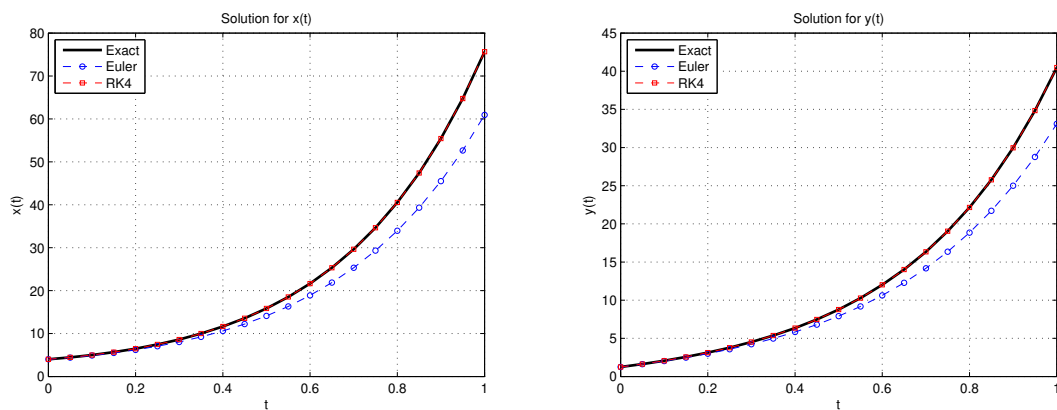


Figure 2.7: Comparison of Euler method and RK4 method.

2.3.3 Higher-Order Equations and Systems of IVPs

Consider the initial-value problem for an m -th order differential equation:

$$\begin{aligned} y^{(m)} &= f(t, y, y', y'', \dots, y^{(m-1)}), & a \leq t \leq b \\ y(a) &= \alpha_1, \quad y'(a) = \alpha_2, \quad \dots, \quad y^{(m-1)}(a) = \alpha_m \end{aligned}$$

To apply numerical methods designed for systems of first-order equations, we convert this higher-order ODE into a system of m first-order differential equations.

Substitution to Form a System: Define new variables:

$$\begin{aligned} y_1 &= y \\ y_2 &= y' \\ y_3 &= y'' \\ &\vdots \\ y_m &= y^{(m-1)} \end{aligned}$$

Then the system becomes:

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= y_3 \\ y_3' &= y_4 \\ &\vdots \\ y_{m-1}' &= y_m \\ y_m' &= f(t, y_1, y_2, \dots, y_m) \end{aligned}$$

with the corresponding initial conditions:

$$y_1(a) = \alpha_1, \quad y_2(a) = \alpha_2, \quad \dots, \quad y_m(a) = \alpha_m$$

This system can now be solved using standard numerical methods such as Euler's method or the Runge–Kutta method for systems of equations.

Example 2.9. *Convert the initial-value problem*

$$y''(t) + 4y'(t) + 3y(t) = 0, \quad y(0) = 1.5, \quad y'(0) = 2.5,$$

into a system of first-order differential equations, and describe how to apply Euler's method and the fourth-order Runge–Kutta (RK4) method to approximate the solution.

Solution: We begin by solving the given second-order linear homogeneous ODE for the highest derivative:

$$y''(t) = -3y(t) - 4y'(t).$$

To reduce the order, introduce new state variables:

$$y_1(t) := y(t),$$

$$y_2(t) := y'(t).$$

Differentiating these definitions and substituting yields the equivalent first-order system:

$$y_1'(t) = y_2(t),$$

$$y_2'(t) = -3y_1(t) - 4y_2(t).$$

The initial conditions transform accordingly:

$$y_1(0) = y(0) = 1.5, \quad y_2(0) = y'(0) = 2.5.$$

In vector form, define $\mathbf{Y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \end{bmatrix}$. Then the system becomes

$$\mathbf{Y}'(t) = \begin{bmatrix} 0 & 1 \\ -3 & -4 \end{bmatrix} \mathbf{Y}(t), \quad \mathbf{Y}(0) = \begin{bmatrix} 1.5 \\ 2.5 \end{bmatrix}.$$

Exact Solution: The characteristic equation of the original ODE is $r^2 + 4r + 3 = 0$, with roots $r = -1$ and $r = -3$. Hence, the exact solution is $y(t) = e^{-t} + e^{-3t}$.

Numerical Approximation

This first-order system is now ready for numerical integration. On a chosen interval (e.g., $t \in [0, 2]$) and with a step size $h > 0$, one may apply:

- **Euler's method:**

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h \mathbf{F}(t_n, \mathbf{Y}_n),$$

$$\text{where } \mathbf{F}(t, \mathbf{Y}) = \begin{bmatrix} y_2 \\ -3y_1 - 4y_2 \end{bmatrix}.$$

- **Classical RK4 method:**

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + \frac{1}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4),$$

with stage vectors computed as usual.

Students are encouraged to implement both methods in MATLAB and compare the numerical approximations of $y_1(t) \approx y(t)$ against the exact solution $y(t) = e^{-t} + e^{-3t}$. Plotting the results and computing the maximum absolute error will illustrate the superior accuracy of RK4 over Euler's method for the same step size.

Example 2.10. *Convert the initial-value problem:*

$$\sin(t) y''' + \cos(ty) y' + \sin(t^2 + y') + (y'')^3 = \log(t), \quad 2 \leq t \leq 3$$

with initial conditions:

$$y(2) = 7, \quad y'(2) = 3, \quad y''(2) = 4$$

into a system of first-order differential equations with initial values. This IVP does not have a known closed-form (exact) solution.

Solution: We begin by solving for the highest derivative:

$$y''' = \frac{1}{\sin(t)} [\log(t) - \cos(ty) y' - \sin(t^2 + y') - (y'')^3]$$

Now introduce new variables to reduce the order:

$$\begin{aligned} y_1 &= y \\ y_2 &= y' \\ y_3 &= y'' \end{aligned}$$

Then we can write the third-order equation as a system of three first-order equations:

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= y_3 \\ y_3' &= \frac{1}{\sin(t)} [\log(t) - \cos(ty_1) y_2 - \sin(t^2 + y_2) - y_3^3] \end{aligned}$$

The corresponding initial conditions are:

$$y_1(2) = 7$$

$$y_2(2) = 3$$

$$y_3(2) = 4$$

This system is now in a form suitable for numerical solution using methods such as Euler's method or the Runge-Kutta method for systems.

This MATLAB script below solves the third-order initial value problem:

$$\sin(t) y''' + \cos(ty) y' + \sin(t^2 + y') + (y'')^3 = \log(t)$$

with initial conditions:

$$y(2) = 7, \quad y'(2) = 3, \quad y''(2) = -4$$

by converting it to a system of first-order differential equations and solving numerically using both Euler's method and the fourth-order Runge-Kutta method.

```

clc; clear;
% Define the right-hand side functions
f1 = @(t,Y) Y(2);
f2 = @(t,Y) Y(3);
f3 = @(t,Y) (log(t) - Y(2).^3 - sin(t.^2 + Y(3)) - cos(t.*Y(1))) ./ sin(t);
Fun = @(t,Y) [f1(t,Y); f2(t,Y); f3(t,Y)];
% Initial condition
Y0 = [7; 3; -4];
% Time interval
a = 2;
b = 3;
N = 10;
% Solve using Euler and RK4 methods
[t1,Y1] = eulsys(Fun,a,b,Y0,N);
[t2,Y2] = rk4sys(Fun,a,b,Y0,N);
% Display results
disp('-----Euler-----          RK4 -----');
disp('t_i      y1      y2      y3      y1      y2      y3')
disp('-----')
formatSpec = '%.2f  % 7.5f  % 7.5f  % 9.5f  % 7.5f  % 7.5f  % 7.5f\n';
fprintf(formatSpec,[t1; Y1(1,:); Y1(2,:); Y1(3,:); Y2(1,:); Y2(2,:); Y2(3,:)])
;
% Plot solution y(t) using both methods
figure(1)

```



```

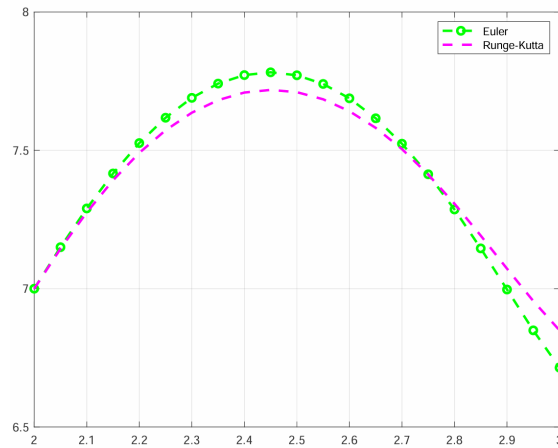
yeu = Y1(1,:);
yrk = Y2(1,:);
clf
plot(t1, yeu, 'go--', 'LineWidth', 2)
hold on
plot(t2, yrk, 'm--', 'LineWidth', 2)
hold off
legend('Euler', 'Runge-Kutta')
axis([2, 3, 6.5, 8])
grid on
title('Numerical Solution of y(t)')
xlabel('t'), ylabel('y(t)')

```

The following table compares the numerical solutions obtained using: Euler's method and Fourth-order Runge–Kutta (RK4) method for the third-order differential equation converted into a system of first-order equations.

t_i	Euler			RK4		
	y_1	y_2	y_3	y_1	y_2	y_3
2.00	7.00000	3.00000	-4.00000	7.00000	3.00000	-4.00000
2.10	7.30000	2.60000	-6.90813	7.27572	2.48000	-6.16181
2.20	7.56000	1.90919	-8.68113	7.49092	1.80996	-7.07103
2.30	7.75092	1.04107	-9.44944	7.63605	1.09032	-7.24269
2.40	7.85503	0.09613	-9.67309	7.70898	0.37004	-7.14576
2.50	7.86464	-0.87118	-9.79487	7.71046	-0.33832	-7.02207
2.60	7.77752	-1.85067	-9.71182	7.64177	-1.03273	-6.83439
2.70	7.59245	-2.82185	-8.29861	7.50510	-1.69108	-6.19855
2.80	7.31027	-3.65171	-2.59211	7.30770	-2.22219	-3.99587
2.90	6.94510	-3.91092	12.52291	7.07318	-2.39173	0.93035
3.00	6.55401	-2.65863	37.49101	6.84989	-1.97744	7.32965

Remark 2.6. Only y_1 represents the approximation to the original function $y(t)$. The other variables y_2 and y_3 represent the first and second derivatives, respectively. The Euler method shows increasing deviation in higher derivatives compared to the RK4 method. The RK4 method provides smoother and more accurate approximations, especially for y_3 , which corresponds to $y''(t)$. This comparison demonstrates how different numerical methods perform when applied to higher-order differential equations reformulated as systems.



2.3.4 System of Higher-Order Differential Equations

Solve the following system of second-order differential equations on the interval $(1, 2)$:

$$\begin{aligned}x'' + te^y + y' &= x - t \\y'' + \cos(xy) + \sin(tx'y) &= x'\end{aligned}$$

with initial conditions:

$$x(1) = 1, \quad x'(1) = 2, \quad y(1) = 3, \quad y'(1) = 4$$

Both $x(t)$ and $y(t)$ are unknown functions to be determined.

Solution: We reduce the system to four first-order differential equations by introducing new variables:

$$z_1 = x$$

$$z_2 = x'$$

$$z_3 = y$$

$$z_4 = y'$$

Then the system becomes:

$$z_1' = z_2$$

$$z_2' = z_2 - z_1 - z_4 - te^{z_3}$$

$$z_3' = z_4$$

$$z_4' = z_1 - \sin(tz_2z_3) - \cos(z_1z_4)$$

The corresponding initial conditions are:

$$z_1(1) = 1, \quad z_2(1) = 2, \quad z_3(1) = 3, \quad z_4(1) = 4$$

This reformulated system can now be solved numerically using methods such as Euler's method or the Runge–Kutta method for systems of equations.

This MATLAB script solves the following system of second-order differential equations:

$$\begin{aligned} x'' + te^y + y' &= x - t \\ y'' + \cos(xy) + \sin(tx'y) &= x' \end{aligned}$$

with initial conditions:

$$x(1) = 1, \quad x'(1) = 2, \quad y(1) = 3, \quad y'(1) = 4$$

The solution is approximated using both Euler's method and the fourth-order Runge–Kutta (RK4) method.

```
clc; clear;
% Define the right-hand side functions
f1 = @(t,Z) Z(2); % z1' = z2 => x' = x'
f2 = @(t,Z) Z(2) - Z(1) - Z(4) - t.*exp(Z(3)); % z2' => x''
f3 = @(t,Z) Z(4); % z3' = z4 => y' = y'
f4 = @(t,Z) (Z(1) - sin(t.*Z(2).*Z(3)) + cos(Z(1).*Z(4)))./Z(3); % z4'=>y''
Fun = @(t,Z) [f1(t,Z); f2(t,Z); f3(t,Z); f4(t,Z)];
% Initial condition vector
Z0 = [1; 2; 3; 4]; % z1 = x, z2 = x', z3 = y, z4 = y'
% Time interval
a = 1;
b = 2;
N = 20;
% Solve using Euler and RK4 methods
[t, Zeu] = eulsys(Fun, a, b, Z0, N);
[t, Zrk] = rk4sys(Fun, a, b, Z0, N);
%% Display Solution
disp(['N = ', int2str(N)])
disp('t_i      Eul x      RK4 x      Eul y      RK4 y      ')
disp('-----')
formatSpec = '%.2f    % 10.5f    % 10.5f    % 10.5f    % 10.5f    \n';
fprintf(formatSpec,[t; Zeu(1,:); Zrk(1,:); Zeu(3,:); Zrk(3,:)]);
%% Plot Solutions % Extract solutions
```

```

xeu = Zeu(1,:);
yeu = Zeu(3,:);
xrk = Zrk(1,:);
yrk = Zrk(3,:);
% Plot x(t)
figure(1); clf
plot(t, xeu, 'go--', 'LineWidth', 2)
hold on
plot(t, xrk, 'm--', 'LineWidth', 2)
hold off
legend('Euler', 'Runge-Kutta')
xlabel('t'), ylabel('x(t)')
title('Numerical Solution for x(t)')
grid on
axis([1, 2, -120, 20])
% Plot y(t)
figure(2); clf
plot(t, yeu, 'go--', 'LineWidth', 2)
hold on
plot(t, yrk, 'm--', 'LineWidth', 2)
hold off
legend('Euler', 'Runge-Kutta')
xlabel('t'), ylabel('y(t)')
title('Numerical Solution for y(t)')
grid on
axis([1, 2, 3, 7])

function [t, Y] = eulsys(f, a, b, y0, N)
    h = (b - a) / N;
    t = linspace(a, b, N+1);
    Y = zeros(length(y0), N+1);
    Y(:,1) = y0;
    for i = 1:N
        Y(:,i+1) = Y(:,i) + h * f(t(i), Y(:,i));
    end
end

function [t, Y] = rk4sys(f, a, b, y0, N)
    h = (b - a) / N;
    t = linspace(a, b, N+1);
    Y = zeros(length(y0), N+1);

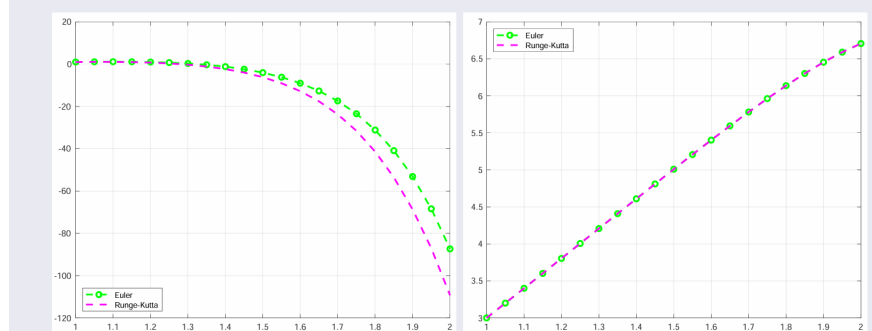
```

```

Y(:,1) = y0;
for i = 1:N
    k1 = f(t(i), Y(:,i));
    k2 = f(t(i) + h/2, Y(:,i) + h*k1/2);
    k3 = f(t(i) + h/2, Y(:,i) + h*k2/2);
    k4 = f(t(i) + h, Y(:,i) + h*k3);
    Y(:,i+1) = Y(:,i) + h*(k1 + 2*k2 + 2*k3 + k4)/6;
end
end

```

The solution curves for $x(t)$ (left) and $y(t)$ (right) are



Chapter 3

Numerical Methods for Boundary Value Problems

A variety of numerical methods have been developed to solve boundary value problems (BVPs) for ordinary differential equations problems in which solution conditions are specified at more than one point in the domain, typically at the endpoints of an interval. Among the most fundamental and widely used of these techniques are the Shooting Method and the Finite Difference Method (FDM). In the following sections, we will explore both approaches in detail: the Shooting Method, which reformulates the BVP as an initial value problem and iteratively adjusts unknown initial conditions to satisfy the boundary constraints; and the Finite Difference Method, which discretizes the differential equation over a grid and replaces derivatives with algebraic approximations to yield a system of linear or nonlinear equations. Each method offers distinct advantages and limitations, and understanding both provides a solid foundation for tackling a broad class of boundary value problems in scientific and engineering applications.

3.1 Shooting Method

The two-point boundary-value problems in this course involve a second-order differential equation of the form:

$$y'' = f(x, y, y'), \quad a < x < b \quad (3.1)$$

together with the boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta. \quad (3.2)$$

3.1.1 Linear Shooting Method

Consider the linear boundary-value problem

$$y'' = p(x)y' + q(x)y + r(x), \quad a < x < b, \quad (3.3)$$

subject to the boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta. \quad (3.4)$$

Assume that:

1. The coefficient functions $p(x)$, $q(x)$, and $r(x)$ are continuous on the closed interval $[a, b]$;
2. The coefficient $q(x)$ is strictly positive on $[a, b]$, i.e., $q(x) > 0$ for all $x \in [a, b]$.

Then the boundary-value problem (3.3)–(3.4) has a unique solution on $[a, b]$.

To solve a linear second-order boundary value problem of the form (3.3)–(3.4), we reformulate it as a combination of two initial value problems (IVPs):

$$y_1'' = p(x)y_1' + q(x)y_1 + r(x), \quad a < x < b, \quad y_1(a) = \alpha, \quad y_1'(a) = 0, \quad (3.5)$$

and

$$y_2'' = p(x)y_2' + q(x)y_2, \quad a < x < b, \quad y_2(a) = 0, \quad y_2'(a) = 1. \quad (3.6)$$

The existence and uniqueness of solutions to such IVPs were established in the previous chapter, provided p , q , and r are continuous on $[a, b]$.

Let $y_1(x)$ and $y_2(x)$ denote the (unique) solutions to (3.5) and (3.6), respectively. Under the hypotheses of Theorem 1.2.2, one can show that $y_2(b) \neq 0$; indeed, the case $y_2(b) = 0$ would contradict the uniqueness of the boundary-value problem (see Exercise ?).

Define the function

$$y(x) = y_1(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2(x). \quad (3.7)$$

We claim that $y(x)$ solves the original boundary-value problem (3.3)–(3.4).

Verification. First, differentiate (3.7):

$$y'(x) = y_1'(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2'(x), \quad y''(x) = y_1''(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2''(x).$$

Substituting the differential equations for y_1'' and y_2'' :

$$\begin{aligned} y''(x) &= [p(x)y_1' + q(x)y_1 + r(x)] + \frac{\beta - y_1(b)}{y_2(b)} [p(x)y_2' + q(x)y_2] \\ &= p(x) \left(y_1' + \frac{\beta - y_1(b)}{y_2(b)} y_2' \right) + q(x) \left(y_1 + \frac{\beta - y_1(b)}{y_2(b)} y_2 \right) + r(x) \\ &= p(x)y'(x) + q(x)y(x) + r(x). \end{aligned}$$

Thus, $y(x)$ satisfies the differential equation (3.3).

Next, check the boundary conditions:

$$y(a) = y_1(a) + \frac{\beta - y_1(b)}{y_2(b)} y_2(a) = \alpha + \frac{\beta - y_1(b)}{y_2(b)} \cdot 0 = \alpha,$$

$$y(b) = y_1(b) + \frac{\beta - y_1(b)}{y_2(b)} y_2(b) = y_1(b) + (\beta - y_1(b)) = \beta.$$

Hence, both boundary conditions (3.4) are satisfied.

Therefore, the solution to the linear BVP is expressed as a linear combination of the solutions to two IVPs. In practice, the *linear shooting method* proceeds by numerically solving the IVPs (3.5) and (3.6) (e.g., using Runge–Kutta methods), and then constructing the approximate BVP solution via (3.7).

- Graphically, the method is illustrated in the figure below.

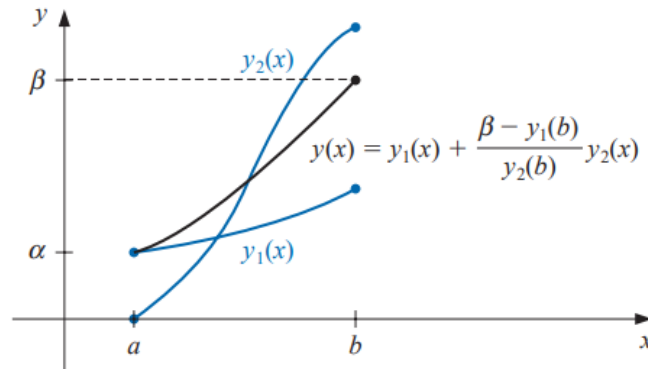


Figure 3.1: Graphical representation of the linear shooting method.

- This “shooting” hits the target after one trial because the problem is linear. For nonlinear problems (next section), multiple iterations are required.

- We use the fourth-order Runge-Kutta method to approximate $y_1(x)$ and $y_2(x)$, though other IVP solvers are also applicable.

Procedures for the Linear Shooting Method

To approximate the solution of the boundary value problem:

$$y'' = p(x)y' + q(x)y + r(x), \quad a < x < b,$$

with boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta,$$

we proceed as follows:

Step 1: Convert to Two IVPs

Solve the following two second-order IVPs:

$$\begin{cases} y_1'' = p(x)y_1' + q(x)y_1 + r(x), \\ y_1(a) = \alpha, \quad y_1'(a) = 0, \end{cases} \quad \begin{cases} y_2'' = p(x)y_2' + q(x)y_2, \\ y_2(a) = 0, \quad y_2'(a) = 1. \end{cases}$$

Step 2: Reduce to First-Order Systems

Let $u_1 = y_1$, $u_2 = y_1'$, $v_1 = y_2$, $v_2 = y_2'$. Then:

For u_1, u_2 :

$$\begin{cases} u_1' = u_2, & u_1(a) = \alpha, \\ u_2' = p(x)u_2 + q(x)u_1 + r(x), & u_2(a) = 0. \end{cases}$$

For v_1, v_2 :

$$\begin{cases} v_1' = v_2, & v_1(a) = 0, \\ v_2' = p(x)v_2 + q(x)v_1, & v_2(a) = 1. \end{cases}$$

Step 3: Apply Fourth-Order Runge-Kutta Method

For each system, use the classical RK4 method on a grid $x_i = a + ih$, $i = 0, 1, \dots, N$, $h = (b - a)/N$.

For the u -system:

$$\begin{aligned}
k_{1,1} &= hu_{2,i}, \\
k_{1,2} &= h [p(x_i)u_{2,i} + q(x_i)u_{1,i} + r(x_i)], \\
k_{2,1} &= h (u_{2,i} + \frac{1}{2}k_{1,2}), \\
k_{2,2} &= h [p(x_i + \frac{h}{2}) (u_{2,i} + \frac{1}{2}k_{1,2}) + q(x_i + \frac{h}{2}) (u_{1,i} + \frac{1}{2}k_{1,1}) + r(x_i + \frac{h}{2})], \\
k_{3,1} &= h (u_{2,i} + \frac{1}{2}k_{2,2}), \\
k_{3,2} &= h [p(x_i + \frac{h}{2}) (u_{2,i} + \frac{1}{2}k_{2,2}) + q(x_i + \frac{h}{2}) (u_{1,i} + \frac{1}{2}k_{2,1}) + r(x_i + \frac{h}{2})], \\
k_{4,1} &= h (u_{2,i} + k_{3,2}), \\
k_{4,2} &= h [p(x_i + h) (u_{2,i} + k_{3,2}) + q(x_i + h) (u_{1,i} + k_{3,1}) + r(x_i + h)], \\
u_{1,i+1} &= u_{1,i} + \frac{1}{6}(k_{1,1} + 2k_{2,1} + 2k_{3,1} + k_{4,1}), \\
u_{2,i+1} &= u_{2,i} + \frac{1}{6}(k_{1,2} + 2k_{2,2} + 2k_{3,2} + k_{4,2}).
\end{aligned}$$

For the v -system:

$$\begin{aligned}
k'_{1,1} &= hv_{2,i}, \\
k'_{1,2} &= h [p(x_i)v_{2,i} + q(x_i)v_{1,i}], \\
k'_{2,1} &= h (v_{2,i} + \frac{1}{2}k'_{1,2}), \\
k'_{2,2} &= h [p(x_i + \frac{h}{2}) (v_{2,i} + \frac{1}{2}k'_{1,2}) + q(x_i + \frac{h}{2}) (v_{1,i} + \frac{1}{2}k'_{1,1})], \\
k'_{3,1} &= h (v_{2,i} + \frac{1}{2}k'_{2,2}), \\
k'_{3,2} &= h [p(x_i + \frac{h}{2}) (v_{2,i} + \frac{1}{2}k'_{2,2}) + q(x_i + \frac{h}{2}) (v_{1,i} + \frac{1}{2}k'_{2,1})], \\
k'_{4,1} &= h (v_{2,i} + k'_{3,2}), \\
k'_{4,2} &= h [p(x_i + h) (v_{2,i} + k'_{3,2}) + q(x_i + h) (v_{1,i} + k'_{3,1})], \\
v_{1,i+1} &= v_{1,i} + \frac{1}{6}(k'_{1,1} + 2k'_{2,1} + 2k'_{3,1} + k'_{4,1}), \\
v_{2,i+1} &= v_{2,i} + \frac{1}{6}(k'_{1,2} + 2k'_{2,2} + 2k'_{3,2} + k'_{4,2}).
\end{aligned}$$

Step 4: Combine Solutions

The approximate solution to the BVP at each grid point x_i is:

$$w_i = u_{1,i} + \frac{\beta - u_{1,N}}{v_{1,N}}v_{1,i},$$

where $u_{1,N} \approx y_1(b)$, $v_{1,N} \approx y_2(b)$.

This $w_i \approx y(x_i)$ satisfies the boundary conditions and approximates the solution across $[a, b]$.

Example 3.1. Apply the Linear Shooting Method with $N = 10$ to the boundary-value problem

$$y'' = -\frac{4}{x}y' - \frac{2}{x^2}y + \frac{2 \ln x}{x^2}, \quad 1 < x < 2, \quad (3.8)$$

with boundary conditions

$$y(1) = \frac{1}{2}, \quad y(2) = \ln 2. \quad (3.9)$$

Solution: First, transform the BVP into two second-order initial value problems (IVPs):

$$y_1'' = -\frac{4}{x}y_1' - \frac{2}{x^2}y_1 + \frac{2 \ln x}{x^2}, \quad y_1(1) = \frac{1}{2}, \quad y_1'(1) = 0, \quad (3.10)$$

$$y_2'' = -\frac{4}{x}y_2' - \frac{2}{x^2}y_2, \quad y_2(1) = 0, \quad y_2'(1) = 1. \quad (3.11)$$

Note: In (3.11), the right-hand side depends only on y_2 , not y_1' — this was a typo in the original.

Next, reduce each second-order IVP to a system of first-order equations.

For (3.10), let $w_1 = y_1$, $w_2 = y_1'$. Then:

$$\begin{cases} w_1' = w_2, & w_1(1) = \frac{1}{2}, \\ w_2' = -\frac{4}{x}w_2 - \frac{2}{x^2}w_1 + \frac{2 \ln x}{x^2}, & w_2(1) = 0. \end{cases} \quad (3.12)$$

For (3.11), let $u_1 = y_2$, $u_2 = y_2'$. Then:

$$\begin{cases} u_1' = u_2, & u_1(1) = 0, \\ u_2' = -\frac{4}{x}u_2 - \frac{2}{x^2}u_1, & u_2(1) = 1. \end{cases} \quad (3.13)$$

We now apply the fourth-order Runge–Kutta method to both systems over $[1, 2]$ with $N = 10$, so $h = 0.1$.

```
% Linear Shooting Method for Example \ref{ex1}
% BVP: y'' = -4/x y' - 2/x^2 y + 2*log(x)/x^2, y(1)=1/2, y(2)=ln(2)
clc; clear all;
% Define coefficient functions
p = @(x) -4/x;
q = @(x) -2/x^2;
```

```

r = @(x) 2*log(x)/x^2;
% Exact solution (corrected)
yexact = @(x) (1/2) - (1/x) + (1/x.^2) - (log(x)./x.^2); % Verified to satisfy
    BVP
a = 1; b = 2;
alpha = 1/2; beta = log(2);
N = 10;
h = (b - a)/N;
% Initial conditions for y1 and y2 systems
u1 = alpha; u2 = 0; % y1(1), y1'(1)
v1 = 0; v2 = 1; % y2(1), y2'(1)
% Storage arrays
U = zeros(2, N+1); U(:,1) = [u1; u2];
V = zeros(2, N+1); V(:,1) = [v1; v2];
% Time-stepping loop
for i = 1:N
    x = a+(i-1)*h;
    % RK4 for u-system (particular solution)
    k11 = h*u2;
    k12 = h*(p(x)*u2+q(x)*u1+r(x));
    k21 = h*(u2+0.5*k12);
    k22 = h*(p(x+0.5*h)*(u2+0.5*k12)+q(x+0.5*h)*(u1+0.5*k11)+r(x+0.5*h));
    k31 = h*(u2+0.5*k22);
    k32 = h*(p(x+0.5*h)*(u2+0.5*k22)+q(x+0.5*h)*(u1+0.5*k21)+r(x+0.5*h));
    k41 = h*(u2+k32);
    k42 = h*(p(x+h)*(u2+k32)+q(x+h)*(u1 + k31)+r(x+h));
    u1 = u1 + (k11 + 2*k21 + 2*k31 + k41)/6;
    u2 = u2 + (k12 + 2*k22 + 2*k32 + k42)/6;

    % RK4 for v-system (homogeneous solution)
    k11_v = h * v2;
    k12_v = h * (p(x)*v2 + q(x)*v1);
    k21_v = h*(v2+0.5*k12_v);
    k22_v = h*(p(x+0.5*h)*(v2+0.5*k12_v)+q(x+0.5*h)*(v1+0.5*k11_v));
    k31_v=h*(v2+0.5*k22_v);
    k32_v = h*(p(x+0.5*h)*(v2+0.5*k22_v)+q(x+0.5*h)*(v1+0.5*k21_v));

    k41_v = h*(v2+k32_v);
    k42_v = h*(p(x+h)*(v2+k32_v)+q(x+h)*(v1+k31_v));
    v1 = v1+(k11_v+2*k21_v+2*k31_v+k41_v)/6;
    v2 = v2 + (k12_v+2*k22_v+2*k32_v+k42_v)/6;

```

```

    % Store values
    U(:,i+1) = [u1; u2];
    V(:,i+1) = [v1; v2];
end
% Compute combination coefficient
z = (beta - U(1,N+1)) / V(1,N+1);
% Output and plot
x_vals = linspace(a, b, N+1);
w = U(1,2:end) + z * V(1,2:end); % Approximate solution at x_i (i=1..N)
x_plot = x_vals(2:end);          % x from 1.1 to 2.0
fprintf('x(i)      Exact      Shooting      |Error|\n');
fprintf('%s\n', repmat('-',1,45));
for i = 1:N
    x = x_plot(i);
    Y = yexact(x);
    fprintf('%5.4f    %11.6f    %11.6f    %1.4e\n', x, Y, w(i), abs(Y - w(i)));
end
% Plot results
figure;
subplot(2,1,1);
plot(x_plot, w, 'ro-', x_plot, arrayfun(yexact, x_plot), 'b*-');
xlabel('x'); ylabel('y(x)');
legend('Shooting Approximation', 'Exact Solution', 'Location', 'best');
title('Linear Shooting Method: Approximation vs Exact');

subplot(2,1,2);
error = abs(arrayfun(yexact, x_plot) - w);
plot(x_plot, error, 'g+-');
xlabel('x'); ylabel('|Error|');
title('Absolute Error');
grid on;

```

The high accuracy is due to the fourth-order convergence ($O(h^4)$) of the Runge–Kutta method used to solve the IVPs.

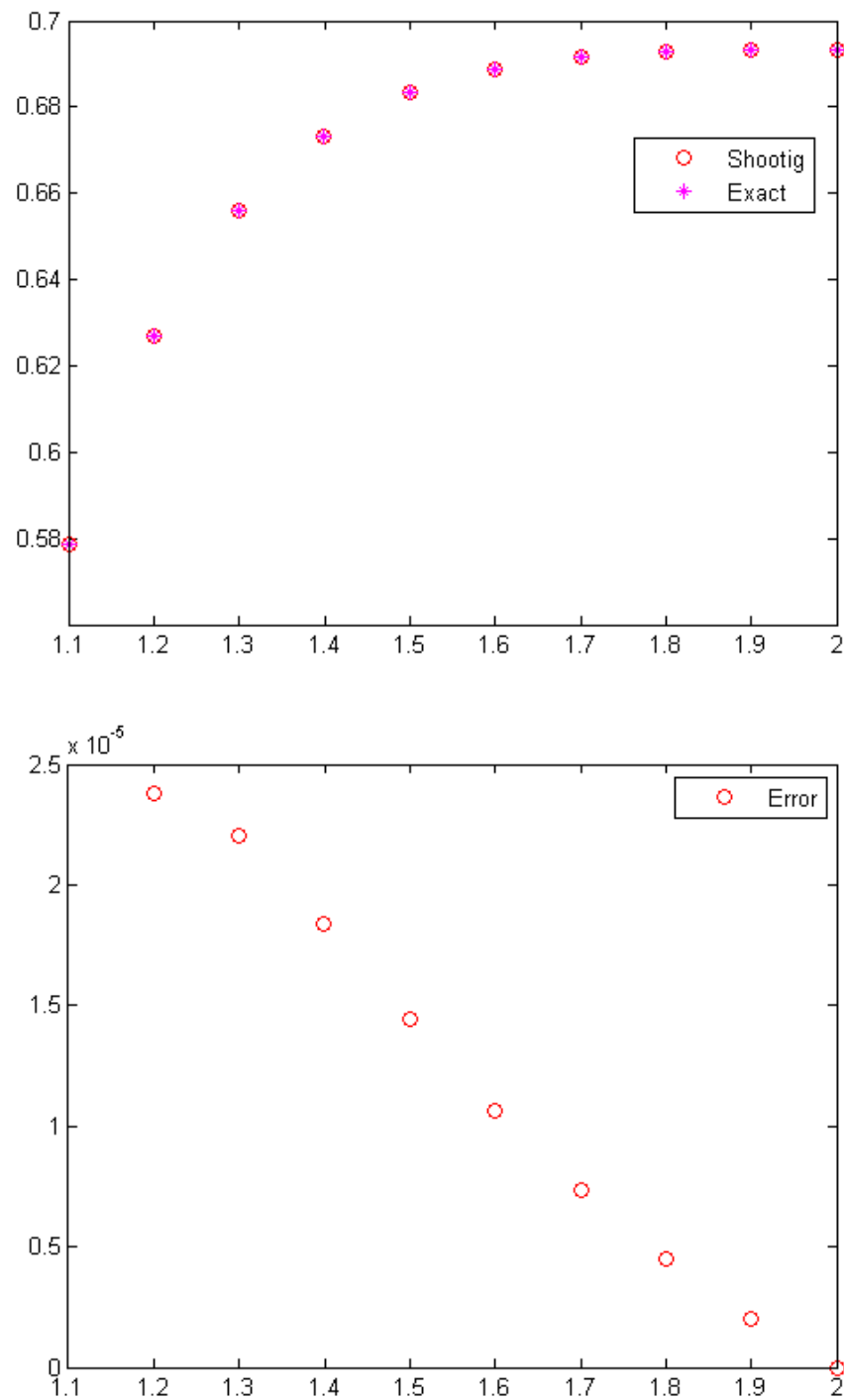


Figure 3.2: Top: Approximate and exact solutions. Bottom: Absolute error.

Table 3.1: Numerical results for Example 3.1 ($N = 10$).

x_i	Exact	Shooting	$ y_{\text{exact}} - w_i $
1.1000	0.578781	0.578762	1.97e-05
1.2000	0.626766	0.626742	2.38e-05
1.3000	0.655855	0.655833	2.21e-05
1.4000	0.673207	0.673189	1.84e-05
1.5000	0.683243	0.683228	1.44e-05
1.6000	0.688754	0.688743	1.07e-05
1.7000	0.691528	0.691521	7.32e-06
1.8000	0.692725	0.692720	4.46e-06
1.9000	0.693100	0.693098	2.04e-06
2.0000	0.693147	0.693147	0.00e+00

Exercise 3.1 (Lab. Exercise.). **Assignment**

1. Solve the BVP,

$$y'' = y' + 2y + \cos x, \quad 0 < x < \pi/2, \quad y(0) = -0.3, \quad y(\pi/2) = -0.1.$$

using the linear shooting method with $N = 10, N = 20, N = 40, N = 80$, the exact solution is $y(x) = -1/10(\sin(x) + 3\cos(x))$. Discuss, the rate of convergence and absolute error of the method.

2. Solve the BVP,

$$y'' = 2y' - y + xe^x - x, \quad 0 < x < 2, \quad y(0) = 0, \quad y(2) = -4.$$

using the linear shooting method with $N = 10, N = 20, N = 40, N = 80$, the exact solution is $y(x) = \frac{1}{6}x^3e^x - \frac{5}{3}xe^x + 2e^x - x - 2$. Discuss, the rate of convergence and absolute error of the method.

3.1.2 Nonlinear Shooting Method

- The shooting technique for the nonlinear second-order boundary-value problem

$$y'' = f(x, y, y'), \quad a < x < b, \quad y(a) = \alpha, \quad y(b) = \beta \quad (3.14)$$

is similar to the linear technique, except that the solution to a nonlinear problem cannot be expressed as a linear combination of the solutions to two initial-value problems.

- Instead, we approximate the solution to the boundary-value problem by using the solutions to a sequence of initial-value problems involving a parameter t .
- These problems have the form

$$y'' = f(x, y, y'), \quad a < x < b, \quad y(a) = \alpha, \quad y'(a) = t \quad (3.15)$$

- We do this by choosing the parameters $t = t_k$ in a manner to ensure that

$$\lim_{k \rightarrow \infty} y(b, t_k) = y(b) = \beta \quad (3.16)$$

where $y(x, t_k)$ denotes the solution to the initial-value problem (3.15) with $t = t_k$, and $y(x)$ denotes the solution to the boundary-value problem (3.14)

- Shooting methods for nonlinear problems require iterations to approach the “target”.
- We start with a parameter t_0 that determines the initial elevation at which the object is fired from the point (a, α) and along the curve described by the solution to the initial-value problem:

$$y'' = f(x, y, y'), \quad a < x < b, \quad y(a) = \alpha, \quad y'(a) = t_0$$

- If $y(b, t_0)$ is not sufficiently close to β , we correct our approximation by choosing elevations t_1, t_2 , and so on, until $y(b, t_k)$ is sufficiently close to “hitting” β . (See Figure Below).
- If $y(x, t)$ denotes the solution to the initial-value problem, we next determine t with

$$y(b, t) - \beta = 0$$

which is a nonlinear equation in the variable t .

- Problems of this type can be solved using secant method or Newton-Raphson methods.

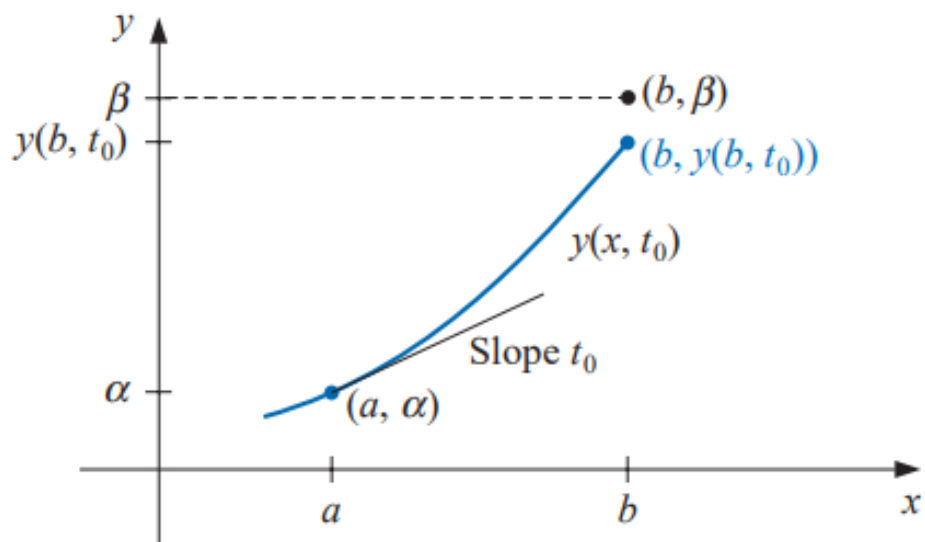


Figure 3.3: The procedure of shooting to determine the initial elevation.

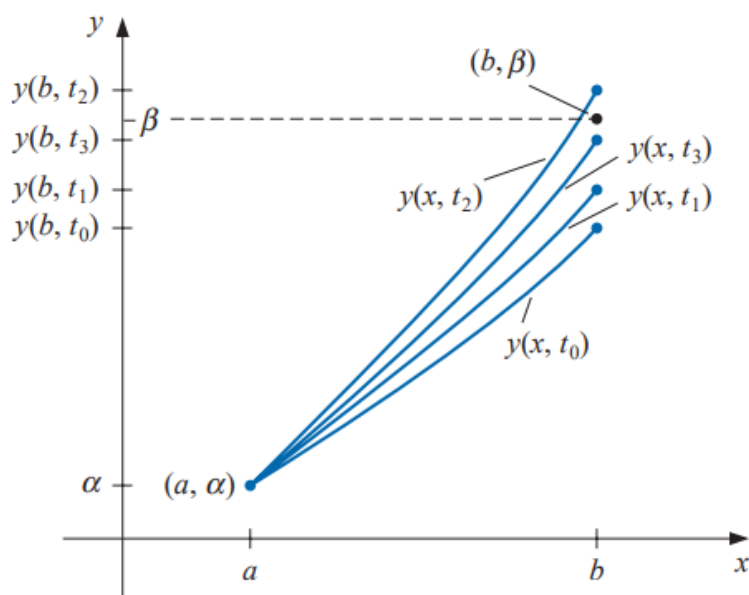


Figure 3.4: Shooting method for different level.

- To use the Secant method to solve the problem, we need to choose initial approximations t_0 and t_1 , and then generate the remaining terms of the sequence by

$$t_k = t_{k-1} - \frac{(y(b, t_{k-1}) - \beta)(t_{k-1} - t_{k-2})}{y(b, t_{k-1}) - y(b, t_{k-2})}, \quad k = 2, 3, \dots$$

- To use the more powerful Newton's method to generate the sequence t_k , only one initial approximation, t_0 , is needed. However, the iteration has the form

$$t_k = t_{k-1} - \frac{y(b, t_{k-1}) - \beta}{y'(b, t_{k-1})}, \quad k = 1, 2, 3, \dots$$

Example 3.2 (Lab work). *Using Nonlinear Shooting method approximate the solution of $y'' = (32 + 2x^3 - yy')/8$, for $1 < x < 3$ with $y(1) = 17$ and $y(3) = 43/3$ for $N = 10, 20, 40, 80$. Take tolerance $\text{tol} = 10^{-5}$ and number of iteration $m = 10$. The exact solution is $y(x) = x^2 + 16/x$.*

Solution: The method converts the BVP into a sequence of IVPs parameterized by an unknown initial slope $s = y'(1)$.

Step 1: Define the Shooting Function

Let $y(x; s)$ be the solution of the IVP:

$$\begin{cases} y'' = f(x, y, y') = \frac{32 + 2x^3 - y y'}{8}, \\ y(1) = 17, \quad y'(1) = s. \end{cases}$$

Define the **shooting function**:

$$\Phi(s) := y(3; s) - \frac{43}{3}.$$

Goal: Find s^* such that $\Phi(s^*) = 0$.

Step 2: Solve IVPs Numerically (e.g., RK4)

Convert to a first-order system:

$$\begin{aligned} u_1 &= y, \\ u_2 &= y', \\ u_1' &= u_2, \\ u_2' &= f(x, u_1, u_2) = \frac{32 + 2x^3 - u_1 u_2}{8}. \end{aligned}$$

Given s , integrate from $x = 1$ to $x = 3$ using step size

$$h = \frac{3-1}{N} = \frac{2}{N}, \quad \text{for } N = 10, 20, 40, 80.$$

Use RK4 (or any high-order method) for accuracy.

Step 3: Root-Finding for s

Use the **secant method** (recommended for nonlinear shooting) or Newton's method.

Secant Method (robust, no derivative needed): Given two initial guesses s_0, s_1 , for $k = 1, 2, \dots, m$ (max iterations $m = 10$):

$$s_{k+1} = s_k - \Phi(s_k) \cdot \frac{s_k - s_{k-1}}{\Phi(s_k) - \Phi(s_{k-1})}.$$

Stopping criteria:

$$|\Phi(s_k)| < \text{tol} = 10^{-5} \quad \text{or} \quad |s_{k+1} - s_k| < \text{tol}.$$

Initial guesses: From the exact solution, $s^* = y'(1) = 2(1) - 16/1^2 = -14$. Use $s_0 = -12$, $s_1 = -16$ (bracketing the root).

Step 4: Newton's Method (Faster, Requires Variational Equation)

To compute $\Phi'(s) = \frac{\partial y}{\partial s}(3; s)$, solve the **variational IVP** alongside the main IVP.

Let

$$w_1 = \frac{\partial y}{\partial s}, \quad w_2 = \frac{\partial y'}{\partial s}.$$

Then w_1, w_2 satisfy:

$$\begin{cases} w_1' = w_2, \\ w_2' = f_y w_1 + f_{y'} w_2, \\ w_1(1) = 0, \quad w_2(1) = 1, \end{cases}$$

where the partial derivatives of f are:

$$f_y = -\frac{y'}{8}, \quad f_{y'} = -\frac{y}{8}.$$

Then $\Phi'(s) = w_1(3)$, and the Newton update is:

$$s_{k+1} = s_k - \frac{\Phi(s_k)}{\Phi'(s_k)}.$$

Remark: Newton's method converges quadratically if s_0 is sufficiently close to s^* ; the secant method is more robust for unknown problems.

Step 5: Output and Error Analysis

For each N :

- Compute the approximate solution $y_N(x_i)$, $i = 0, 1, \dots, N$.
- Compute the maximum absolute error:

$$E_N = \max_{0 \leq i \leq N} \left| y_N(x_i) - \left(x_i^2 + \frac{16}{x_i} \right) \right|.$$

- Record:
 - Final estimate s^* ,
 - Number of iterations to converge,
 - E_N ,
 - Observed order of convergence: $\frac{\log(E_N/E_{2N})}{\log 2}$.

```
% Nonlinear shooting method
% Approximate the solution of y'=(32+2x^3-yy)/8,
%for 1<=x<=3 with y(1)=17 and y(3)=43/3
clc; clear all
f = @(x,y,z) (32+2*x^3-y*z)/8;
fy = @(x,y,z) (-z/8);
fz = @(x,y,z) (-y/8);
exact=@(x)(x^2+16/x);
a=1; b=3; alpha =17; beta =43/3; tol=10^(-5); N=60; m=10;
tk = (beta-alpha)/(b-a);
fprintf('Nonlinear shooting method\n\n');
fprintf('          x(i)          w(i)      Exact      Abs Error \n');
w1 = zeros(1,N+1);
```

```

w2 = zeros(1,N+1);
h = (b-a)/N;
k = 1;
while k <= m
w1(1) = alpha;
w2(1) = tk;
u1 = 0 ;
u2 = 1;
for i = 1:N
x = a+(i-1)*h;
t = x+0.5*h;
k11 = h*w2(i);
k12 = h*f(x,w1(i),w2(i));
k21 = h*(w2(i)+0.5*k12);
k22 = h*f(t,w1(i)+0.5*k11,w2(i)+0.5*k12);
k31 = h*(w2(i)+0.5*k22);
k32 = h*f(t,w1(i)+0.5*k21,w2(i)+0.5*k22);
k41 = h*(w2(i)+k32);
k42 = h*f(x+h,w1(i)+k31,w2(i)+k32);
w1(i+1) = w1(i)+(k11+2*(k21+k31)+k41)/6;
w2(i+1) = w2(i)+(k12+2*(k22+k32)+k42)/6;
%%
k11 = h*u2;
k12 = h*(fy(x,w1(i),w2(i))*u1+fz(x,w1(i),w2(i))*u2);
k21 = h*(u2+0.5*k12);
k22 = h*(fy(t,w1(i),w2(i))*(u1+0.5*k11)+fz(t,w1(i),w2(i))*(u2+0.5*k21));
k31 = h*(u2+0.5*k22);
k32 = h*(fy(t,w1(i),w2(i))*(u1+0.5*k21)+fz(t,w1(i),w2(i))*(u2+0.5*k22));
k41 = h*(u2+k32);
k42 = h*(fy(x+h,w1(i),w2(i))*(u1+k31)+fz(x+h,w1(i),w2(i))*(u2+k32));
u1 = u1+(k11+2*(k21+k31)+k41)/6;
u2 = u2+(k12+2*(k22+k32)+k42)/6;
end
if abs(w1(N+1)-beta) < tol
i = 0;
fprintf(' %11.3f %11.8f %11.8f %11.8e\n', a, alpha, alpha, 0);
for i = 1:N
j = i+1;
x = a+i*h;
Y=exact(x);
fprintf(' %11.3f %11.8f %11.8f %11.8e\n', x, w1(j), Y, abs(w1(j)-Y))

```

```

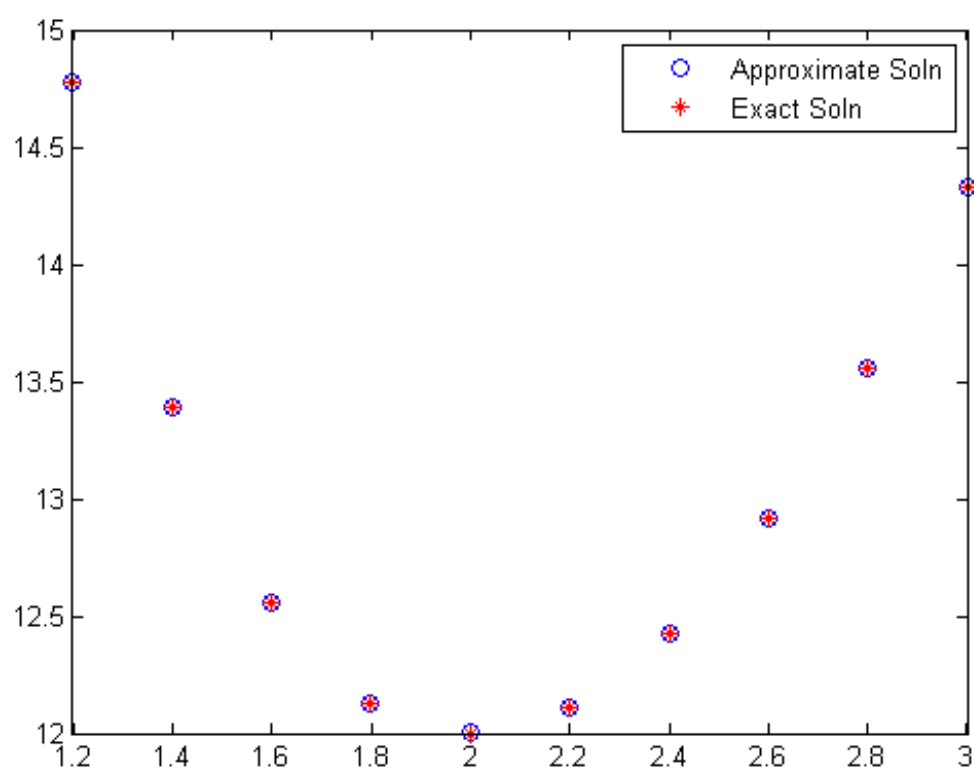
;
plot(x',w1(j)','0', x', Y','*'); hold on
end
fprintf('\nConvergence in %d iterations t=%11.7f \n\n', k, tk);
break;
else
tk = tk-(w1(N+1)-beta)/u1;
k = k+1;
end
end
end

```

Table 3.2: Solution of nonlinear BVP using Shooting method with Newton iteration with $Tol = 10^{-5}$, $N = 10$, $m = 10$.

$x(i)$	$w(i)$	Exact	Abs Error
1.000	17.000000000	17.000000000	0.000000000e+00
1.200	14.77439044	14.773333333	1.05710399e-03
1.400	13.38967167	13.38857143	1.10023972e-03
1.600	12.56092080	12.56000000	9.20801993e-04
1.800	12.12960124	12.12888889	7.12352522e-04
2.000	12.00052412	12.00000000	5.24117946e-04
2.200	12.11309364	12.11272727	3.66366820e-04
2.400	12.42690517	12.42666667	2.38501862e-04
2.600	12.91398330	12.91384615	1.37143243e-04
2.800	13.55434419	13.55428571	5.84790979e-05
3.000	14.33333222	14.33333333	1.10953130e-06

Convergence holds in 7 iterations



3.2 Finite Difference Method for BVPs

3.2.1 Finite Difference Approximations

To approximate the first derivative $u'(\bar{x})$ at the point $\bar{x} = 1$, we consider three finite difference formulas:

$$D_+u(\bar{x}) = \frac{1}{h} [u(\bar{x} + h) - u(\bar{x})] \quad (\text{Forward difference})$$

$$D_0u(\bar{x}) = \frac{1}{2h} [u(\bar{x} + h) - u(\bar{x} - h)] \quad (\text{Central difference})$$

$$D_3u(\bar{x}) = \frac{1}{6h} [2u(\bar{x} + h) + 3u(\bar{x}) - 6u(\bar{x} - h) + u(\bar{x} - 2h)] \quad (\text{Third-order backward difference})$$

We evaluate these approximations on the following test functions:

$$u_1(x) = \sin(10x),$$

$$u_2(x) = 4x^3 - 12x^2 + 14x - 4,$$

$$u_3(x) = |x - 1.001|^{3/2}.$$

For each function, the derivative is approximated at $x = 1$ using step sizes $h = 10^{-j}$ for $j = 1, 2, \dots, 7$. This allows us to study:

- The observed order of convergence,
- The effect of function smoothness on accuracy,
- The onset of roundoff error as h becomes very small.

Theoretical Accuracy

We analyze the local truncation errors of three finite difference formulas for the first derivative $u'(x)$, assuming u is sufficiently smooth (e.g., $u \in C^k$ in a neighborhood of x). All expansions are performed via Taylor series about the point x , with step size $h > 0$.

Forward Difference

The forward difference approximation is

$$D_+u(x) = \frac{u(x+h) - u(x)}{h}. \quad (3.17)$$

Using the Taylor expansion

$$u(x+h) = u(x) + hu'(x) + \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u^{(4)}(x) + O(h^5),$$

we obtain

$$\begin{aligned} D_+u(x) &= \frac{1}{h} \left[hu'(x) + \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u^{(4)}(x) + O(h^5) \right] \\ &= u'(x) + \frac{h}{2}u''(x) + \frac{h^2}{6}u'''(x) + \frac{h^3}{24}u^{(4)}(x) + O(h^4). \end{aligned}$$

Hence, the truncation error is

$$D_+u(x) - u'(x) = \frac{h}{2}u''(x) + \frac{h^2}{6}u'''(x) + \frac{h^3}{24}u^{(4)}(x) + O(h^4) \quad (3.18)$$

with leading term $\frac{h}{2}u''(x)$. Therefore, D_+ is **first-order accurate**:

$$D_+u(x) - u'(x) = O(h).$$

Central Difference

The central difference approximation is

$$D_0u(x) = \frac{u(x+h) - u(x-h)}{2h}. \quad (3.19)$$

Taylor expansions about x :

$$\begin{aligned} u(x+h) &= u + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u^{(4)} + \frac{h^5}{120}u^{(5)} + O(h^6), \\ u(x-h) &= u - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u^{(4)} - \frac{h^5}{120}u^{(5)} + O(h^6). \end{aligned}$$

Subtracting:

$$u(x+h) - u(x-h) = 2hu' + \frac{h^3}{3}u''' + \frac{h^5}{60}u^{(5)} + O(h^7),$$

so

$$D_0u(x) = u' + \frac{h^2}{6}u''' + \frac{h^4}{120}u^{(5)} + O(h^6).$$

Thus,

$$D_0u(x) - u'(x) = \frac{h^2}{6}u'''(x) + \frac{h^4}{120}u^{(5)}(x) + O(h^6) \quad (3.20)$$

with leading term $\frac{h^2}{6}u'''(x)$. Therefore, D_0 is **second-order accurate**:

$$D_0u(x) - u'(x) = O(h^2).$$

Third-Order Approximation

Consider the four-point formula

$$D_3u(x) = \frac{2u(x+h) + 3u(x) - 6u(x-h) + u(x-2h)}{6h}. \quad (3.21)$$

Although it uses a point ahead ($x+h$), it is often referred to as a “third-order backward-type” scheme due to its asymmetry (three points behind, one ahead).

We expand each term about x :

$$\begin{aligned} u(x+h) &= u + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u^{(4)} + \frac{h^5}{120}u^{(5)} + O(h^6), \\ u(x-h) &= u - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u^{(4)} - \frac{h^5}{120}u^{(5)} + O(h^6), \\ u(x-2h) &= u - 2hu' + 2h^2u'' - \frac{4h^3}{3}u''' + \frac{2h^4}{3}u^{(4)} - \frac{4h^5}{15}u^{(5)} + O(h^6). \end{aligned}$$

Compute the numerator:

$$\begin{aligned} 2u(x+h) + 3u(x) - 6u(x-h) + u(x-2h) &= [6h]u' + [0]h^2u'' + [0]h^3u''' \\ &\quad + \left[\frac{1}{2}h^4\right]u^{(4)} - \left[\frac{1}{5}h^5\right]u^{(5)} + O(h^6). \end{aligned}$$

Dividing by $6h$:

$$D_3u(x) = u'(x) + \frac{1}{12}h^3u^{(4)}(x) - \frac{1}{30}h^4u^{(5)}(x) + O(h^5).$$

Hence,

$$D_3u(x) - u'(x) = \frac{1}{12} h^3 u^{(4)}(x) + O(h^4) \quad (3.22)$$

with leading term $\frac{1}{12}h^3u^{(4)}(x)$. Therefore, D_3 is **third-order accurate**:

$$D_3u(x) - u'(x) = O(h^3).$$

Summary of Accuracy

Table 3.3: Finite difference approximations for $u'(x)$: formulas, stencils, leading error terms, and orders.

Scheme	Formula	Leading Error Term	Order
D_+	$\frac{u(x+h) - u(x)}{h}$	$\frac{h}{2}u''(x)$	$O(h)$
D_0	$\frac{u(x+h) - u(x-h)}{2h}$	$\frac{h^2}{6}u'''(x)$	$O(h^2)$
D_3	$\frac{2u(x+h) + 3u(x) - 6u(x-h) + u(x-2h)}{6h}$	$\frac{h^3}{12}u^{(4)}(x)$	$O(h^3)$

Consequently, as $h \rightarrow 0$, we expect the errors to behave like:

$$\text{Error}(D_+) \propto h, \quad \text{Error}(D_0) \propto h^2, \quad \text{Error}(D_3) \propto h^3,$$

provided the solution is sufficiently smooth and roundoff effects are negligible.

Numerical Results

First Test Function: $u_1(x) = \sin(10x)$

This function is smooth and infinitely differentiable. The exact derivative at $x = 1$ is

$$u'_1(1) = 10 \cos(10) \approx -8.390718.$$

The table below reports the absolute errors in the three finite difference approximations for decreasing values of h .

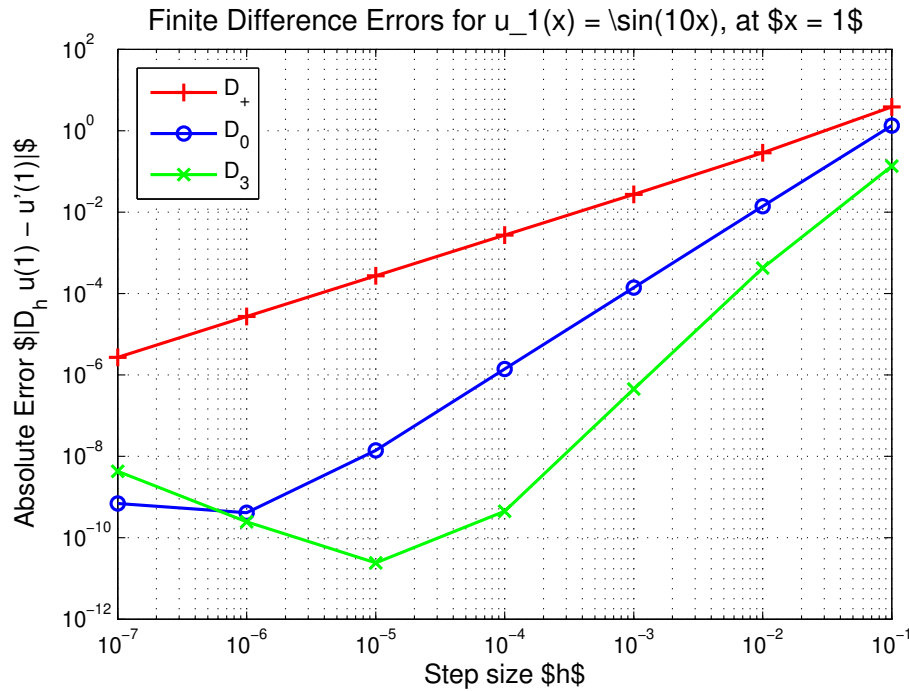
Observations:

- For moderate h (e.g., $h \geq 10^{-4}$), the errors decrease at the expected rates: roughly by a factor of 10 for D_+ , 100 for D_0 , and 1000 for D_3 when h is reduced by a factor of 10.

Table 3.4: Absolute errors in derivative approximations for $u_1(x) = \sin(10x)$ at $x = 1$.

h	$ D_+ - u' $	$ D_0 - u' $	$ D_3 - u' $
1.0×10^{-1}	3.8310×10^0	1.3302×10^0	1.3495×10^{-1}
1.0×10^{-2}	2.8576×10^{-1}	1.3978×10^{-2}	4.2466×10^{-4}
1.0×10^{-3}	2.7341×10^{-2}	1.3984×10^{-4}	4.5055×10^{-7}
1.0×10^{-4}	2.7215×10^{-3}	1.3985×10^{-6}	4.4782×10^{-10}
1.0×10^{-5}	2.7202×10^{-4}	1.4007×10^{-8}	2.4212×10^{-11}
1.0×10^{-6}	2.7201×10^{-5}	4.1279×10^{-10}	2.4625×10^{-10}
1.0×10^{-7}	2.7116×10^{-6}	6.9743×10^{-10}	4.2986×10^{-9}

- As h becomes very small ($h \leq 10^{-6}$), roundoff errors dominate, especially for the higher-order method D_3 , whose error begins to increase due to cancellation in finite-precision arithmetic.
- The central difference D_0 remains accurate down to $h = 10^{-5}$, after which roundoff effects become noticeable.

Figure 3.5: Plot of errors vs. h for $u_1(x) = \sin(10x)$.

For larger h , the errors decrease as expected: by a factor of 10 for D_+ , 100 for D_0 , and 1000 for D_3 . However, for $h < 10^{-5}$, roundoff errors dominate due to cancellation in finite

precision arithmetic (MATLAB uses double precision, 15 digits). The minimal achievable error is about 10^{-9} – 10^{-10} , beyond which accuracy degrades.

Note: In solving differential equations, such small h are rarely used, so roundoff is typically not an issue.

Second Test Function: $u_2(x) = 4x^3 - 12x^2 + 14x - 4$

This is a cubic polynomial. The exact derivative at $x = 1$ is:

$$u'_2(1) = 12(1)^2 - 24(1) + 14 = 2$$

Table 3.5: Errors in derivative approximations for $u_2(x)$ at $x = 1$.

h	D_+ Error	D_0 Error	D_3 Error
1.0×10^{-1}	4.0000×10^{-2}	4.0000×10^{-2}	7.5495×10^{-15}
1.0×10^{-2}	4.0000×10^{-4}	4.0000×10^{-4}	-1.2879×10^{-14}
1.0×10^{-3}	4.0000×10^{-6}	4.0000×10^{-6}	9.6412×10^{-13}
1.0×10^{-4}	3.9999×10^{-8}	4.0008×10^{-8}	1.3102×10^{-11}
1.0×10^{-5}	4.5719×10^{-10}	5.0160×10^{-10}	1.4633×10^{-10}
1.0×10^{-6}	-1.4968×10^{-9}	-6.0862×10^{-10}	-1.6504×10^{-11}
1.0×10^{-7}	5.6086×10^{-9}	-3.2732×10^{-9}	-1.2155×10^{-8}

Observations: D_+ and D_0 errors behave similarly because $u''_2(1) = 0$, eliminating the leading error term in D_+ . D_3 is nearly exact because u_2 is a cubic polynomial, and D_3 is designed to be exact for polynomials up to degree 3. Small nonzero errors in D_3 are due to roundoff.

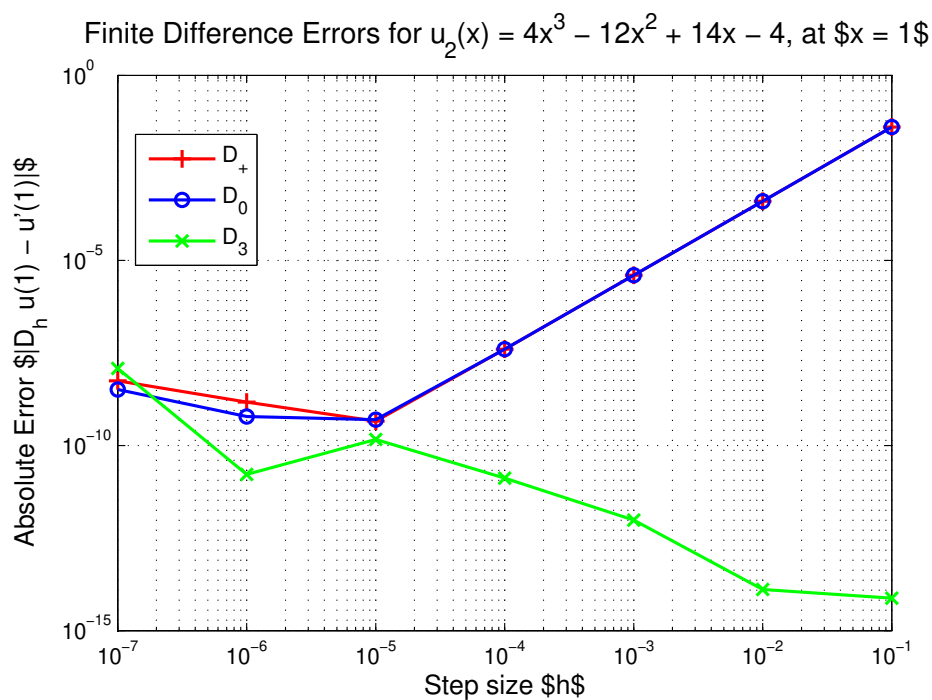
Third Test Function: $u_3(x) = |x - 1.001|^{3/2}$

This function is not smooth at $x = 1.001$. Its first derivative behaves like $|x - 1.001|^{1/2}$, and higher derivatives blow up at this point.

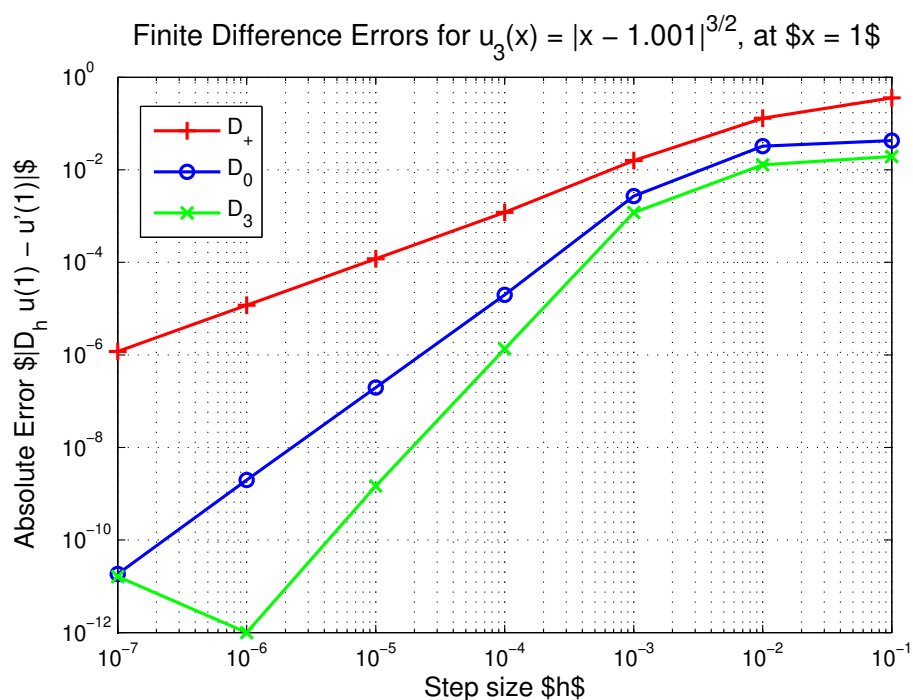
Exact derivative at $x = 1$:

$$u'_3(1) = \frac{3}{2}|1 - 1.001|^{1/2} \cdot (-1) = -\frac{3}{2}\sqrt{0.001} \approx -0.047434$$

For $h > 10^{-3}$, the difference stencils cross the singularity at $x = 1.001$, violating smoothness assumptions. Thus, convergence is poor. For $h < 10^{-3}$, all stencil points lie on the

Figure 3.6: Plot of errors vs. h for $u_2(x)$.Table 3.6: Errors in derivative approximations for $u_3(x)$ at $x = 1$.

h	D_+ Error	D_0 Error	D_3 Error
1.0×10^{-1}	3.5861×10^{-1}	4.2691×10^{-2}	-1.9368×10^{-2}
1.0×10^{-2}	1.2965×10^{-1}	3.2440×10^{-2}	1.2827×10^{-2}
1.0×10^{-3}	1.5811×10^{-2}	2.7128×10^{-3}	1.1890×10^{-3}
1.0×10^{-4}	1.2064×10^{-3}	1.9801×10^{-5}	1.3533×10^{-6}
1.0×10^{-5}	1.1878×10^{-4}	1.9765×10^{-7}	1.4678×10^{-9}
1.0×10^{-6}	1.1861×10^{-5}	1.9777×10^{-9}	9.9724×10^{-13}
1.0×10^{-7}	1.1858×10^{-6}	1.8403×10^{-11}	1.6190×10^{-11}

Figure 3.7: Plot of errors vs. h for $u_3(x)$.

smooth side ($x < 1.001$), and expected convergence rates are observed until roundoff dominates.

Below is a MATLAB script that computes and plots the errors for a given test function and its derivative.

```
% Finite difference error analysis for:
%   u1(x) = sin(10x)
%   u2(x) = 4x^3 - 12x^2 + 14x - 4
%   u3(x) = |x - 1.001|^(3/2)
% Evaluated at x = 1 (smooth for all three)
clear; clc; close all;
x0 = 1; % safe evaluation point (u3 is C^1 here since 1 < 1.001)
% Functions and exact derivatives
u1 = @(x) sin(10*x);
up1 = @(x) 10*cos(10*x);

u2 = @(x) 4*x.^3 - 12*x.^2 + 14*x - 4;
up2 = @(x) 12*x.^2 - 24*x + 14;

a = 1.001;
u3 = @(x) abs(x - a).^(3/2);
```



```

up3 = @(x) (3/2) * sign(x - a) .* sqrt(abs(x - a));

tests = {
    u1, up1, 'u_1(x) = \sin(10x)';
    u2, up2, 'u_2(x) = 4x^3 - 12x^2 + 14x - 4';
    u3, up3, 'u_3(x) = |x - 1.001|^{3/2}';

nh = 7;
%%
for t = 1:size(tests,1)
    u = tests{t,1};
    up = tests{t,2};
    label = tests{t,3};

    fprintf('\n%s\n', repmat('=', 1, 65));
    fprintf('Test %d: %s\n', t, label);
    fprintf('Derivative errors at x = %.3f (kink at 1.001)\n', x0);
    fprintf('          h          D_+ error      D_0 error      D_3 error\n');
    fprintf('-----\n');
    errors = zeros(nh, 4);
    for i = 1:nh
        h = 10^(-i);
        up_val = u(x0 + h);
        u0_val = u(x0);
        um_val = u(x0 - h);
        umm_val = u(x0 - 2*h);

        D_plus = (up_val - u0_val) / h;
        D_0 = (up_val - um_val) / (2*h);
        D_3 = (2*up_val + 3*u0_val - 6*um_val + umm_val) / (6*h);
        %%
        d_exact = up(x0);
        errors(i, :) = [h, D_plus - d_exact, D_0 - d_exact, D_3 - d_exact];

        fprintf(' %12.4e %12.4e %12.4e %12.4e\n', ...
            h, errors(i,2), errors(i,3), errors(i,4));
    end
    % === Plot (no reference slopes) ===
    figure('Name', ['Error: ' label], ...
        'Position', [100 + 200*(t-1), 100, 600, 420]);
    h_vals = errors(:,1);

```

```

loglog(h_vals, abs(errors(:,2)), 'r+-', 'LineWidth', 1.4, 'MarkerSize', 8,
...
'DisplayName', '$D_{+}$: Forward (1st order)');
hold on;
loglog(h_vals, abs(errors(:,3)), 'bo-', 'LineWidth', 1.4, 'MarkerSize', 6,
...
'DisplayName', '$D_{0}$: Central (2nd order)');
loglog(h_vals, abs(errors(:,4)), 'g x-', 'LineWidth', 1.4, 'MarkerSize',
8, ...
'DisplayName', '$D_{3}$: 3rd-order Backward');

xlabel('Step size $h$', 'Interpreter', 'tex', 'FontSize', 12);
ylabel('Absolute Error $|D_h u(1) - u'(1)|$', 'Interpreter', 'tex', '
FontSize', 12);
title(['Finite Difference Errors for ', regexp(label, '\$', ''), ...
', at $x = ', num2str(x0), '$'], ...
'Interpreter', 'tex', 'FontSize', 13);
lgd = legend('show', 'Location', 'southwest', 'D_+', 'D_0', 'D_3');
lgd.Interpreter = 'tex';
grid on; box on;
end

```

Finite difference methods achieve their theoretical convergence rates only when the solution is sufficiently smooth and h is not too small.

Roundoff errors dominate for very small h , limiting achievable accuracy.

Singularities or nonsmooth points in the solution degrade convergence unless h is small enough to resolve the smooth region.

Choosing the appropriate difference formula and step size is crucial in numerical methods for differential equations.

3.2.2 Finite Difference Method for Linear BVPs

The linear and nonlinear shooting methods for boundary value problems (BVPs) can suffer from numerical instability, particularly when the differential equation is convection-dominated (i.e., when the first-order term $p(x)u'(x)$ is large). In contrast, finite difference methods (FDM) offer better stability by directly discretizing the differential operator over the entire domain. While FDM requires solving a system of linear equations (and thus more computational effort), it avoids the sensitivity to initial guesses inherent in shooting methods.

Finite difference methods approximate derivatives using difference quotients. The choice of scheme affects both accuracy and stability. We consider two standard approaches: the **central difference scheme** and the **upwind scheme**.

Consider the linear second-order BVP:

$$u''(x) + p(x)u'(x) + q(x)u(x) = f(x), \quad a < x < b, \quad (3.23)$$

with boundary conditions

$$u(a) = \alpha, \quad u(b) = \beta. \quad (3.24)$$

We discretize the interval $[a, b]$ into $N + 1$ equally spaced points:

$$x_i = a + ih, \quad i = 0, 1, \dots, N, \quad h = \frac{b - a}{N}.$$

Let $u_i \approx u(x_i)$. The goal is to approximate u_i for $i = 1, 2, \dots, N - 1$, with $u_0 = \alpha$, $u_N = \beta$.

Central Finite Difference Scheme

We use the following approximations: First derivative:

$$u'(x_i) \approx \frac{u_{i+1} - u_{i-1}}{2h} + O(h^2)$$

Second derivative:

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + O(h^2)$$

Substitute into (3.23) at $x = x_i$:

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + p_i \cdot \frac{u_{i+1} - u_{i-1}}{2h} + q_i u_i = f_i,$$

where $p_i = p(x_i)$, $q_i = q(x_i)$, $f_i = f(x_i)$.

Multiply through by h^2 and rearrange:

$$\left(1 + \frac{h}{2}p_i\right) u_{i-1} + (-2 + h^2 q_i) u_i + \left(1 - \frac{h}{2}p_i\right) u_{i+1} = h^2 f_i.$$

Matrix Formulation

This leads to a tridiagonal system:

$$AU = \mathbf{b},$$

where: $\mathbf{U} = [u_1, u_2, \dots, u_{N-1}]^T$; A is a tridiagonal matrix with entries:

$$A_{i,i-1} = 1 + \frac{h}{2}p_i, \quad A_{i,i} = -2 + h^2q_i, \quad A_{i,i+1} = 1 - \frac{h}{2}p_i$$

(for $i = 1, \dots, N-1$, with $A_{1,0}$ and $A_{N-1,N}$ omitted).

The right-hand side vector $\mathbf{b}$ incorporates boundary conditions:

$$b_1 = h^2 f_1 - \left(1 + \frac{h}{2}p_1\right) \alpha, \quad b_{N-1} = h^2 f_{N-1} - \left(1 - \frac{h}{2}p_{N-1}\right) \beta,$$

and $b_i = h^2 f_i$ for $i = 2, \dots, N-2$.

Thus:

$$\mathbf{b} = \begin{pmatrix} f(x_1)h^2 - \left(1 + \frac{h}{2}p(x_1)\right) \alpha \\ f(x_2)h^2 \\ \vdots \\ f(x_{N-2})h^2 \\ f(x_{N-1})h^2 - \left(1 - \frac{h}{2}p(x_{N-1})\right) \beta \end{pmatrix}$$

Stability and Convergence

- **Accuracy:** The method is second-order accurate: $\text{Error} = O(h^2)$ if $p, q, f \in C^2[a, b]$.
- **Stability:** The scheme is stable if the matrix A is strictly diagonally dominant or an M-matrix.
- **Diagonal Dominance:** Requires:

$$|A_{i,i}| \geq |A_{i,i-1}| + |A_{i,i+1}|$$

For central differences, this fails when $|p(x_i)|$ is large. Specifically, instability occurs when the **Péclet number** $\text{Pe}_i = \frac{|p(x_i)|h}{2} > 1$.

- **Oscillations:** When $\text{Pe}_i > 1$, the solution may exhibit nonphysical oscillations due to loss of monotonicity.
- **Convergence:** If $q(x) \geq 0$ and $p(x)$ is bounded, the method converges as $h \rightarrow 0$ with order $O(h^2)$.

Upwind Finite Difference Scheme

When $|p(x)|$ is large (especially $|p(x)| \sim 1/h$), central differences become unstable. The **upwind scheme** uses a backward difference for u' if $p(x) > 0$ (flow to the right), and forward if $p(x) < 0$ (flow to the left), mimicking the direction of information propagation.

Discretization

For $p_i \geq 0$: use backward difference

$$u'(x_i) \approx \frac{u_i - u_{i-1}}{h} \Rightarrow \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + p_i \frac{u_i - u_{i-1}}{h} + q_i u_i = f_i$$

Multiply by h^2 :

$$(1 - hp_i)u_{i-1} + (-2 + hp_i + h^2 q_i)u_i + u_{i+1} = h^2 f_i$$

For $p_i < 0$: use forward difference

$$u'(x_i) \approx \frac{u_{i+1} - u_i}{h} \Rightarrow u_{i-1} + (-2 - hp_i + h^2 q_i)u_i + (1 + hp_i)u_{i+1} = h^2 f_i$$

Matrix Formulation

The system is $A\mathbf{U} = \mathbf{b}$, with:

$$A_{i,i-1} = \begin{cases} 1 - hp_i, & p_i \geq 0 \\ 1, & p_i < 0 \end{cases} \quad A_{i,i} = \begin{cases} -2 + hp_i + h^2 q_i, & p_i \geq 0 \\ -2 - hp_i + h^2 q_i, & p_i < 0 \end{cases} \quad A_{i,i+1} = \begin{cases} 1, & p_i \geq 0 \\ 1 + hp_i, & p_i < 0 \end{cases}$$

Boundary terms: $b_1 = h^2 f_1 - A_{1,0}\alpha$; $b_{N-1} = h^2 f_{N-1} - A_{N-1,N}\beta$

Stability and Convergence

- **Accuracy:** First-order: $O(h)$
- **Stability:** The upwind scheme is unconditionally stable for $q(x) \geq 0$
- **Diagonal Dominance:** If $q(x) \geq 0$, the matrix is diagonally dominant:

$$|A_{i,i}| \geq |A_{i,i-1}| + |A_{i,i+1}|$$

This ensures stability and prevents oscillations.

- **Monotonicity:** The discrete solution satisfies a maximum principle if $q(x) \geq 0$.

```
% Finite Difference Method for: u'' + p(x)u' + q(x)u = f(x), u(0)=alpha, u(1)=
    beta
clc; clear all; close all;

% Problem data
alpha = 1; beta = 3;
p = @(x) -1; % Convection coefficient
q = @(x) 0; % Reaction coefficient
f = @(x) -1; % Source term
u_exact = @(x) 1 + x + (exp(x) - 1)/(exp(1) - 1); % True solution

% Discretization
N_values = [16, 32, 64, 128, 256, 512, 1024];
errors_cent = zeros(size(N_values));
errors_up = zeros(size(N_values));

for k = 1:length(N_values)
    N = N_values(k);
    h = 1/N;
    x = linspace(0, 1, N+1)';

    P = arrayfun(p, x(2:N)); % Interior values
    Q = arrayfun(q, x(2:N));
    F = arrayfun(f, x(2:N));

    % Central Difference
    A_cent = zeros(N-1);
    b_cent = zeros(N-1,1);
    for i = 1:N-1
        if i == 1
            A_cent(i,i) = -2/h^2 + Q(i);
            A_cent(i,i+1) = 1/h^2 + P(i)/(2*h);
            b_cent(i) = F(i) - (1/h^2 - P(i)/(2*h)) * alpha;
        elseif i == N-1
            A_cent(i,i-1) = 1/h^2 - P(i)/(2*h);
            A_cent(i,i) = -2/h^2 + Q(i);
            b_cent(i) = F(i) - (1/h^2 + P(i)/(2*h)) * beta;
        else

```

```

        A_cent(i,i-1) = 1/h^2 - P(i)/(2*h);
        A_cent(i,i) = -2/h^2 + Q(i);
        A_cent(i,i+1) = 1/h^2 + P(i)/(2*h);
        b_cent(i) = F(i);

    end

end

U_cent = A_cent \ b_cent;
U_cent_full = [alpha; U_cent; beta];

% Upwind (p < 0 => forward difference)
A_up = zeros(N-1);
b_up = zeros(N-1,1);
for i = 1:N-1
    if i == 1
        A_up(i,i) = -2/h^2 - P(i)/h + Q(i);
        A_up(i,i+1) = 1/h^2 + P(i)/h;
        b_up(i) = F(i) - (1/h^2) * alpha;
    elseif i == N-1
        A_up(i,i-1) = 1/h^2;
        A_up(i,i) = -2/h^2 - P(i)/h + Q(i);
        b_up(i) = F(i) - (1/h^2 + P(i)/h) * beta;
    else
        A_up(i,i-1) = 1/h^2;
        A_up(i,i) = -2/h^2 - P(i)/h + Q(i);
        A_up(i,i+1) = 1/h^2 + P(i)/h;
        b_up(i) = F(i);
    end
end

U_up = A_up \ b_up;
U_up_full = [alpha; U_up; beta];

% Compute errors
u_true = arrayfun(u_exact, x);
errors_cent(k) = max(abs(U_cent_full - u_true));
errors_up(k) = max(abs(U_up_full - u_true));
end

% Convergence analysis
N_sq = N_values.^2;
C_cent = errors_cent .* N_sq;
C_up = errors_up .* N_values; % First-order

```

```

% Display table
fprintf('\nN\tError (Central)\t\tC = Error*N^2\tError (Upwind)\tC = Error*N\n'
);
for k = 1:length(N_values)
    fprintf('%d\t%.2e\t\t%.2e\t\t%.2e\t\t%.2e\n', ...
        N_values(k), errors_cent(k), C_cent(k), errors_up(k), C_up(k));
end

```

Table 3.7: Convergence analysis: errors and experimental order of convergence

N	Error (Central)	EOC (Central)	Error (Upwind)	EOC (Upwind)
16	5.004×10^{-5}	—	1.013×10^{-3}	—
32	1.256×10^{-5}	1.99	5.072×10^{-4}	1.00
64	3.140×10^{-6}	2.00	2.537×10^{-4}	1.00
128	7.850×10^{-7}	2.00	1.269×10^{-4}	1.00
256	1.962×10^{-7}	2.00	6.345×10^{-5}	1.00
512	4.906×10^{-8}	2.00	3.172×10^{-5}	1.00
1024	1.227×10^{-8}	2.00	1.586×10^{-5}	1.00

Conclusion

- The **central difference** method is second-order accurate but unstable when $Pe_i > 1$. Use only for mildly convection-dominated problems.
- The **upwind** method is first-order accurate but unconditionally stable for $q(x) \geq 0$. It avoids oscillations and is preferred for stiff or convection-dominated BVPs.
- Both schemes lead to tridiagonal systems solvable in $O(N)$ time using the Thomas algorithm.
- Always check the Péclet number and ensure diagonal dominance for reliable results.
- For higher accuracy and stability, consider **exponential fitting** or **hybrid schemes**.

Extension to Neumann and Robin Boundary Conditions

So far, we have considered Dirichlet boundary conditions $u(a) = \alpha$, $u(b) = \beta$. We now extend the finite difference framework to **Neumann** and **Robin** boundary conditions. The

same uniform grid is used:

$$x_i = a + ih, \quad i = 0, 1, \dots, N, \quad h = \frac{b-a}{N},$$

with $u_i \approx u(x_i)$. To maintain second-order accuracy at boundaries involving derivatives, we introduce *ghost points* u_{-1} and u_{N+1} , which are eliminated algebraically using the boundary conditions.

Neumann Boundary Conditions Consider

$$u'(a) = \gamma, \quad u'(b) = \delta. \quad (3.25)$$

Using central differences:

$$u'(x_0) \approx \frac{u_1 - u_{-1}}{2h} = \gamma \implies u_{-1} = u_1 - 2h\gamma,$$

$$u'(x_N) \approx \frac{u_{N+1} - u_{N-1}}{2h} = \delta \implies u_{N+1} = u_{N-1} + 2h\delta.$$

Substitute these into the discretized differential equation at $i = 0$ and $i = N$, respectively.

- At x_0 :

$$\begin{aligned} & \frac{u_1 - 2u_0 + u_{-1}}{h^2} + p_0 \frac{u_1 - u_{-1}}{2h} + q_0 u_0 = f_0 \\ \implies & \frac{2(u_1 - u_0)}{h^2} - \frac{2\gamma}{h} + p_0 \gamma + q_0 u_0 = f_0. \end{aligned}$$

Rearranging yields the modified left boundary equation:

$$\left(-\frac{2}{h^2} + q_0\right) u_0 + \frac{2}{h^2} u_1 = f_0 + \gamma \left(\frac{2}{h} - p_0\right). \quad (3.26)$$

- At x_N :

$$\begin{aligned} & \frac{u_{N+1} - 2u_N + u_{N-1}}{h^2} + p_N \frac{u_{N+1} - u_{N-1}}{2h} + q_N u_N = f_N \\ \implies & \frac{2(u_{N-1} - u_N)}{h^2} + \frac{2\delta}{h} + p_N \delta + q_N u_N = f_N, \end{aligned}$$

which leads to:

$$\frac{2}{h^2}u_{N-1} + \left(-\frac{2}{h^2} + q_N\right)u_N = f_N - \delta \left(\frac{2}{h} + p_N\right). \quad (3.27)$$

The unknown vector becomes $\mathbf{U} = [u_0, u_1, \dots, u_N]^T$, and the linear system is $(N+1) \times (N+1)$ tridiagonal.

Robin Boundary Conditions Now consider mixed (Robin) conditions:

$$u'(a) + \eta u(a) = \gamma, \quad u'(b) + \zeta u(b) = \delta, \quad (3.28)$$

where $\eta, \zeta \geq 0$ are given constants.

- Left boundary ($x = a$): Central approximation gives

$$\frac{u_1 - u_{-1}}{2h} + \eta u_0 = \gamma \implies u_{-1} = u_1 + 2h(\eta u_0 - \gamma).$$

Substituting into the differential equation at $i = 0$ and simplifying yields:

$$\frac{2}{h^2}u_1 + \left(-\frac{2}{h^2} + \frac{2\eta}{h} - \eta p_0 + q_0\right)u_0 = f_0 + \gamma \left(\frac{2}{h} - p_0\right). \quad (3.29)$$

- Right boundary ($x = b$): Similarly,

$$\frac{u_{N+1} - u_{N-1}}{2h} + \zeta u_N = \delta \implies u_{N+1} = u_{N-1} + 2h(\delta - \zeta u_N),$$

and substitution gives:

$$\frac{2}{h^2}u_{N-1} + \left(-\frac{2}{h^2} - \frac{2\zeta}{h} - \zeta p_N + q_N\right)u_N = f_N - \delta \left(\frac{2}{h} + p_N\right). \quad (3.30)$$

When $\eta = 0$ (resp. $\zeta = 0$), (3.29) (resp. (3.30)) reduces to the Neumann case (3.26) (resp. (3.27)). As $\eta, \zeta \rightarrow \infty$ (suitably scaled), the conditions approach Dirichlet type.

Matrix Structure and Stability With Neumann or Robin conditions, the coefficient matrix A remains *banded* (tridiagonal), but the first and last rows are modified. For the

central scheme, diagonal dominance at the boundaries requires:

$$\begin{aligned} \text{Left:} \quad & \left| -\frac{2}{h^2} + \frac{2\eta}{h} - \eta p_0 + q_0 \right| \geq \frac{2}{h^2}, \\ \text{Right:} \quad & \left| -\frac{2}{h^2} - \frac{2\zeta}{h} - \zeta p_N + q_N \right| \geq \frac{2}{h^2}. \end{aligned}$$

These are guaranteed if $q(x) \geq 0$ and $\eta, \zeta > 0$, especially for sufficiently small h . In convection-dominated regimes ($|p(x)|$ large), combining Robin/Neumann treatment with the **upwind scheme** in the interior is recommended to preserve monotonicity.

Table 3.8: Summary of boundary condition treatments in finite difference discretization.

Type	Unknowns	Size of System	Notes
Dirichlet–Dirichlet	$u_1, \dots, u_{N-1}$	$(N-1) \times (N-1)$	BCs move to RHS.
Neumann–Neumann	$u_0, \dots, u_N$	$(N+1) \times (N+1)$	May be singular if $q \equiv 0$.
Dirichlet–Neumann	$u_1, \dots, u_N$	$N \times N$	use (3.27) at x_N .
Robin–Robin	$u_0, \dots, u_N$	$(N+1) \times (N+1)$	ensures well-posedness for $\eta, \zeta > 0$.

Example with Robin Boundary Condition

Example 3.3. Consider the linear boundary value problem

$$u''(x) + \beta u'(x) = -1, \quad 0 < x < 1, \quad (3.31)$$

with mixed boundary conditions:

$$u(0) = 0, \quad u'(1) + u(1) = 1, \quad (3.32)$$

where $\beta \gg 1$ (e.g., $\beta = 50$). This models steady advection-diffusion in which convection strongly dominates diffusion, leading to a thin boundary layer near $x = 0$.

Exact Solution Equation (3.31) has constant coefficients. The characteristic equation is $r^2 + \beta r = 0$, yielding roots $r = 0$ and $r = -\beta$. A particular solution is $u_p(x) = -x/\beta$. Thus the general solution is

$$u(x) = C_1 + C_2 e^{-\beta x} - \frac{x}{\beta}.$$

Apply $u(0) = 0$: $C_1 + C_2 = 0 \Rightarrow C_1 = -C_2$, so

$$u(x) = C_2 (e^{-\beta x} - 1) - \frac{x}{\beta}.$$

Now impose the Robin condition at $x = 1$:

$$u'(x) = -\beta C_2 e^{-\beta x} - \frac{1}{\beta},$$

$$u'(1) + u(1) = \left(-\beta C_2 e^{-\beta} - \frac{1}{\beta}\right) + \left(C_2(e^{-\beta} - 1) - \frac{1}{\beta}\right) = 1.$$

Simplifying:

$$C_2(-\beta + 1)e^{-\beta} - C_2 - \frac{2}{\beta} = 1 \implies C_2 = \frac{-1 - \frac{2}{\beta}}{1 + (\beta - 1)e^{-\beta}}.$$

For large β , $e^{-\beta} \approx 0$, so $C_2 \approx -1 - 2/\beta \approx -1$. Hence, to leading order:

$$u(x) \approx (1 - e^{-\beta x}) - \frac{x}{\beta}. \quad (3.33)$$

A boundary layer of width $O(1/\beta)$ forms near $x = 0$. For $\beta = 100$, the layer width is ~ 0.01 .

Finite Difference Discretization Let $x_i = ih$, $i = 0, \dots, N$, $h = 1/N$. Unknowns: $\mathbf{U} = [u_1, \dots, u_N]^\top$ ($u_0 = 0$ given).

- **Central difference** at interior points $i = 1, \dots, N - 1$:

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \beta \frac{u_{i+1} - u_{i-1}}{2h} = -1.$$

Multiply by h^2 :

$$\left(1 + \frac{\beta h}{2}\right) u_{i-1} + (-2)u_i + \left(1 - \frac{\beta h}{2}\right) u_{i+1} = -h^2.$$

This scheme is stable only if $\text{Pe}_i = \frac{\beta h}{2} < 1$; for $\beta = 100$, need $h < 0.02$ ($N > 50$) barely sufficient, and oscillations appear for $N = 50$.

- **Upwind scheme** ($\beta > 0$, use backward difference for u'):

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \beta \frac{u_i - u_{i-1}}{h} = -1.$$

Multiply by h^2 :

$$(1 - \beta h)u_{i-1} + (-2 + \beta h)u_i + u_{i+1} = -h^2.$$

Diagonal dominance holds if $\beta h \leq 1$, but even for $\beta h > 1$, the matrix remains an M-matrix when $q = 0$ and the Robin condition is used ensuring monotonicity and stability.

At $i = N$, apply the Robin condition $u'(1) + u(1) = 1$ using a second-order central approximation:

$$\frac{u_{N+1} - u_{N-1}}{2h} + u_N = 1 \quad \implies \quad u_{N+1} = u_{N-1} + 2h(1 - u_N).$$

Substitute into the upwind discretization of the ODE at $i = N$:

$$\frac{u_{N+1} - 2u_N + u_{N-1}}{h^2} + \beta \frac{u_N - u_{N-1}}{h} = -1.$$

After substitution and simplification, the last equation becomes:

$$(1 - \beta h)u_{N-1} + \left(-2 + \beta h - \frac{2}{h}\right)u_N = -h^2 - 2. \quad (3.34)$$

Implementation in MATLAB: Exercise

3.2.3 Finite Difference Method for Nonlinear BVPs

Discretizing a nonlinear differential equation generally results in a system of nonlinear algebraic equations. Solving this system numerically yields an approximate solution to the original boundary value problem (BVP). In this section, we present the finite difference method (FDM) for nonlinear BVPs, using Newton's method for solving the resulting nonlinear system.

We focus on a class of quasilinear second-order ODEs, but the method extends to more general forms.

Consider the nonlinear BVP:

$$-u''(x) + F(u(x)) = f(x), \quad x \in (0, 1), \quad (3.35)$$

with Dirichlet boundary conditions:

$$u(0) = 0, \quad u(1) = 0.$$

We assume $F \in C^1(\mathbb{R})$, ensuring differentiability of the nonlinear term. This includes examples like $F(u) = e^u, u^3, \sin u$, etc.

Finite Difference Discretization

Let:

- N : number of interior subintervals,
- $h = \frac{1}{N}$: mesh size,
- $x_i = ih, i = 0, 1, \dots, N$,
- $u_i \approx u(x_i)$.

Using the central finite difference approximation for the second derivative:

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + O(h^2),$$

the discrete form of (3.35) at interior points $i = 1, 2, \dots, N - 1$ becomes:

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + F(u_i) = f(x_i). \quad (3.36)$$

Since $u_0 = u_N = 0$ (from boundary conditions), we define the vector of unknowns:

$$\mathbf{u} = [u_1, u_2, \dots, u_{N-1}]^T \in \mathbb{R}^{N-1}.$$

The system becomes:

$$\mathbf{G}(\mathbf{u}) = \mathbf{0},$$

where $\mathbf{G} : \mathbb{R}^{N-1} \rightarrow \mathbb{R}^{N-1}$ is defined component-wise by:

$$G_i(\mathbf{u}) = -\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + F(u_i) - f(x_i), \quad i = 1, 2, \dots, N - 1,$$

with $u_0 = 0, u_N = 0$.

Newton's Iteration Method

To solve $\mathbf{G}(\mathbf{u}) = \mathbf{0}$, we apply Newton's method, which is highly effective for smooth nonlinear systems.

Given an initial guess $\mathbf{u}^{(0)}$, iterate:

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} - J(\mathbf{u}^{(k)})^{-1} \mathbf{G}(\mathbf{u}^{(k)}), \quad k = 0, 1, 2, \dots \quad (3.37)$$

where $J(\mathbf{u})$ is the Jacobian matrix of $\mathbf{G}$, defined by:

$$J_{ij} = \frac{\partial G_i}{\partial u_j}.$$

For our system, J is tridiagonal:

$$J(\mathbf{u}) = \begin{bmatrix} \frac{2}{h^2} + F'(u_1) & -\frac{1}{h^2} & & & \\ -\frac{1}{h^2} & \frac{2}{h^2} + F'(u_2) & \ddots & & \\ & & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + F'(u_{N-1}) \\ & & & & -\frac{1}{h^2} \end{bmatrix}$$

At each Newton step:

- Evaluate residual $\mathbf{G}(\mathbf{u}^{(k)})$,
- Compute Jacobian $J(\mathbf{u}^{(k)})$,
- Solve linear system: $J(\mathbf{u}^{(k)})\mathbf{w}^{(k)} = \mathbf{G}(\mathbf{u}^{(k)})$,
- Update: $\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} - \mathbf{w}^{(k)}$.
- The iteration stops when $\|\mathbf{G}(\mathbf{u}^{(k)})\| < \text{TOL}$, e.g., $\text{TOL} = 10^{-6}$.

Remark 3.1 (Remarks on Newton's Method).

The Jacobian must be recomputed at each iteration (unless using a simplified Newton method).

It can be computationally expensive for large N , but the tridiagonal structure allows efficient $O(N)$ solution via the Thomas algorithm.

*Newton's method is **locally quadratically convergent**: if $\mathbf{u}^{(0)}$ is sufficiently close to the solution, convergence is very fast.*

However, it is **not globally convergent**; a poor initial guess may lead to divergence. Use a linear approximation or continuation method if needed.

3.2.4 Stability and Convergence Analysis

Consistency

The finite difference scheme is derived from a second-order accurate approximation of the second derivative. Specifically, the local truncation error at node x_i is

$$\tau_i = u''(x_i) - \frac{u(x_{i-1}) - 2u(x_i) + u(x_{i+1}))}{h^2} = -\frac{h^2}{12} u^{(4)}(\xi_i), \quad \xi_i \in (x_{i-1}, x_{i+1}).$$

Hence, if the exact solution satisfies $u \in C^4[0, 1]$, then $|\tau_i| \leq Ch^2$ for some constant $C > 0$ independent of h . Since $F(u(x_i))$ and $f(x_i)$ are evaluated exactly (no discretization error in the nonlinear and source terms), the residual of the discrete system satisfies

$$G_i(\mathbf{u}_{\text{exact}}) = \tau_i = O(h^2), \quad i = 1, \dots, N-1,$$

where $\mathbf{u}_{\text{exact}} = [u(x_1), \dots, u(x_{N-1})]^T$. Thus, the scheme is second-order consistent.

Stability

Stability here refers to continuous dependence of the discrete solution on the data, and is guaranteed by the structure of the discrete operator.

The linear part of the scheme corresponds to the discrete Laplacian

$$(L_h \mathbf{u})_i = -\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}, \quad i = 1, \dots, N-1,$$

with $u_0 = u_N = 0$. The associated matrix $A_h \in \mathbb{R}^{(N-1) \times (N-1)}$ is:

- symmetric,
- strictly diagonally dominant,
- positive definite,
- with eigenvalues $\lambda_k = \frac{4}{h^2} \sin^2\left(\frac{k\pi h}{2}\right)$, $k = 1, \dots, N-1$,
- and smallest eigenvalue satisfying $\lambda_1 \geq \pi^2$ (independent of h).

Now suppose $F \in C^1(\mathbb{R})$ and satisfies the monotonicity condition

$$F'(s) \geq 0 \quad \text{for all } s \in \mathbb{R}.$$

Then the nonlinear discrete operator $\mathcal{L}_h(\mathbf{u}) = L_h \mathbf{u} + \mathbf{F}(\mathbf{u})$, where $\mathbf{F}(\mathbf{u})_i = F(u_i)$, is:

- **monotone:** $(\mathcal{L}_h(\mathbf{u}) - \mathcal{L}_h(\mathbf{v}))^\top (\mathbf{u} - \mathbf{v}) \geq 0$ for all $\mathbf{u}, \mathbf{v}$,
- **coercive:** $\mathcal{L}_h(\mathbf{u})^\top \mathbf{u} \rightarrow \infty$ as $\|\mathbf{u}\| \rightarrow \infty$.

These properties ensure:

- existence and uniqueness of the discrete solution $\mathbf{u}_h$,
- uniform boundedness: $\|\mathbf{u}_h\|_\infty \leq C$, where C depends only on f and F , not on h ,
- absence of nonphysical oscillations (maximum principle holds).

Moreover, the Jacobian matrix

$$J(\mathbf{u}) = A_h + \text{diag}(F'(u_1), \dots, F'(u_{N-1}))$$

inherits the same structural properties: it is symmetric, positive definite, and in fact an M-matrix (all off-diagonal entries ≤ 0 , diagonal dominance, and positive diagonal). This guarantees well-conditioned linear solves within Newton's method.

Convergence of Newton's Method

Newton's method applied to $\mathbf{G}(\mathbf{u}) = \mathbf{0}$ converges *quadratically* under the following standard conditions:

1. $\mathbf{G} \in C^1$ in a neighbourhood of the solution (satisfied since $F \in C^1$),
2. the initial guess $\mathbf{u}^{(0)}$ is sufficiently close to the exact discrete solution $\mathbf{u}_h$,
3. the Jacobian $J(\mathbf{u})$ is nonsingular and its inverse is uniformly bounded in that neighbourhood.

The monotonicity condition $F'(u) \geq 0$ ensures $J(\mathbf{u}) \succeq A_h$ (in the sense of positive definiteness), and since the smallest eigenvalue of A_h is bounded below by $\pi^2 > 0$, we have

$$\|J(\mathbf{u})^{-1}\|_2 \leq \frac{1}{\pi^2},$$

uniformly in h and $\mathbf{u}$. Consequently, once the iterate enters the basin of attraction, Newton's method exhibits quadratic convergence:

$$\|\mathbf{u}^{(k+1)} - \mathbf{u}_h\| \leq C \|\mathbf{u}^{(k)} - \mathbf{u}_h\|^2.$$

Overall Convergence of the Finite Difference Method

Combining consistency and stability yields convergence of the numerical solution to the true solution.

Theorem 3.2.1 (Second-Order Convergence). *Assume:*

- the exact solution $u \in C^4[0, 1]$,
- $F \in C^1(\mathbb{R})$ with $F'(s) \geq 0$ for all s ,
- $f \in C[0, 1]$.

Let $\mathbf{u}_h = [u_1, \dots, u_{N-1}]^T$ be the unique solution of the discrete nonlinear system. Then there exists a constant $C > 0$, independent of h , such that

$$\max_{0 \leq i \leq N} |u_i - u(x_i)| \leq Ch^2.$$

Sketch. Let $\mathbf{e} = \mathbf{u}_h - \mathbf{u}_{\text{exact}}$ be the error vector. Using a Taylor expansion and the mean value theorem, one obtains

$$\mathbf{G}(\mathbf{u}_h) - \mathbf{G}(\mathbf{u}_{\text{exact}}) = J(\boldsymbol{\xi}) \mathbf{e},$$

for some $\boldsymbol{\xi}$ on the line segment between $\mathbf{u}_h$ and $\mathbf{u}_{\text{exact}}$. Since $\mathbf{G}(\mathbf{u}_h) = \mathbf{0}$ and $\|\mathbf{G}(\mathbf{u}_{\text{exact}})\| = O(h^2)$ by consistency, it follows that

$$\|\mathbf{e}\| \leq \|J(\boldsymbol{\xi})^{-1}\| \cdot \|\mathbf{G}(\mathbf{u}_{\text{exact}})\| \leq Ch^2,$$

where uniform boundedness of $\|J^{-1}\|$ (stability) and the $O(h^2)$ truncation error (consistency) are used. A discrete maximum principle then extends the bound from the interior nodes to the full grid. $\square$

Example 3.4. Consider the nonlinear BVP:

$$-u''(x) + 1 + e^{u(x)} + u(x)^4 = x + x^2, \quad x \in (0, 1),$$

with $u(0) = u(1) = 0$.

Here, $F(u) = 1 + e^u + u^4$, so $F'(u) = e^u + 4u^3 \geq 0$ for all $u \in \mathbb{R}$.

The residual components:

$$G_i(\mathbf{u}) = -\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + 1 + e^{u_i} + u_i^4 - x_i - x_i^2.$$

The Jacobian matrix is:

$$J(\mathbf{u}) = \begin{bmatrix} \frac{2}{h^2} + e^{u_1} + 4u_1^3 & -\frac{1}{h^2} & & & \\ -\frac{1}{h^2} & \frac{2}{h^2} + e^{u_2} + 4u_2^3 & \ddots & & \\ & & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + e^{u_{N-1}} + 4u_{N-1}^3 \end{bmatrix}$$

Using initial guess $\mathbf{u}^{(0)} = \mathbf{0}$ and $\text{TOL} = 10^{-6}$, Newton's method typically converges in 3-5 iterations.

```
% Nonlinear BVP: -u'' + F(u) = f(x),    u(0) = u(1) = 0
clc; clear; close all;

% Problem data
F = @(u) 1 + exp(u) + u.^4;
dF = @(u) exp(u) + 4*u.^3;
f_func = @(x) x + x.^2;
u0 = 0; u1 = 0;
TOL = 1e-6;
max_iter = 20;

% Grids
N_vals = [16, 32, 64];

fprintf('N\tIterations\tResidual\n');
fprintf('-----\n');

% To store finest solution and residual for plotting
U_plot = []; x_plot = []; R_plot = [];

for k = 1:length(N_vals)
    N = N_vals(k);
    h = 1/N;
    x = linspace(0, 1, N+1)';
```

```

U = zeros(N+1, 1); % initial guess

iter = 0;
residual = Inf;
while iter < max_iter
    iter = iter + 1;

    % Residual R(U)
    R = zeros(N+1, 1);
    R(1) = U(1) - u0;
    R(N+1) = U(N+1) - u1;
    for i = 2:N
        neg_u2 = (-U(i-1) + 2*U(i) - U(i+1)) / h^2; % -u''
        R(i) = neg_u2 + F(U(i)) - f_func(x(i));
    end
    residual = norm(R(2:N), inf);
    if residual < TOL, break; end

    % Jacobian (CORRECTED)
    J = spalloc(N+1, N+1, 3*(N+1));
    J(1,1) = 1;
    J(N+1,N+1) = 1;
    for i = 2:N
        J(i, i-1) = -1/h^2;
        J(i, i) = 2/h^2 + dF(U(i));
        J(i, i+1) = -1/h^2;
    end

    % Newton update
    dU = J \ R;
    U = U - dU;
end

fprintf('%d\t%d\t\t%.2e\n', N, iter, residual);

% Save finest grid solution and residual for plotting
if N == max(N_vals)
    U_plot = U;
    x_plot = x;
    R_plot = R;
end

```

```

end

% ----- Plotting -----
figure('Position',[100,100,1000,400]);

% Plot 1: Approximate solution
subplot(1,2,1);
plot(x_plot, U_plot, '-o', 'MarkerSize',4, 'LineWidth',1.5);
xlabel('x');
ylabel('u_h(x)');
title(sprintf('Approximate Solution (N = %d)', max(N_vals)));
grid on;

% Plot 2: Residual R(x)
subplot(1,2,2);
plot(x_plot, R_plot, '-s', 'MarkerSize',4, 'LineWidth',1.5);
xlabel('x');
ylabel('Residual R_i');
title('Residual Vector at Convergence');
grid on;
ylim([-max(abs(R_plot))*1.1, max(abs(R_plot))*1.1]);
% Optional: highlight interior vs boundaries
hold on;
plot(x_plot([1,end]), R_plot([1,end]), 'ro', 'MarkerSize',8, 'DisplayName','
    Boundary');
legend('R_i','Boundary');
hold off;

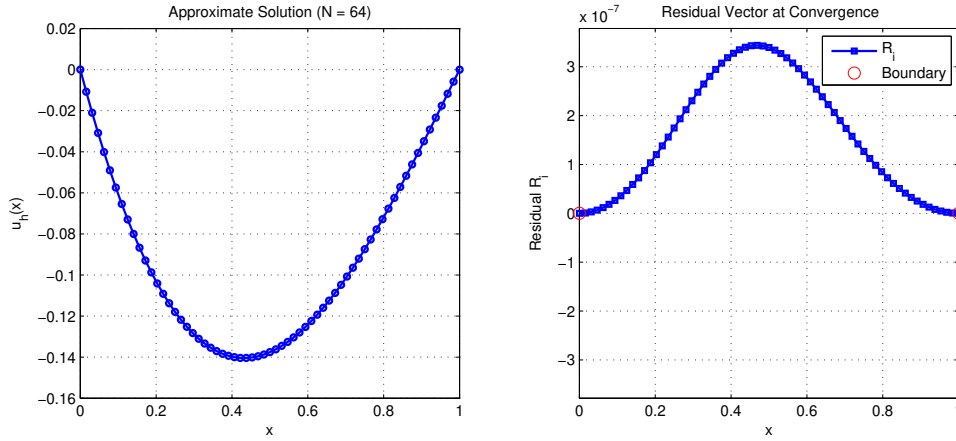
```

Table 3.9: Newton iteration performance for the nonlinear BVP $-u'' + 1 + e^u + u^4 = x + x^2$, $u(0) = u(1) = 0$.

N	Iterations	Residual ($\ R\ _\infty$)
16	3	3.42×10^{-7}
32	3	3.44×10^{-7}
64	3	3.44×10^{-7}

The finite difference method combined with Newton's iteration is a powerful tool for solving nonlinear BVPs:

- **High accuracy:** $O(h^2)$ spatial convergence,
- **Efficient:** tridiagonal Jacobian allows $O(N)$ solve,

Figure 3.8: Numerical solution and residual for $N = 64$

- **Robust:** under monotonicity of F ,
- **Stable:** due to M-matrix structure.

3.2.5 General Nonlinear BVP: $u'' = f(x, u, u')$

Consider the second-order nonlinear boundary value problem

$$u''(x) = f(x, u(x), u'(x)), \quad 0 < x < 1, \quad (3.38)$$

subject to Dirichlet boundary conditions

$$u(0) = u_0, \quad u(1) = u_1,$$

where $f : [0, 1] \times \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is a given smooth function.

Finite Difference Discretization

Introduce a uniform grid

$$x_i = ih, \quad i = 0, 1, \dots, N+1, \quad h = \frac{1}{N+1},$$

so that $x_0 = 0$, $x_{N+1} = 1$. Let $u_i \approx u(x_i)$ denote the numerical approximation, with $u_0 = u_0$ and $u_{N+1} = u_1$ prescribed.

We approximate derivatives using second-order central differences:

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}, \quad (3.39)$$

$$u'(x_i) \approx \frac{u_{i+1} - u_{i-1}}{2h}. \quad (3.40)$$

Substituting (3.39)–(3.40) into (3.38) yields, for each interior node $i = 1, \dots, N$, the nonlinear algebraic equation:

$$G_i(\mathbf{u}) = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} - f\left(x_i, u_i, \frac{u_{i+1} - u_{i-1}}{2h}\right) = 0, \quad (3.41)$$

where $\mathbf{u} = [u_1, u_2, \dots, u_N]^\top \in \mathbb{R}^N$ is the vector of unknowns.

Thus, the discrete problem is to find $\mathbf{u} \in \mathbb{R}^N$ such that

$$\mathbf{G}(\mathbf{u}) = \mathbf{0}, \quad \mathbf{G} = [G_1, \dots, G_N]^\top.$$

Newton's Method

Since $\mathbf{G}$ is nonlinear (in general), we solve $\mathbf{G}(\mathbf{u}) = \mathbf{0}$ using Newton's method:

Given an initial guess $\mathbf{u}^{(0)}$, iterate for $k = 0, 1, 2, \dots$:

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} - [J(\mathbf{u}^{(k)})]^{-1} \mathbf{G}(\mathbf{u}^{(k)}), \quad (3.42)$$

where $J(\mathbf{u}) = \partial \mathbf{G} / \partial \mathbf{u} \in \mathbb{R}^{N \times N}$ is the Jacobian matrix.

The entries of J are obtained by differentiating (3.41):

$$\begin{aligned} \frac{\partial G_i}{\partial u_{i-1}} &= \frac{1}{h^2} + \frac{1}{2h} f_{u'}\left(x_i, u_i, \frac{u_{i+1} - u_{i-1}}{2h}\right), \\ \frac{\partial G_i}{\partial u_i} &= -\frac{2}{h^2} - f_u\left(x_i, u_i, \frac{u_{i+1} - u_{i-1}}{2h}\right), \\ \frac{\partial G_i}{\partial u_{i+1}} &= \frac{1}{h^2} - \frac{1}{2h} f_{u'}\left(x_i, u_i, \frac{u_{i+1} - u_{i-1}}{2h}\right), \end{aligned}$$

where $f_u = \partial f / \partial u$ and $f_{u'} = \partial f / \partial u'$ are evaluated at the current iterate. All other partials are zero, so $J(\mathbf{u})$ is **tridiagonal**.

At each Newton step:

1. Evaluate residual $\mathbf{G}(\mathbf{u}^{(k)})$,

2. Assemble Jacobian $J(\mathbf{u}^{(k)})$,
3. Solve the linear system $J(\mathbf{u}^{(k)})\mathbf{w}^{(k)} = \mathbf{G}(\mathbf{u}^{(k)})$ (e.g., via Thomas algorithm, $O(N)$),
4. Update $\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} - \mathbf{w}^{(k)}$.

The iteration terminates when $\|\mathbf{G}(\mathbf{u}^{(k)})\|_\infty < \text{TOL}$ (e.g., $\text{TOL} = 10^{-8}$).

Consistency

Assume the exact solution $u \in C^4[0, 1]$. The truncation error of the second derivative approximation is

$$u''(x_i) - \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1}))}{h^2} = -\frac{h^2}{12}u^{(4)}(\xi_i),$$

and for the first derivative,

$$u'(x_i) - \frac{u(x_{i+1}) - u(x_{i-1}))}{2h} = -\frac{h^2}{6}u^{(3)}(\eta_i),$$

for some $\xi_i, \eta_i \in (x_{i-1}, x_{i+1})$.

Using a Taylor expansion of f and the smoothness of f , one finds the local residual at the exact solution satisfies

$$G_i(\mathbf{u}_{\text{exact}}) = O(h^2), \quad i = 1, \dots, N,$$

i.e., the scheme is second-order consistent.

Stability

Stability of the nonlinear scheme hinges on the properties of f . A standard sufficient condition is the **uniform monotonicity** of the continuous operator, which translates to assumptions on the partial derivatives of f . Suppose there exist constants $\alpha > 0$, $\beta \geq 0$ such that for all (x, u, p) ,

$$f_u(x, u, p) \geq \alpha > 0, \quad |f_{u'}(x, u, p)| \leq \beta. \quad (3.43)$$

Under (3.43), the Jacobian $J(\mathbf{u})$ is strictly diagonally dominant for sufficiently small h :

$$\left| \frac{\partial G_i}{\partial u_i} \right| = \left| \frac{2}{h^2} + f_u \right| \geq \frac{2}{h^2} + \alpha,$$

while

$$\left| \frac{\partial G_i}{\partial u_{i-1}} \right| + \left| \frac{\partial G_i}{\partial u_{i+1}} \right| \leq \frac{2}{h^2} + \frac{\beta}{h}.$$

Thus, diagonal dominance holds provided

$$\frac{2}{h^2} + \alpha > \frac{2}{h^2} + \frac{\beta}{h} \iff h < \frac{\alpha}{\beta} \quad (\text{if } \beta > 0).$$

In particular:

- If $f_u \equiv 0$ (no dependence on u'), then J is diagonally dominant for all $h > 0$,
- If $f_u \neq 0$, diagonal dominance (and hence invertibility) is guaranteed for sufficiently fine meshes.

Moreover, the diagonal entries of J are positive and off-diagonal entries are bounded, ensuring that J is an **M-matrix** when h is small enough—hence $J^{-1} \geq 0$ elementwise and $\|J^{-1}\|_\infty$ is uniformly bounded. This guarantees well-posed linear solves and prevents spurious oscillations.

Convergence of the Discrete Solution

We now establish convergence of the numerical solution u_i to the exact solution $u(x_i)$.

Theorem 3.2.2 (Convergence). *Assume:*

1. $u \in C^4[0, 1]$ is the unique solution of (3.38),
2. $f \in C^2([0, 1] \times \mathbb{R} \times \mathbb{R})$,
3. The monotonicity condition (3.43) holds,
4. The Newton iteration converges to a solution $\mathbf{u}_h$ of $\mathbf{G}(\mathbf{u}) = \mathbf{0}$.

Then there exists $h_0 > 0$ and a constant $C > 0$, independent of h , such that for all $0 < h \leq h_0$,

$$\max_{0 \leq i \leq N+1} |u_i - u(x_i)| \leq Ch^2.$$

Proof. Let $\mathbf{e} = \mathbf{u}_h - \mathbf{u}_{\text{exact}}$ be the error vector (restricted to interior nodes). By the mean value theorem applied componentwise to $\mathbf{G}$, there exists $\boldsymbol{\xi}$ on the segment between $\mathbf{u}_h$ and $\mathbf{u}_{\text{exact}}$ such that

$$\mathbf{0} - \mathbf{G}(\mathbf{u}_{\text{exact}}) = \mathbf{G}(\mathbf{u}_h) - \mathbf{G}(\mathbf{u}_{\text{exact}}) = J(\boldsymbol{\xi}) \mathbf{e}.$$

Hence,

$$\mathbf{e} = -J(\boldsymbol{\xi})^{-1} \mathbf{G}(\mathbf{u}_{\text{exact}}).$$

By consistency, $\|\mathbf{G}(\mathbf{u}_{\text{exact}})\|_{\infty} = O(h^2)$. By stability (M-matrix property), $\|J(\boldsymbol{\xi})^{-1}\|_{\infty} \leq M$ for some $M > 0$ independent of h . Therefore,

$$\|\mathbf{e}\|_{\infty} \leq M \cdot O(h^2) = O(h^2).$$

Since u_0 and u_{N+1} are exact by construction, the full-grid error satisfies

$$\max_{0 \leq i \leq N+1} |u_i - u(x_i)| = O(h^2).$$

□

Convergence of Newton's Iteration

Newton's method converges *quadratically* if the initial guess is sufficiently close to the discrete solution $\mathbf{u}_h$, and if $J(\mathbf{u})$ is Lipschitz continuous and uniformly nonsingular in a neighbourhood of $\mathbf{u}_h$. The latter holds under (3.43), since

$$\lambda_{\min}(J(\mathbf{u})) \geq \frac{2}{h^2} + \alpha - \frac{\beta}{h} \geq c > 0$$

for $h < \alpha/\beta$ (or all h if $\beta = 0$). Hence, once $\|\mathbf{u}^{(0)} - \mathbf{u}_h\|$ is small enough,

$$\|\mathbf{u}^{(k+1)} - \mathbf{u}_h\| \leq C \|\mathbf{u}^{(k)} - \mathbf{u}_h\|^2.$$

In practice, a good initial guess can be obtained from:

- the solution of the linearized problem (f linearized about $u = \text{linear interpolant}$),
- continuation in a parameter (e.g., scale $f \rightarrow \lambda f$ with $\lambda \in [0, 1]$),
- or simply $\mathbf{u}^{(0)} = \text{linear interpolation between } u_0 \text{ and } u_1$ (often sufficient for moderate nonlinearity).

Thus, under reasonable smoothness and monotonicity assumptions, the central-difference discretization of the general nonlinear BVP (3.38) is second-order accurate, stable, and efficiently solvable via Newton's method with quadratic convergence.

Example 3.5 (Lab work). *Using finite difference method approximate the solution of $y'' = (32 + 2x^3 - yy')$, for $1 < x < 3$ with $y(1) = 17$ and $y(3) = 43/3$ for $N = 10, 20, 40, 80$. Take tolerance $tol = 10^{-5}$ and number of iteration $m = 10$. The exact solution is $y(x) = x^2 + 16/x$.*

Solution: Let $N \in \mathbb{N}$ be the number of subintervals and define the uniform step size

$$h = \frac{3-1}{N} = \frac{2}{N}. \quad (3.44)$$

The grid points are given by

$$x_i = 1 + ih, \quad i = 0, 1, \dots, N. \quad (3.45)$$

Denote the numerical approximation to $u(x_i)$ by u_i . The boundary values are fixed:

$$u_0 = 17, \quad u_N = \frac{43}{3}. \quad (3.46)$$

Using second-order central differences:

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}, \quad (3.47)$$

$$u'(x_i) \approx \frac{u_{i+1} - u_{i-1}}{2h}, \quad (3.48)$$

the differential equation at interior node x_i ($i = 1, 2, \dots, N-1$) becomes

$$\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} = \frac{1}{8} \left(32 + 2x_i^3 - u_i \cdot \frac{u_{i+1} - u_{i-1}}{2h} \right). \quad (3.49)$$

Multiplying both sides by h^2 and rearranging yields the nonlinear finite difference equation:

$$\left(1 + \frac{h u_i}{16}\right) u_{i-1} - 2u_i + \left(1 - \frac{h u_i}{16}\right) u_{i+1} = \frac{h^2}{8} (32 + 2x_i^3) \quad (i = 1, \dots, N-1). \quad (3.50)$$

Nonlinear System and Solution Strategy

Define the residual vector $\mathbf{F}(\mathbf{u}) = [F_1, F_2, \dots, F_{N-1}]^\top$ with components

$$F_i(\mathbf{u}) = \left(1 + \frac{h u_i}{16}\right) u_{i-1} - 2u_i + \left(1 - \frac{h u_i}{16}\right) u_{i+1} - \frac{h^2}{8} (32 + 2x_i^3), \quad (3.51)$$

where $u_0 = 17$ and $u_N = 43/3$ are imposed.

The system $\mathbf{F}(\mathbf{u}) = \mathbf{0}$ is solved using Newton's method:

- (i) Choose an initial guess $\mathbf{u}^{(0)}$ (e.g., linear interpolation or exact solution).
- (ii) For $k = 0, 1, \dots, m - 1$ (with $m = 10$):
 - Assemble the Jacobian matrix $J^{(k)} = \frac{\partial \mathbf{F}}{\partial \mathbf{u}} \big|_{\mathbf{u}^{(k)}}$, which is tridiagonal. For $2 \leq i \leq N - 2$,

$$\frac{\partial F_i}{\partial u_{i-1}} = 1 + \frac{h u_i^{(k)}}{16}, \quad (3.52)$$

$$\frac{\partial F_i}{\partial u_i} = \frac{h}{16} (u_{i-1}^{(k)} - u_{i+1}^{(k)}) - 2, \quad (3.53)$$

$$\frac{\partial F_i}{\partial u_{i+1}} = 1 - \frac{h u_i^{(k)}}{16}. \quad (3.54)$$

Boundary-adjacent rows ($i = 1$ and $i = N - 1$) incorporate the known u_0 and u_N analogously.

- Solve the linear system $J^{(k)} \Delta \mathbf{u}^{(k)} = -\mathbf{F}(\mathbf{u}^{(k)})$.
- Update: $\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \Delta \mathbf{u}^{(k)}$.
- Stop if $\|\Delta \mathbf{u}^{(k)}\|_\infty < \text{tol} = 10^{-5}$.

Table 3.10: Convergence results for the nonlinear BVP.

N	h	$\ e\ _\infty$	Order	Newton iters
10	0.1818	8.152e-3	—	4
20	0.0952	2.227e-3	1.87	4
40	0.0488	5.840e-4	1.93	4
80	0.0247	1.496e-4	1.96	4

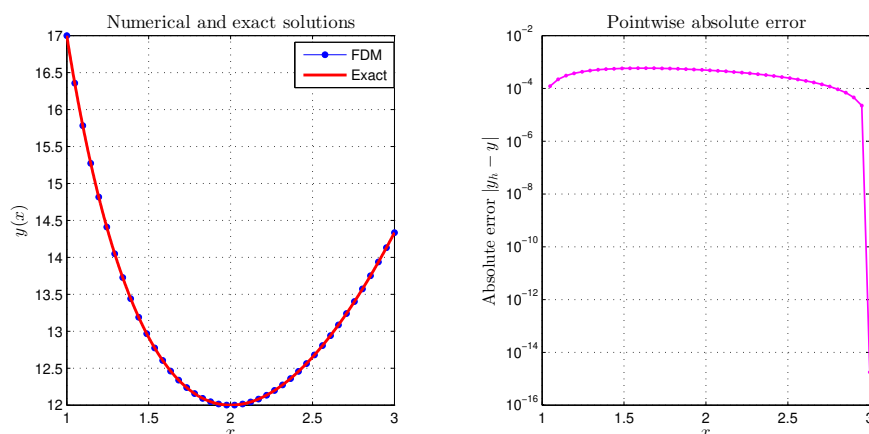


Figure 3.9: Comparison of Exact solution with FDM in left side, Absolute error on right side.

```

%% Lab Work: Nonlinear BVP  $y' = (32 + 2x^3 - y y')/8$ ,  $y(1)=17$ ,  $y(3)=43/3$ 
% Exact solution:  $y(x) = x^2 + 16/x$ 
clear; clc; close all;
% Problem data
a = 1; b = 3; % domain [a,b]
ya = 17; yb = 43/3; % Dirichlet BCs
f = @(x, y, yp) (32 + 2*x.^3 - y.*yp)/8;
y_exact = @(x) x.^2 + 16./x;
% Grid refinements
Ns = [10, 20, 40, 80];
tol = 1e-5;
max_iter = 10;
errors_inf = zeros(size(Ns));
errors_l2 = zeros(size(Ns));
fprintf('Nonlinear BVP:  $y' = (32 + 2x^3 - y y')/8$ ,  $y(1)=17$ ,  $y(3)=43/3$ \n');
;
fprintf('Exact:  $y(x) = x^2 + 16/x$ \n');
fprintf(' N | h | ||e||_inf | | Newton iters\n');
fprintf('-----+-----+-----+-----\n');
for k = 1:length(Ns)
    N = Ns(k);
    h = (b - a) / (N + 1); % uniform step
    x = linspace(a, b, N + 2)'; %  $x_0=1$ ,  $x_{N+1}=3$ 
    % Initial guess: linear interpolation
    y = linspace(ya, yb, N + 2)'; % full vector
    y = y(2:end-1); % interior only:  $y_1 \dots y_N$ 

```

```

% Newton iteration
for iter = 1:max_iter
    G = zeros(N, 1);
    J = spalloc(N, N, 3*N);           % sparse tridiagonal preallocation
    for i = 1:N
        xi = x(i+1);
        yi = y(i);
        % Neighboring values (include BCs)
        if i == 1
            y_im1 = ya;
        else
            y_im1 = y(i-1);
        end
        if i == N
            y_ip1 = yb;
        else
            y_ip1 = y(i+1);
        end
        % Central differences
        ypp = (y_ip1 - 2*yi + y_im1) / h^2;
        yp = (y_ip1 - y_im1) / (2*h);
        % Residual: G_i = y''_i - f(x_i, y_i, y'_i)
        G(i) = ypp - f(xi, yi, yp);
        % Partial derivatives of f
        fy = -yp / 8;           % f/y = -y'/8
        fyp = -yi / 8;          % f/y' = -y/8
        % Jacobian: dG_i / dy_j
        J(i,i) = -2/h^2 - fy;   % diagonal
        if i > 1
            J(i,i-1) = 1/h^2 + (1/(2*h)) * fyp; % sub-diagonal
        end
        if i < N
            J(i,i+1) = 1/h^2 - (1/(2*h)) * fyp; % super-diagonal
        end
    end
    % Solve linear system: J * delta = -G
    delta = J \ (-G);
    y = y + delta;
    % Check convergence
    if norm(G, inf) < tol
        break;
    end
end

```

```

        end
    end
    % Assemble full solution (with boundaries)
    y_num = [ya; y; yb];
    y_ex = y_exact(x);
    e = y_num - y_ex;
    err_inf = norm(e, inf);
    err_l2 = sqrt(h) * norm(e, 2);
    errors_inf(k) = err_inf;
    errors_l2(k) = err_l2;
    fprintf('%3d | %6.4f | %11.3e | %8.2e | %13d\n', ...
        N, h, err_inf, err_l2, iter);
    % --- Plot: Numerical vs Exact Solution ---
figure('Position', [200, 120, 1000, 420]);

% Fine grid for exact solution (smooth curve)
x_fine = linspace(a, b, 1000);
y_ex_fine = y_exact(x_fine);
% Left subplot: solution
subplot(1, 2, 1);
plot(x, y_num, 'bo-', 'MarkerFaceColor', 'b', 'MarkerSize', 4, ...
    'DisplayName', sprintf('FD (N = %d)', N));
hold on;
plot(x_fine, y_ex_fine, 'r-', 'LineWidth', 2, 'DisplayName', 'Exact');
xlabel('$x$', 'Exact', 'FDM', 'FontSize', 12);
ylabel('$y(x)$', 'Exact', 'FDM', 'FontSize', 12);
title('Numerical and exact solutions', 'Interpreter', 'latex', 'FontSize', 13);
;
legend('Location', 'northwest', 'Exact', 'FDM', 'FontSize', 11);
grid on; box on;

% Right subplot: absolute error
subplot(1, 2, 2);
semilogy(x, abs(e), 'm.-', 'MarkerSize', 8, 'LineWidth', 1.2);
xlabel('$x$', 'Exact', 'DFM', 'FontSize', 12);
ylabel('Absolute error $|y_h - y|$', 'Interpreter', 'latex', 'FontSize', 12);
title('Pointwise absolute error', 'Exact', 'FDM', 'FontSize', 13);
grid on; box on;
end
% Convergence rates
fprintf('\nConvergence (||e||_{\infty}):\n');

```

```
fprintf('  N      |  Order\n');  
fprintf('-----+\n');  
for k = 2:length(Ns)  
    order = log(errors_inf(k-1)/errors_inf(k)) / log(2);  
    fprintf('%4d      |  %5.2f\n', Ns(k), order);  
end
```


Chapter 4

Explicit Numerical Method for PDEs

Notation for Functions of Several Variables

Assume u is a function of two independent variables: space x and time t . To numerically solve partial differential equations involving $u(x, t)$, we discretize the x - t plane using a uniform grid.

Discretization

We discretize the domain $[a, b] \times [0, T]$ using a uniform grid. Let: N : number of spatial subintervals, M : number of temporal subintervals, $h = \frac{b-a}{N}$: spatial step size, $\Delta t = \frac{T}{M}$: time step size.

Define grid points:

$$\begin{aligned}x_i &= a + ih, \quad i = 0, 1, \dots, N, \\t_j &= j\Delta t, \quad j = 0, 1, \dots, M.\end{aligned}$$

Let $U_{i,j}$ denote the numerical approximation to $u(x_i, t_j)$. For the unit square, often $N = M$, so the number of nodal points in both directions is equal.

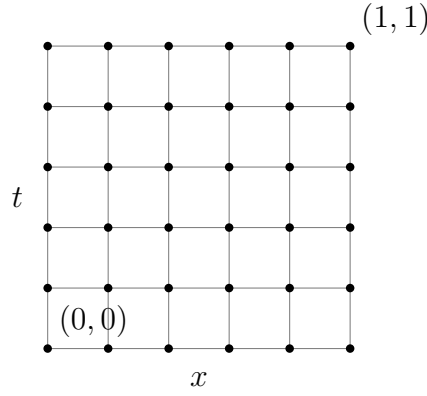


Figure 4.1: Representation of 2D grid on (x, t) coordinate.

Discrete Notation

Let $U_{i,j}$ denote the approximate value of u at the mesh point (x_i, t_j) :

$$U_{i,j} \approx u(x_i, t_j) = u(ih, j\tau).$$

Finite Difference Approximations

We now derive finite difference approximations for the partial derivatives of u , using Taylor series expansions.

Second-Order Central Difference in Space

The second derivative $\frac{\partial^2 u}{\partial x^2}$ at point (x_i, t_j) is approximated using the central difference formula:

$$\frac{\partial^2 u}{\partial x^2}(x_i, t_j) \approx \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2}$$

From Taylor's theorem:

$$U_{i\pm 1,j} = U_{i,j} \pm h \frac{\partial u}{\partial x} + \frac{h^2}{2!} \frac{\partial^2 u}{\partial x^2} \pm \frac{h^3}{3!} \frac{\partial^3 u}{\partial x^3} + \frac{h^4}{4!} \frac{\partial^4 u}{\partial x^4} + O(h^5)$$

Adding $U_{i-1,j}$ and $U_{i+1,j}$:

$$U_{i-1,j} + U_{i+1,j} = 2U_{i,j} + h^2 \frac{\partial^2 u}{\partial x^2} + \frac{h^4}{12} \frac{\partial^4 u}{\partial x^4} + O(h^6)$$

Solving for $\frac{\partial^2 u}{\partial x^2}$:

$$\frac{\partial^2 u}{\partial x^2} = \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2} - \frac{h^2}{12} \frac{\partial^4 u}{\partial x^4} + O(h^4)$$

Thus, the approximation is:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2}, \quad \text{with leading error } O(h^2) \quad (4.1)$$

Second-Order Central Difference in Time

Similarly, for the second derivative in time:

$$\frac{\partial^2 u}{\partial t^2}(x_i, t_j) \approx \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{\tau^2}$$

Using Taylor expansion in time:

$$U_{i,j\pm 1} = U_{i,j} \pm \tau \frac{\partial u}{\partial t} + \frac{\tau^2}{2!} \frac{\partial^2 u}{\partial t^2} \pm \frac{\tau^3}{3!} \frac{\partial^3 u}{\partial t^3} + \frac{\tau^4}{4!} \frac{\partial^4 u}{\partial t^4} + O(\tau^5)$$

Adding:

$$U_{i,j-1} + U_{i,j+1} = 2U_{i,j} + \tau^2 \frac{\partial^2 u}{\partial t^2} + \frac{\tau^4}{12} \frac{\partial^4 u}{\partial t^4} + O(\tau^6)$$

So:

$$\frac{\partial^2 u}{\partial t^2} = \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{\tau^2} - \frac{\tau^2}{12} \frac{\partial^4 u}{\partial t^4} + O(\tau^4)$$

Hence:

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{\tau^2}, \quad \text{with leading error } O(\tau^2) \quad (4.2)$$

Forward-Difference Approximation in Time

The first-order partial derivative $\frac{\partial u}{\partial t}$ at point $P = (x_i, t_j)$ can be approximated using the forward difference:

$$\left. \frac{\partial u}{\partial t} \right|_{(x_i, t_j)} \approx \frac{U_{i,j+1} - U_{i,j}}{\tau}$$

From Taylor expansion:

$$U_{i,j+1} = U_{i,j} + \tau \frac{\partial u}{\partial t} + \frac{\tau^2}{2} \frac{\partial^2 u}{\partial t^2} + O(\tau^3)$$

Solving:

$$\frac{\partial u}{\partial t} = \frac{U_{i,j+1} - U_{i,j}}{\tau} - \frac{\tau}{2} \frac{\partial^2 u}{\partial t^2} + O(\tau^2)$$

Thus:

$$\frac{\partial u}{\partial t} \approx \frac{U_{i,j+1} - U_{i,j}}{\tau}, \quad \text{with leading error } O(\tau) \quad (4.3)$$

Summary of Approximations

Derivative	Finite Difference Formula	Truncation Error
$\frac{\partial^2 u}{\partial x^2}$	$\frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2}$	$O(h^2)$
$\frac{\partial^2 u}{\partial t^2}$	$\frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{\tau^2}$	$O(\tau^2)$
$\frac{\partial u}{\partial t}$ (forward)	$\frac{U_{i,j+1} - U_{i,j}}{\tau}$	$O(\tau)$

4.1 Explicit Method for Parabolic PDE

We consider the one-dimensional heat (diffusion) equation:

$$\frac{\partial u}{\partial t} = a \frac{\partial^2 u}{\partial x^2}, \quad x \in (0, 1), \quad t \in (0, T], \quad (4.4)$$

with initial condition $u(x, 0) = f(x)$ and boundary conditions:

$$u(0, t) = \phi(t), \quad u(1, t) = \psi(t),$$

where $a > 0$ is the diffusion coefficient.

The **Forward-Time Central-Space (FTCS)** scheme is an explicit finite difference method that uses: Forward difference in time and Central difference in space.

Finite Difference Approximation

Let: $h = 1/N$: spatial step size, $\Delta t = T/M$: time step size, $x_i = ih$, $t_j = j\Delta t$, $U_{i,j} \approx u(x_i, t_j)$.

Using finite differences:

$$\frac{\partial u}{\partial t} \approx \frac{U_{i,j+1} - U_{i,j}}{\Delta t}, \quad \frac{\partial^2 u}{\partial x^2} \approx \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2},$$

we obtain the FTCS scheme:

$$\frac{U_{i,j+1} - U_{i,j}}{\Delta t} = a \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2}, \quad i = 1, 2, \dots, N-1, \quad j = 0, 1, \dots, M-1 \quad (4.5)$$

Solving for $U_{i,j+1}$:

$$U_{i,j+1} = U_{i,j} + r(U_{i-1,j} - 2U_{i,j} + U_{i+1,j}) \quad (4.6)$$

where $r = a \frac{\Delta t}{h^2}$ is the **mesh Fourier number**. This can be rewritten as:

$$U_{i,j+1} = rU_{i-1,j} + (1 - 2r)U_{i,j} + rU_{i+1,j} \quad (4.7)$$

Matrix Formulation

Let $\mathbf{U}^j = \begin{bmatrix} U_{1,j} \\ U_{2,j} \\ \vdots \\ U_{N-1,j} \end{bmatrix}$ be the vector of interior unknowns at time t_j . The boundary values $U_{0,j} = \phi(t_j)$, $U_{N,j} = \psi(t_j)$ are known. Then the update becomes:

$$\mathbf{U}^{j+1} = A\mathbf{U}^j + \mathbf{b}^j \quad (4.8)$$

where A is a $(N-1) \times (N-1)$ tridiagonal matrix:

$$A = \begin{bmatrix} 1-2r & r & & & \\ r & 1-2r & r & & \\ & \ddots & \ddots & \ddots & \\ & & r & 1-2r \end{bmatrix}$$

and $\mathbf{b}^j$ is a correction vector due to boundary conditions:

$$\mathbf{b}^j = r \begin{bmatrix} \phi(t_j) \\ 0 \\ \vdots \\ 0 \\ \psi(t_j) \end{bmatrix}$$

Only the first and last components are non-zero.

4.1.1 Stability Analysis (Von Neumann Method)

We perform von Neumann stability analysis assuming:

- Periodic or infinite domain (ignore boundaries),
- Homogeneous equation ($\phi = \psi = 0$),
- Constant coefficients.

Assume a solution of the form:

$$U_{i,j} = \xi^j e^{ikx_i},$$

where k is the wave number and ξ is the amplification factor.

Substitute into the homogeneous FTCS scheme:

$$\xi^{j+1} e^{ikx_i} = r \xi^j e^{ikx_{i-1}} + (1 - 2r) \xi^j e^{ikx_i} + r \xi^j e^{ikx_{i+1}}$$

Using trigonometric identities:

$$e^{i\theta} + e^{-i\theta} = (\cos\theta + i \sin\theta) + (\cos\theta - i \sin\theta) = 2 \cos\theta,$$

$$e^{i\theta} - 2 + e^{-i\theta} = 2(\cos\theta - 1) = -4 \sin^2(\theta/2),$$

$$1 - e^{-i\theta} = 1 - \cos\theta + i \sin\theta = 2 \sin^2(\theta/2) + 2i \sin(\theta/2) \cos(\theta/2),$$

Divide by $\xi^j e^{ikx_i}$:

$$\xi = r e^{-ikh} + (1 - 2r) + r e^{ikh} = (1 - 2r) + 2r \cos(kh) = 1 - 4r \sin^2\left(\frac{kh}{2}\right)$$

For stability, we require $|\xi| \leq 1$ for all k . Let $\theta = kh \in [0, \pi]$. Then:

$$\xi = 1 - 4r \sin^2(\theta/2).$$

The maximum deviation occurs when $\sin^2(\theta/2) = 1$, giving $\xi = 1 - 4r$. We require:

$$-1 \leq 1 - 4r \leq 1$$

The right inequality holds for $r \geq 0$. The left gives:

$$1 - 4r \geq -1 \quad \Rightarrow \quad 4r \leq 2 \quad \Rightarrow \quad r \leq \frac{1}{2}.$$

Stability Condition:

$$r = a \frac{\Delta t}{h^2} \leq \frac{1}{2}. \quad (4.9)$$

This is a necessary and sufficient condition for stability.

4.1.2 Consistency

Local Truncation Error (LTE)

The LTE at (x_i, t_j) is:

$$\tau_{i,j} = \frac{u(x_i, t_{j+1}) - u(x_i, t_j)}{\Delta t} - a \frac{u(x_{i-1}, t_j) - 2u(x_i, t_j) + u(x_{i+1}, t_j))}{h^2}$$

Using Taylor expansions:

$$\begin{aligned} u(x_i, t_{j+1}) &= u + u_t \Delta t + \frac{1}{2} u_{tt} \Delta t^2 + O(\Delta t^3) \\ u(x_{i\pm 1}, t_j) &= u \pm u_x h + \frac{1}{2} u_{xx} h^2 \pm \frac{1}{6} u_{xxx} h^3 + \frac{1}{24} u_{xxxx} h^4 + O(h^5) \end{aligned}$$

Then:

$$\begin{aligned} \frac{U_{i,j+1} - U_{i,j}}{\Delta t} &= u_t + \frac{1}{2} u_{tt} \Delta t + O(\Delta t^2) \\ \frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2} &= u_{xx} + \frac{1}{12} u_{xxxx} h^2 + O(h^4) \end{aligned}$$

So:

$$\tau_{i,j} = (u_t - a u_{xx}) + \frac{1}{2} u_{tt} \Delta t - \frac{a}{12} u_{xxxx} h^2 + O(\Delta t^2 + h^4)$$

Since u satisfies the PDE, $u_t = a u_{xx}$, and:

$$|\tau_{i,j}| = O(\Delta t + h^2)$$

The scheme is **consistent** with order $O(\Delta t + h^2)$.

4.1.3 Convergence

By the **Lax Equivalence Theorem**, for a linear, consistent finite difference scheme, **stability is equivalent to convergence**.

We have: Consistency: $O(\Delta t + h^2)$, Conditional stability: $r \leq 1/2$. Therefore, the FTCS

scheme is **convergent** under the stability condition, with global error:

$$\max_{i,j} |U_{i,j} - u(x_i, t_j)| = O(\Delta t + h^2)$$

as $\Delta t, h \rightarrow 0$ with $r \leq 1/2$.

Summary

Property	Result
Type	Explicit
Time accuracy	$O(\Delta t)$
Space accuracy	$O(h^2)$
LTE	$O(\Delta t + h^2)$
Stability	Conditional: $r \leq \frac{1}{2}$
Convergence	Yes, under $r \leq 1/2$
Advantages	Simple, low memory
Disadvantages	Restrictive time step

The FTCS scheme is a simple and intuitive method for solving the heat equation. However, it is **conditionally stable**, requiring $\Delta t \leq \frac{h^2}{2a}$. This becomes computationally expensive for fine spatial grids. For practical applications, **implicit methods** such as Backward Euler or Crank–Nicolson are preferred due to their unconditional stability.

Despite its limitations, the FTCS method is valuable for teaching and understanding the fundamentals of finite difference schemes.

Example 4.1. *We consider the one-dimensional heat equation:*

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, \pi], \quad t \in [0, 3]$$

with initial and boundary conditions:

$$\begin{cases} u(x, 0) = \sin(2\pi x), \\ u(0, t) = u(\pi, t) = 0. \end{cases}$$

The exact solution is: $u(x, t) = \sin(2\pi x) e^{-4\pi^2 t}$.

The numerical solutions are computed using: Explicit (FTCS) method, At each grid

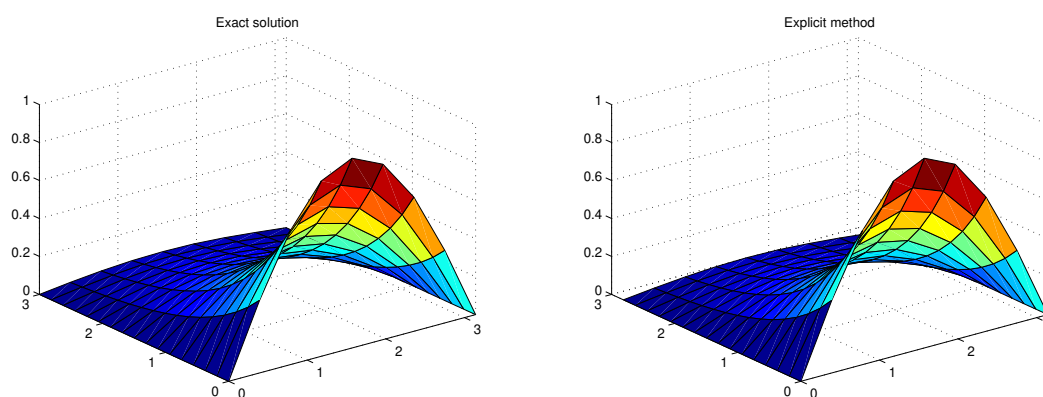


Figure 4.2: Surface plot of the solution

point (x_i, t_j) , the absolute error is computed as:

$$\text{Absolute Error} = |U_{i,j}^{\text{numerical}} - u(x_i, t_j)^{\text{exact}}|.$$

Error Comparison at Selected Points: We evaluate the absolute error at the following selected points:

```
% Numerical solution of heat equation using explicit methods
%      u_t = u_xx + for 0<x<L, t>0
% with given initial and boundary conditions u(x,0)=sinx and u(0,t)=0, u(pi,t)=0
clc; clear all; close all
fprintf('\n\n Numerical Solution of Heat Equation Using Explicit Methods\n');
T=3; % Final time
L=pi; % Domain is (0, pi)
N=2^4; % Number of intervals in x-direction
h = L/N; % x- stepsize
x = 0:h:L; % x-mesh
M=2^8; % Number of intervals in t-direction
dt = T/M; % time stepsize
t = 0:dt:T; %t-mesh
%% Plot the exact solution
figure(1);
[X,Y]=meshgrid(x,t); % full grid
u = exp(-Y).*sin(X); % solution
surf(X,Y,u); axis([0 L 0 T 0 1])
title('Exact solution');
fprintf('Figure 1  the exact solution');
```

Table 4.1: Absolute errors for the Explicit method at selected (x, t) using 20 equally spaced spatial points on $[0, \pi]$ and $M = 256$ time steps.

x	t	Explicit Absolute Error
0.0000	0.10	0.0
0.1653	0.10	1.82e-04
0.3307	0.10	3.48e-04
0.4960	0.10	4.82e-04
0.6614	0.10	5.72e-04
0.8267	0.10	6.10e-04
0.9921	0.10	5.93e-04
1.1574	0.10	5.22e-04
1.3228	0.10	4.03e-04
1.4881	0.10	2.45e-04
1.5708	0.10	1.95e-04
1.7361	0.10	2.45e-04
1.9015	0.10	4.03e-04
2.0668	0.10	5.22e-04
2.2322	0.10	5.93e-04
2.3975	0.10	6.10e-04
2.5629	0.10	5.72e-04
2.7282	0.10	4.82e-04
2.8936	0.10	3.48e-04
3.1416	0.10	0.0

```

%% Compute the numerical solution
U = zeros(M+1,N+1)+NaN; %
U(1,:) = sin(x); % initial value
U(:,1) = 0; % boundary values
U(:, N+1)=0;
% Explicit method (Forward Euler method)
for j=2:M % for each time step
    for i=2:N % for space step in the interior
        Delta2u = (U(j-1,i-1) -2*U(j-1,i) + U(j-1,i+1))/h^2;
        U(j,i) = U(j-1,i) + dt*Delta2u;
    end
    figure(2);
    surf(X,Y,U); axis([0 L 0 T min(min(U)) max(max(U))]); title('Explicit method
');
end
fprintf('Figure 2 the numerical solution using the explicit method \n');

```

```

%% Maximum absolute error computation
fprintf('\n\n-----Error Computation ----- \n');
format short e
MaxabsoExplicit=max(max(u-U))

```

4.2 Finite Difference Methods for First-Order Hyperbolic PDEs

We consider the linear advection equation:

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad c > 0 \quad (4.10)$$

with initial condition $u(x, 0) = f(x)$. We analyze several explicit finite difference schemes.

4.2.1 Forward-Time Forward-Space (FTFS) Scheme

Use forward difference in time and forward difference in space:

$$\frac{U_{i,j+1} - U_{i,j}}{\tau} + c \frac{U_{i+1,j} - U_{i,j}}{h} = 0$$

Solving:

$$U_{i,j+1} = U_{i,j} - \lambda(U_{i+1,j} - U_{i,j}) = (1 + \lambda)U_{i,j} - \lambda U_{i+1,j} \quad (4.11)$$

where $\lambda = c\tau/h$ is the **Courant number**.

Consistency and Order

The local truncation error is $O(\tau + h)$, so the scheme is **first-order accurate**.

Stability Analysis

Substitute $U_{i,j} = \xi^j e^{ikh}$ into (4.11):

$$\xi = 1 + \lambda - \lambda e^{ikh} = 1 + \lambda(1 - \cos(kh)) - i \sin(kh)$$

Then:

$$|\xi|^2 = [1 + \lambda(1 - \cos kh)]^2 + [\lambda \sin kh]^2$$

After simplification:

$$|\xi|^2 = 1 + 2\lambda(1 - \cos kh)(1 + \lambda) > 1 \quad \text{for } \lambda > 0$$

So $|\xi| > 1$ for all $\lambda > 0 \rightarrow$ **unconditionally unstable**.

4.2.2 Forward-Time Central-Space (FTCS) Scheme

$$\frac{U_{i,j+1} - U_{i,j}}{\tau} + c \frac{U_{i+1,j} - U_{i-1,j}}{2h} = 0 \quad \Rightarrow \quad U_{i,j+1} = U_{i,j} - \frac{\lambda}{2}(U_{i+1,j} - U_{i-1,j})$$

Stability Analysis

Substitute Fourier mode:

$$\xi = 1 - i\lambda \sin(kh) \quad \Rightarrow \quad |\xi|^2 = 1 + \lambda^2 \sin^2(kh) > 1 \quad \text{for } \lambda \neq 0$$

$|\xi| > 1 \rightarrow$ **unconditionally unstable**.

4.2.3 Lax–Friedrichs Scheme

Replace $U_{i,j}$ in FTCS with average of neighbors:

$$\begin{aligned} U_{i,j+1} &= \frac{1}{2}(U_{i-1,j} + U_{i+1,j}) - \frac{\lambda}{2}(U_{i+1,j} - U_{i-1,j}) \\ U_{i,j+1} &= \frac{1}{2}(1 - \lambda)U_{i+1,j} + \frac{1}{2}(1 + \lambda)U_{i-1,j} \end{aligned} \tag{4.12}$$

Stability

Substitute:

$$\begin{aligned} \xi &= \frac{1}{2}(1 - \lambda)e^{ikh} + \frac{1}{2}(1 + \lambda)e^{-ikh} = \cos(kh) - i\lambda \sin(kh) \\ |\xi|^2 &= \cos^2(kh) + \lambda^2 \sin^2(kh) \leq \max(1, \lambda^2) \end{aligned}$$

For $|\xi| \leq 1$, need $|\lambda| \leq 1$. So:

$$|\xi|^2 \leq 1 \quad \text{if } \lambda \leq 1$$

Stable if $\lambda = \frac{c\tau}{h} \leq 1$ (CFL condition). Order: $O(\tau + h)$.

4.2.4 Leap-Frog Scheme

Central difference in time and space:

$$\frac{U_{i,j+1} - U_{i,j-1}}{2\tau} + c \frac{U_{i+1,j} - U_{i-1,j}}{2h} = 0 \quad \Rightarrow \quad U_{i,j+1} = U_{i,j-1} - \lambda(U_{i+1,j} - U_{i-1,j})$$

Stability

Substitute $U_{i,j} = \xi^j e^{ikx_i}$:

$$\xi - \xi^{-1} = -2i\lambda \sin(kh) \quad \Rightarrow \quad \xi^2 + 2i\lambda \sin(kh)\xi - 1 = 0$$

Roots:

$$\xi = -i\lambda \sin(kh) \pm \sqrt{1 - \lambda^2 \sin^2(kh)}$$

If $\lambda \leq 1$, roots are complex conjugates with $|\xi| = 1$. **Stable if** $\lambda \leq 1$, and $|\xi| = 1 \rightarrow$ no dissipation. Order: $O(\tau^2 + h^2)$.

4.2.5 Lax-Wendroff Scheme

Obtained by Taylor expansion in time:

$$u(x, t + \tau) = u + \tau u_t + \frac{\tau^2}{2} u_{tt} + O(\tau^3)$$

From PDE: $u_t = -cu_x$, $u_{tt} = c^2 u_{xx}$. So:

$$u(x, t + \tau) = u - c\tau u_x + \frac{c^2 \tau^2}{2} u_{xx} + O(\tau^3)$$

Now approximate derivatives:

$$\begin{aligned} U_{i,j+1} &= U_{i,j} - \frac{\lambda}{2}(U_{i+1,j} - U_{i-1,j}) + \frac{\lambda^2}{2}(U_{i+1,j} - 2U_{i,j} + U_{i-1,j}) \\ U_{i,j+1} &= (1 - \lambda^2)U_{i,j} + \frac{\lambda}{2}(1 - \lambda)U_{i+1,j} + \frac{\lambda}{2}(1 + \lambda)U_{i-1,j} \end{aligned} \quad (4.13)$$

Stability

Substitute Fourier mode:

$$\xi = (1 - \lambda^2) + \frac{\lambda}{2}(1 - \lambda)e^{ikh} + \frac{\lambda}{2}(1 + \lambda)e^{-ikh}$$

After simplification:

$$\xi = 1 - i\lambda \sin(kh) - 2\lambda^2 \sin^2(kh/2)$$

Then:

$$|\xi|^2 = \left(1 - 2\lambda^2 \sin^2 \frac{kh}{2}\right)^2 + (\lambda \sin kh)^2$$

It can be shown that $|\xi| \leq 1$ iff $\lambda \leq 1$. **Stable if** $\lambda \leq 1$. Second-order accurate: $O(\tau^2 + h^2)$.

4.2.6 Summary of Explicit Schemes

Table 4.2: Explicit finite difference schemes for the advection equation

Method	Scheme	Stability	Order
FTFS	$U_{i,j+1} = (1 + \lambda)U_{i,j} - \lambda U_{i+1,j}$	Unstable	$O(\tau + h)$
FTCS	$U_{i,j+1} = U_{i,j} - \frac{\lambda}{2}(U_{i+1,j} - U_{i-1,j})$	Unstable	$O(\tau + h)$
Lax–Friedrichs	$U_{i,j+1} = \frac{1}{2}(U_{i-1,j} + U_{i+1,j}) - \frac{\lambda}{2}(U_{i+1,j} - U_{i-1,j})$	$\lambda \leq 1$	$O(\tau + h)$
Leap-Frog	$U_{i,j+1} = U_{i,j-1} - \lambda(U_{i+1,j} - U_{i-1,j})$	$\lambda \leq 1$	$O(\tau^2 + h^2)$
Lax–Wendroff	$U_{i,j+1} = (1 - \lambda^2)U_{i,j} + \frac{\lambda}{2}(1 - \lambda)U_{i+1,j} + \frac{\lambda}{2}(1 + \lambda)U_{i-1,j}$	$\lambda \leq 1$	$O(\tau^2 + h^2)$

Example 4.2. Solve the advection equation:

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = 0, \quad x \in [0, 2\pi], \quad t > 0$$

with initial condition $u(x, 0) = \sin(x)$. The exact solution is:

$$u(x, t) = \sin(x - t)$$

Use $h = \pi/5 \approx 0.6283$, $\tau = 0.1$, so $\lambda = \tau/h = 0.1/(\pi/5) \approx 0.15915 < 1$. Compute the absolute error using the following methods:

$$FTFS: u_i^{n+1} = u_i^n - \lambda(u_{i+1}^n - u_i^n)$$

$$FTCS: u_i^{n+1} = u_i^n - \frac{\lambda}{2}(u_{i+1}^n - u_{i-1}^n)$$

$$\text{Lax-F: } u_i^{n+1} = \frac{1}{2}(u_{i-1}^n + u_{i+1}^n) - \frac{\lambda}{2}(u_{i+1}^n - u_{i-1}^n)$$

$$\text{Lax-W: } u_i^{n+1} = (1 - \lambda^2)u_i^n + \frac{\lambda(1-\lambda)}{2}u_{i+1}^n + \frac{\lambda(1+\lambda)}{2}u_{i-1}^n$$

$$\text{Leap-F: } u_i^{n+1} = u_i^{n-1} - \lambda(u_{i+1}^n - u_{i-1}^n) \text{ (with Lax-W for first step)}$$

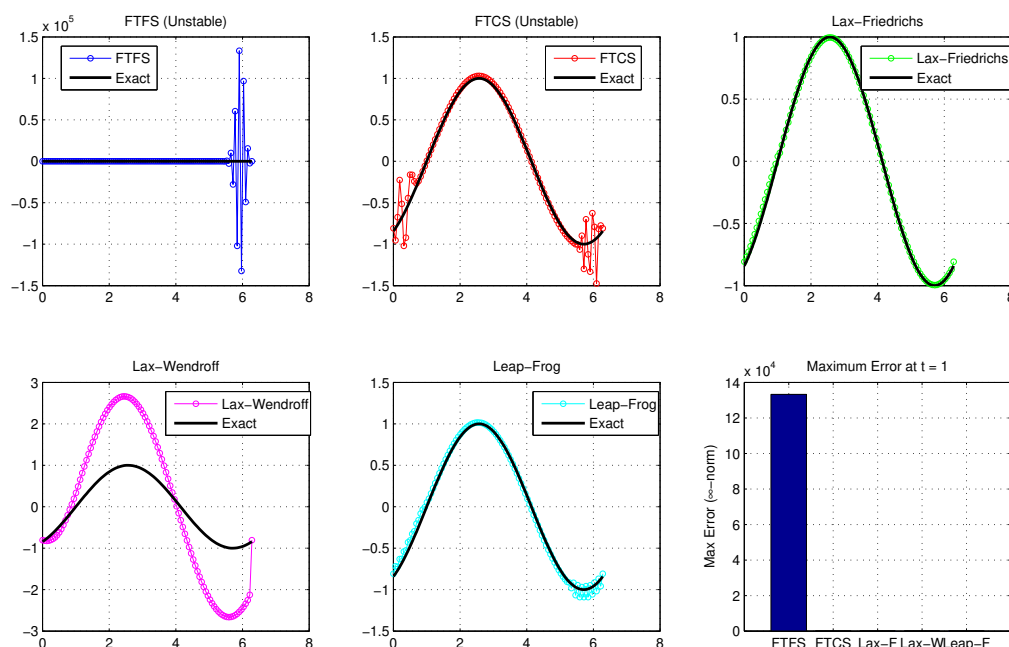


Figure 4.3: Comparison of the explicit methods.

Scheme	Max Absolute Error (∞ -norm) at $t = 1$
FTFS	1.33×10^5
FTCS	5.46×10^{-1}
Lax-Friedrichs	5.70×10^{-2}
Lax-Wendroff	1.67
Leap-Frog	1.02×10^{-1}

Observations:

- **FTFS** exhibits catastrophic error growth (10^5), confirming its unconditional instability for the advection equation with $c > 0$.
- **FTCS**, though formally unconditionally unstable, shows moderate error (0.55) at $t=1$; however, this masks underlying oscillatory instability that would dominate over longer integration times.

- **Lax-Friedrichs** is stable ($CFL < 1$) and produces the smallest maximum error (0.057) among all methods in this simulation, owing to its strong numerical dissipation that damps high-frequency errors even though it is only first-order accurate.
- Contrary to theoretical expectations, **Lax-Wendroff** yields a large error (1.67), likely due to dispersive oscillations amplified by the use of exact (non-periodic) boundary conditions, which introduce inconsistencies with the interior scheme.
- **Leap-Frog** (second-order, conditionally stable) achieves reasonable accuracy (0.10) but is outperformed by Lax-Friedrichs in this specific setup, possibly due to phase errors and sensitivity to the startup step.
- All methods enforce exact boundary values at $x=0$ and $x=2$, which suppresses boundary errors but may distort wave propagation for schemes sensitive to boundary mismatches (e.g., Lax-Wendroff).

This comparison illustrates the importance of numerical stability and order of accuracy in solving hyperbolic PDEs. For smooth solutions, **Lax-Wendroff** and **Leap-Frog** are preferred.

```

%% Finite Difference Methods for 1D Advection Equation
%   u_t + c*u_x = 0,   u(x,0) = sin(x)
%   Exact solution: u(x,t) = sin(x - c*t)
% Implemented schemes:
%   1. FTFS (Forward-Time Forward-Space) - unstable
%   2. FTCS (Forward-Time Central-Space) - unstable
%   3. Lax-Friedrichs - conditionally stable
%   4. Leap-Frog - conditionally stable, 2nd order
%   5. Lax-Wendroff - conditionally stable, 2nd order
% Uses periodic boundary conditions (appropriate for sin(x) on [0, 2])
clear; clc; close all;
%% Problem Parameters
c = 1;                % Wave speed
T = 1;                % Final time
x_left = 0;
x_right = 2*pi;
N = 100;               % Number of spatial points (periodic: x_N = x_0)
h = (x_right - x_left) / N;
CFL = 0.95;           % < 1 for stability
tau = CFL * h / c;    % Time step

```



```

M = ceil(T / tau);           % Number of time steps
tau = T / M;                 % Adjust to hit T exactly
lambda = c * tau / h;        % Courant number
fprintf('CFL number (lambda = c*tau/h): %.3f\n', lambda);
fprintf('N = %d, M = %d, h = %.4f, tau = %.4f\n', N, M, h, tau);

%% Spatial grid (periodic: x_{N+1} = x_1)
x = x_left + (0:N-1)' * h;   % x_i, i = 1,...,N (x_N = 2 - h)
u0 = sin(x);                 % Initial condition (periodic)

%% Exact solution at final time
u_exact_final = sin(x - c*T);

%% Initialize solution vectors (length N, periodic indexing)
u_ftfs = u0;
u_ftcs = u0;
u_lf = u0;                   % Lax-Friedrichs
u_lw = u0;                   % Lax-Wendroff

% Leap-Frog needs two levels
u_lf_prev = u0;              % u^{j-1}
if M >= 1
    % Start with Lax-Wendroff for first step (2nd-order accurate)
    u_lf_curr = zeros(N,1);
    for i = 1:N
        ip1 = mod(i, N) + 1; % i+1 with periodic wrap
        im1 = mod(i-2, N) + 1; % i-1 with periodic wrap
        u_lf_curr(i) = (1 - lambda^2)*u0(i) + ...
            0.5*lambda*(1 - lambda)*u0(ip1) + ...
            0.5*lambda*(1 + lambda)*u0(im1);
    end
else
    u_lf_curr = u0;
end

%% Time stepping
for j = 1:M
    t_j = j * tau;
    % --- FTFS: U_i^{n+1} = (1+lambda)U_i^n - lambda U_{i+1}^n
    u_ftfs_new = zeros(N,1);
    for i = 1:N
        ip1 = mod(i, N) + 1;

```

```

    u_ftfs_new(i) = (1 + lambda)*u_ftfs(i) - lambda*u_ftfs(ip1);
end
u_ftfs = u_ftfs_new;
% --- FTCS:  $U_i^{n+1} = U_i^n - (\lambda/2)(U_{i+1} - U_{i-1})$ 
u_ftcs_new = zeros(N,1);
for i = 1:N
    ip1 = mod(i, N) + 1;
    im1 = mod(i-2, N) + 1;
    u_ftcs_new(i) = u_ftcs(i) - (lambda/2)*(u_ftcs(ip1) - u_ftcs(im1));
end
u_ftcs = u_ftcs_new;
% --- Lax-Friedrichs
u_lf_new = zeros(N,1);
for i = 1:N
    ip1 = mod(i, N) + 1;
    im1 = mod(i-2, N) + 1;
    u_lf_new(i) = 0.5*(u_lf(im1) + u_lf(ip1)) - (lambda/2)*(u_lf(ip1) -
u_lf(im1));
end
u_lf = u_lf_new;

% --- Lax-Wendroff
u_lw_new = zeros(N,1);
for i = 1:N
    ip1 = mod(i, N) + 1;
    im1 = mod(i-2, N) + 1;
    u_lw_new(i) = (1 - lambda^2)*u_lw(i) + ...
        0.5*lambda*(1 - lambda)*u_lw(ip1) + ...
        0.5*lambda*(1 + lambda)*u_lw(im1);
end
u_lw = u_lw_new;
% --- Leap-Frog (for j >= 2; j=1 already done)
if j >= 2
    u_lf_new = zeros(N,1);
    for i = 1:N
        ip1 = mod(i, N) + 1;
        im1 = mod(i-2, N) + 1;
        u_lf_new(i) = u_lf_prev(i) - lambda*(u_lf_curr(ip1) - u_lf_curr(
im1));
    end
    u_lf_prev = u_lf_curr;

```

```

        u_lf_curr = u_lf_new;
    end
end
% Final Leap-Frog solution
u_leapfrog_final = u_lf_curr;
%% Compute errors
err_ftfs = norm(u_ftfs - u_exact_final, inf);
err_ftcs = norm(u_ftcs - u_exact_final, inf);
err_lf    = norm(u_lf - u_exact_final, inf);
err_lw    = norm(u_lw - u_exact_final, inf);
err_lfg   = norm(u_leapfrog_final - u_exact_final, inf);
%% Display errors
fprintf('\nMaximum Errors at t = %.1f:\n', T);
fprintf('FTFS:           %.2e\n', err_ftfs);
fprintf('FTCS:           %.2e\n', err_ftcs);
fprintf('Lax-Friedrichs:   %.2e\n', err_lf);
fprintf('Lax-Wendroff:     %.2e\n', err_lw);
fprintf('Leap-Frog:         %.2e\n', err_lfg);
%% Plotting
figure('Position', [100, 100, 1200, 800]);
methods = {'FTFS', 'FTCS', 'Lax-Friedrichs', 'Lax-Wendroff', 'Leap-Frog'};
solutions = {u_ftfs, u_ftcs, u_lf, u_lw, u_leapfrog_final};
errors = [err_ftfs, err_ftcs, err_lf, err_lw, err_lfg];
for k = 1:5
    subplot(2, 3, k);
    plot(x, solutions{k}, 'bo-', 'MarkerSize', 3, 'DisplayName', methods{k});
    hold on;
    plot(x, u_exact_final, 'k-', 'LineWidth', 1.5, 'DisplayName', 'Exact');
    title(methods{k});
    grid on;
    legend('Location', 'best');
    if k == 1, ylabel('u(x,t)'); end
    if k >= 4, xlabel('x'); end
end
% Error bar plot
subplot(2,3,6);
bar(errors);
set(gca, 'XTickLabel', methods, 'XTick', 1:5);
ylabel('Max Absolute Error  $\infty$ -norm');
title('Error Comparison at t = 1');
ylim([0, max(errors)*1.1]);

```

```
grid on;
set(gca, 'FontSize', 9);
xtickangle(45);
```

4.3 Finite Difference Methods for Second-Order Hyperbolic PDEs

We consider the one-dimensional homogeneous wave equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \quad x \in (0, L), \quad t > 0 \quad (4.14)$$

subject to initial conditions:

$$u(x, 0) = f(x), \quad (4.15)$$

$$\frac{\partial u}{\partial t}(x, 0) = g(x), \quad (4.16)$$

and boundary conditions:

$$u(0, t) = \phi(t), \quad u(L, t) = \psi(t) \quad (4.17)$$

where $c > 0$ is the wave speed, and $f(x), g(x), \phi(t), \psi(t)$ are continuous functions.

This equation models physical phenomena such as vibrating strings, acoustic waves, and electromagnetic pulses.

Finite Difference Approximation

Let: $h = \Delta x = L/N$: spatial step size, $\tau = \Delta t = T/M$: time step size, $x_i = ih$, $t_j = j\tau$, $U_{i,j} \approx u(x_i, t_j)$.

From Taylor series expansions, the second-order central difference approximations are:

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{\tau^2}, \quad (4.18)$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{h^2}. \quad (4.19)$$

Substitute into (4.14):

$$\frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{\tau^2} = c^2 \frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{h^2} \quad (4.20)$$

Define the Courant number:

$$\lambda = c \frac{\tau}{h}$$

Then rearrange (4.20):

$$U_{i,j+1} = 2U_{i,j} - U_{i,j-1} + \lambda^2(U_{i+1,j} - 2U_{i,j} + U_{i-1,j}) \quad (4.21)$$

for $i = 1, 2, \dots, N-1$, $j = 1, 2, \dots, M-1$.

This is an explicit three-level scheme, commonly known as the Courant–Friedrichs–Lewy (CFL) method or the standard finite difference scheme for the wave equation.

Initial Step ($j = 0$)

At $j = 0$, we need $U_{i,1}$. Use the initial velocity condition (4.16) and central difference:

$$\frac{\partial u}{\partial t}(x_i, 0) \approx \frac{U_{i,1} - U_{i,-1}}{2\tau} = g(x_i) \quad \Rightarrow \quad U_{i,-1} = U_{i,1} - 2\tau g(x_i)$$

But from the scheme at $j = 0$, set $U_{i,0} = f(x_i)$, and use symmetry to eliminate $U_{i,-1}$. Substitute into (4.21) with $j = 0$:

$$\begin{aligned} U_{i,1} &= U_{i,0} + \tau g(x_i) + \frac{\lambda^2}{2}(U_{i+1,0} - 2U_{i,0} + U_{i-1,0}) \\ U_{i,1} &= f(x_i) + \tau g(x_i) + \frac{\lambda^2}{2}(f(x_{i+1}) - 2f(x_i) + f(x_{i-1})) \end{aligned} \quad (4.22)$$

This provides the starting value for the time-stepping.

Stability Analysis (Von Neumann)

Assume $U_{i,j} = \xi^j e^{ikx_i}$. Substitute into (4.21):

$$\xi^{j+1} + \xi^{j-1} - 2\xi^j = \lambda^2(\xi^j e^{ikh} - 2\xi^j + \xi^j e^{-ikh}) = -4\lambda^2 \xi^j \sin^2(kh/2)$$

Divide by ξ^{j-1} :

$$\xi^2 - 2\xi + 1 = -4\lambda^2 \xi \sin^2(kh/2) \quad \Rightarrow \quad \xi^2 - 2(1 - 2\lambda^2 \sin^2(kh/2))\xi + 1 = 0$$

where $s = \sin^2(kh/2)$.

Roots:

$$\xi = (1 - 2\lambda^2 s) \pm \sqrt{(1 - 2\lambda^2 s)^2 - 1}$$

For stability, need $|\xi| \leq 1$. The roots are complex when discriminant < 0 , which occurs when $\lambda^2 s < 1$. The magnitude is:

$$|\xi|^2 = 1 \quad \text{if } \lambda \leq 1$$

The scheme is stable if and only if:

$$\lambda = c \frac{\tau}{h} \leq 1.$$

This is the CFL condition for the wave equation.

Order of Accuracy

Time: $O(\tau^2)$. Space: $O(h^2)$. Overall: $O(\tau^2 + h^2)$.

4.4 Elliptic Partial Differential Equations

Elliptic partial differential equations (PDEs) arise in numerous physical processes involving steady-state behavior. Classic examples include:

- Gravitational and electrostatic potentials,
- Steady-state heat conduction in solids,
- Incompressible irrotational fluid flow,
- Flow through porous media.

These equations are characterized by the absence of time derivatives, representing equilibrium or steady-state conditions. Most applications involve boundary value problems (BVPs) of Dirichlet, Neumann, or Robin (mixed) type. Consequently, numerical methods for elliptic PDEs are primarily developed for such boundary conditions.

Elliptic equations describe **steady-state** phenomena. They are typically defined on a closed bounded domain $\Omega \subset \mathbb{R}^2$ with boundary $\partial\Omega$, and involve boundary conditions of the form:

- **Dirichlet:** $u(x, y) = g(x, y)$ on $\partial\Omega$

- **Neumann:** $\frac{\partial u}{\partial \mathbf{n}} = h(x, y)$ on $\partial\Omega$
- **Robin (Mixed):** $au + b\frac{\partial u}{\partial \mathbf{n}} = c$ on $\partial\Omega$

The most fundamental elliptic equations are:

$$\nabla^2 u = 0 \quad (\text{Laplace's Equation}) \quad (3.1a)$$

$$\nabla^2 u = f(x, y) \quad (\text{Poisson's Equation}) \quad (3.1b)$$

4.4.1 Gravitational and Electrostatic Potentials

The gravitational potential U at a point $P(x, y, z)$ due to a mass m at the origin is:

$$U = -\frac{Gm}{r}, \quad r = \sqrt{x^2 + y^2 + z^2}$$

It can be shown that U satisfies Laplace's equation $\nabla^2 U = 0$ (outside the mass). Similarly, the electric potential V in a region with charge density ρ satisfies Poisson's equation:

$$\nabla^2 V = -\frac{\rho}{\varepsilon}$$

where ε is the permittivity of the medium.

4.4.2 Steady-State Heat Conduction

In the absence of heat sources and time dependence, the heat equation reduces to:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

This is Laplace's equation for temperature $T(x, y)$. If internal heat sources exist, it becomes Poisson's equation.

4.4.3 Dirichlet Problem for Laplace's Equation

Consider the Dirichlet problem for the two-dimensional Laplace equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0, \quad (x, y) \in \Omega \quad (3.2)$$

with boundary condition:

$$u(x, y) = g(x, y) \quad \text{on } \partial\Omega$$

We first consider a rectangular domain $\Omega = [0, a] \times [0, b]$, with uniform grid spacing.

4.4.4 Finite Difference Approximation

Let: $h = \Delta x$, $k = \Delta y$, $U_{i,j} \approx u(x_i, y_j)$, where $x_i = ih$, $y_j = jk$.

Using central differences:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{h^2}, \quad \frac{\partial^2 u}{\partial y^2} \approx \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{k^2}$$

Substitute into (3.2):

$$\frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{h^2} + \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{k^2} = 0 \quad (4.23)$$

Let $\lambda = k/h$. Multiply by h^2 :

$$(U_{i+1,j} + U_{i-1,j}) + \lambda^2(U_{i,j+1} + U_{i,j-1}) - 2(1 + \lambda^2)U_{i,j} = 0 \quad (4.24)$$

for $i = 1, \dots, M-1$, $j = 1, \dots, N-1$, where $M = a/h$, $N = b/k$.

This is known as the five-point finite difference scheme, as it involves five grid points.

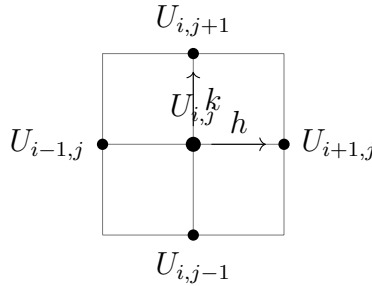


Figure 4.4: Five-point stencil for Laplace's equation.

Square Grid: $h = k$

When $h = k$, $\lambda = 1$, and (4.24) simplifies to:

$$U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{i,j} = 0 \quad (4.25)$$

This is a second-order accurate method.

For Poisson's equation $\nabla^2 u = f(x, y)$, the scheme becomes:

$$U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{i,j} = h^2 f_{i,j} \quad (4.26)$$

For the finite difference discretization of Poisson's equation

$$\nabla^2 u = f(x, y),$$

on a uniform grid with spacing h , the standard 5-point stencil yields the algebraic equation:

$$U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{i,j} = h^2 f_{i,j}. \quad (4.27)$$

This leads to a large sparse linear system $A\mathbf{u} = \mathbf{b}$, where $\mathbf{b}$ incorporates the source term $h^2 f_{i,j}$ and boundary conditions.

Consider the finite difference discretization of Poisson's equation on a uniform Cartesian grid with spacing h :

$$U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{i,j} = h^2 f_{i,j}. \quad (4.28)$$

This is the standard 5-point stencil for $\nabla^2 u = f(x, y)$.

Vectorization and Matrix Form

Let the domain contain n_x interior points in the x -direction and n_y in the y -direction, yielding $N = n_x n_y$ unknowns. We arrange the unknowns into a column vector $\mathbf{u} \in \mathbb{R}^N$ using lexicographic (row-wise) ordering:

$$\mathbf{u} = [U_{1,1}, U_{2,1}, \dots, U_{n_x,1}, U_{1,2}, \dots, U_{n_x,n_y}]^\top.$$

Then (4.28) leads to the linear system

$$A\mathbf{u} = \mathbf{b}, \quad (4.29)$$

where $\mathbf{b}_k = h^2 f_{i,j}$ (plus boundary contributions if Dirichlet data are nonhomogeneous), and $A \in \mathbb{R}^{N \times N}$ is the discrete Laplacian matrix.

Block Structure of A

The matrix A has the following block tridiagonal form:

$$A = \begin{bmatrix} T & -I & & & \\ -I & T & -I & & \\ & \ddots & \ddots & \ddots & \\ & & -I & T & -I \\ & & & -I & T \end{bmatrix}, \quad (4.30)$$

where: I is the $n_x \times n_x$ identity matrix, $T \in \mathbb{R}^{n_x \times n_x}$ is the tridiagonal matrix

$$T = \begin{bmatrix} -4 & 1 & & & \\ 1 & -4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -4 & 1 \\ & & & 1 & -4 \end{bmatrix}.$$

Kronecker Product Representation

Define the 1D discrete Laplacian matrix (with Dirichlet boundary conditions) as

$$L = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & & -1 & 2 \end{bmatrix} \in \mathbb{R}^{n_x \times n_x}.$$

Then the standard positive-definite 2D Laplacian is $I_{n_y} \otimes L + L \otimes I_{n_x}$. However, since (4.28) corresponds to $-\Delta_h \mathbf{u} = \mathbf{b}$, the matrix in (4.29) is

$$A = -(I_{n_y} \otimes L + L \otimes I_{n_x}). \quad (4.31)$$

Example: $n_x = n_y = 3$

For a 3×3 interior grid ($N = 9$), the matrix A is:

$$A = \begin{bmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{bmatrix}. \quad (4.32)$$

Each row enforces (4.28) at one interior grid point, with -4 on the diagonal and $+1$ for each neighboring node.

Three classical stationary iterative methods for solving this system are:

1. Jacobi Method

The Jacobi method computes all new values using only the values from the previous iteration:

$$U_{i,j}^{(k+1)} = \frac{1}{4} \left(U_{i+1,j}^{(k)} + U_{i-1,j}^{(k)} + U_{i,j+1}^{(k)} + U_{i,j-1}^{(k)} - h^2 f_{i,j} \right).$$

This method is simple and fully parallelizable but converges slowly.

2. Gauss–Seidel Method

The Gauss–Seidel method uses the most recently updated values as soon as they are available:

$$U_{i,j}^{(k+1)} = \frac{1}{4} \left(U_{i+1,j}^{(k)} + U_{i-1,j}^{(k+1)} + U_{i,j+1}^{(k)} + U_{i,j-1}^{(k+1)} - h^2 f_{i,j} \right).$$

It converges faster than Jacobi (approximately twice as fast for Poisson problems) but is not easily parallelizable.

3. Successive Over-Relaxation (SOR)

The SOR method accelerates Gauss–Seidel by introducing a relaxation parameter ω ($1 < \omega < 2$):

$$U_{i,j}^{(k+1)} = (1 - \omega)U_{i,j}^{(k)} + \frac{\omega}{4} \left(U_{i+1,j}^{(k)} + U_{i-1,j}^{(k+1)} + U_{i,j+1}^{(k)} + U_{i,j-1}^{(k+1)} - h^2 f_{i,j} \right).$$

When $\omega = 1$, SOR reduces to Gauss–Seidel. For a square domain with Dirichlet boundary conditions, the optimal relaxation parameter is approximately

$$\omega_{\text{opt}} \approx \frac{2}{1 + \sin(\pi h)}.$$

With an appropriate choice of ω , SOR can significantly accelerate convergence.

```
%=====
% 2D Laplace Equation                                     %
%   Features:                                             %
%   1. Animation of Gauss-Seidel convergence (finest grid) %
%   2. Convergence rate vs. grid size                     %
%   3. Error comparison table %   Plate: 2.4m x 3.0m, Top: 100C, Others: 0C%
clear; close all; clc; %% Parameters
L = 2.4;           % Length in x-direction
H = 3.0;           % Height in y-direction
T_top = 100;       % Top boundary temperature
N_terms = 30;      % Terms in analytical series
tol = 1e-5;        % Convergence tolerance
max_iter = 10000;
% Grid sizes for convergence study
grid_sizes = [20, 30, 40, 50, 60]; % Ny (and proportional Nx)
errors_L2 = zeros(size(grid_sizes));
errors_max = zeros(size(grid_sizes));
h_vec = zeros(size(grid_sizes));
iters = zeros(size(grid_sizes));
% Precompute analytical solution on fine grid for comparison
[Xa, Ya] = meshgrid(linspace(0, L, 100), linspace(0, H, 100));
U_analytical = analytical_solution(Xa, Ya, L, H, T_top, N_terms);
% Cell array to store numerical solutions for each grid
U_solutions = cell(size(grid_sizes));
%% Convergence Study over Grid Sizes
figure('Name', 'Convergence Study', 'Position', [100, 100, 1000, 600]);
for idx = 1:length(grid_sizes)
```

```

Ny = grid_sizes(idx);
Nx = round(Ny * L / H); % Maintain aspect ratio
dx = L / (Nx - 1);
dy = H / (Ny - 1);
h_vec(idx) = mean([dx, dy]);
fprintf('Grid %d: %d x %d, h = %.4f\n', idx, Nx, Ny, h_vec(idx));
% Initialize
U = zeros(Ny, Nx);
U(end, :) = T_top; % Top boundary
err_history = [];
% For animation: save frames only for finest grid
frames = {};
frame_interval = 50; % Save every 50th iteration
if idx == length(grid_sizes) % Only animate finest grid
    fprintf('Capturing animation frames for finest grid...\n');
end
% Gauss-Seidel iteration
for iter = 1:max_iter
    U_old = U;
    max_change = 0;
    for i = 2:Ny-1
        for j = 2:Nx-1
            U_new = 0.25 * (U(i+1,j) + U(i-1,j) + U(i,j+1) + U(i,j-1));
            max_change = max(max_change, abs(U_new - U(i,j)));
            U(i,j) = U_new;
        end
    end
    err_history(iter) = max_change;
% Capture frame for animation (only finest grid)
if idx == length(grid_sizes) && ...
    (mod(iter, frame_interval) == 0 || iter == 1 || max_change < tol)
    x_grid = linspace(0, L, Nx);
    y_grid = linspace(0, H, Ny);
    U_interp = interp2(x_grid, y_grid, U, Xa, Ya, 'linear');
    frames{end+1} = {U_interp, iter, max_change};
end
if max_change < tol
    iters(idx) = iter;
    break;
end
end
end

```

```

    if iter == max_iter
        iters(idx) = max_iter;
        warning('Grid %d did not converge in %d iterations.', idx, max_iter);
    end
    % Store solution
    U_solutions{idx} = U;
    % Interpolate to analytical grid for error computation
    x_grid = linspace(0, L, Nx);
    y_grid = linspace(0, H, Ny);
    U_num_interp = interp2(x_grid, y_grid, U, Xa, Ya, 'linear');
    % Compute errors
    abs_error = abs(U_analytical - U_num_interp);
    errors_L2(idx) = norm(abs_error(:)) / sqrt(numel(abs_error));
    errors_max(idx) = max(abs_error(:));
    % Plot convergence history
    subplot(2,3,idx);
    semilogy(1:length(err_history), err_history, 'b-', 'LineWidth', 1.5);
    hold on;
    semilogy(iters(idx), tol, 'ro', 'MarkerSize', 6, 'DisplayName', 'Tolerance');
    title(sprintf('%dx%d Grid', Nx, Ny));
    xlabel('Iteration'); ylabel('Max Change');
    grid on; hold off;
end
sgtitle('Gauss-Seidel Convergence for Different Grid Sizes', 'FontSize', 12);
%% 2. Animation of Convergence (Finest Grid Only)
if ~isempty(frames)
    disp('Creating convergence animation...');
    fig_anim = figure('Name', 'Gauss-Seidel Convergence Animation', 'Position',
        [200, 200, 800, 600]);
    for k = 1:length(frames)
        [U_frame, iter, err] = deal(frames{k}{:});
        contourf(Xa, Ya, U_frame, 50, 'LineStyle', 'none');
        colorbar;
        title(sprintf('Gauss-Seidel Iteration %d, Max Change: %.2e', iter, err));
    end;
    xlabel('x (m)'); ylabel('y (m)');
    axis equal tight;
    drawnow;
    if k == 1, pause(1); end
    if k < length(frames), pause(0.1); end
end

```

```

    end
else
    warning('No animation frames captured.');
```

```
end
%% 3. Error Comparison Table
fprintf('\n');
fprintf('-----\n');
fprintf('  Grid Size      h      Iterations      L2 Error      Max Error\n');
fprintf('-----\n');
for idx = 1:length(grid_sizes)
    Nx = round(grid_sizes(idx) * L / H);
    Ny = grid_sizes(idx);
    fprintf('    %3dx%-3d    %.4f    %4d    %10.2e    %10.2e\n', ...
        Nx, Ny, h_vec(idx), ...
        iters(idx), errors_L2(idx), errors_max(idx));
end
fprintf('-----\n');
%% 4. Convergence Rate Analysis
figure;
loglog(h_vec, errors_L2, 'bo-', 'LineWidth', 2, 'DisplayName', 'L2 Error');
hold on;
loglog(h_vec, errors_max, 'rs-', 'LineWidth', 2, 'DisplayName', 'Max Error');
% Fit line to estimate order
p = polyfit(log(h_vec), log(errors_L2), 1);
slope = p(1);
ref = 0.01 * h_vec.^2; % Reference for  $O(h^2)$ 
loglog(h_vec, ref, 'k--', 'LineWidth', 1, 'DisplayName', [' $O(h^{%.0f})$ '
    sprintf('%.0f', slope)]);
title(['Convergence Rate: Slope = ' num2str(slope, 2)]);
xlabel('Grid Size h'); ylabel('Error');
legend('Location', 'best');
grid on;
set(gca, 'XScale', 'log', 'YScale', 'log');
title('Error vs. Grid Size (Convergence Analysis)');
fprintf('\nEstimated convergence order (L2): %.2f\n', slope);
%% 5. Final Comparison Plot (at y = 1.5 m)
figure;
slice_y_idx = round(0.5 * size(U_analytical,1)); % Mid-height
plot(Xa(1,:), U_analytical(slice_y_idx,:), 'k-', 'LineWidth', 2, 'DisplayName',
    , 'Analytical');
hold on;
```

```

for idx = 1:length(grid_sizes)
    Nx = round(grid_sizes(idx) * L / H);
    Ny = grid_sizes(idx);
    x_vec = linspace(0, L, Nx);
    y_vec = linspace(0, H, Ny);
    U_interp = interp2(x_vec, y_vec, U_solutions{idx}, x_vec, H/2, 'linear');
    plot(x_vec, U_interp, '--', 'DisplayName', sprintf('Numerical %dx%d', Nx,
Ny));
end
xlabel('x (m)'); ylabel('Temperature (C)');
title('Temperature Profile at y = 1.5 m');
legend('Location', 'best');
grid on;
function U = analytical_solution(X, Y, L, H, T_top, N_terms)
    U = zeros(size(X));
    for k = 1:N_terms
        n = 2*k - 1;
        lambda = n * pi / L;
        An = (4 * T_top) / (n * pi);
        sinh_nH = sinh(lambda * H);
        if sinh_nH > 1e10, continue; end % Prevent overflow
        U = U + (An / sinh_nH) * sin(lambda * X) .* sinh(lambda * Y);
    end
end
end

```


Bibliography

- [1] Burden, R. L., & Faires, J. D. (2005). *Numerical Analysis* (8th ed.). Brooks/Cole, Cengage Learning.
- [2] LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM.
- [3] Smith, G. D. (1985). *Numerical Solution of Partial Differential Equations: Finite Difference Methods* (3rd ed.). Oxford University Press.
- [4] Iserles, A. (2009). *A First Course in the Numerical Analysis of Differential Equations* (2nd ed.). Cambridge University Press.
- [5] Hairer, E., Nørsett, S. P., & Wanner, G. (1993). *Solving Ordinary Differential Equations I: Nonstiff Problems* (2nd ed.). Springer.
- [6] Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (2nd ed.). Springer.
- [7] Ascher, U. M., Mattheij, R. M. M., & Russell, R. D. (1995). *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. SIAM.
- [8] Keller, H. B. (1992). *Numerical Methods for Two-Point Boundary-Value Problems*. Dover Publications.
- [9] Strikwerda, J. C. (2004). *Finite Difference Schemes and Partial Differential Equations* (2nd ed.). SIAM.
- [10] Thomas, J. W. (1995). *Numerical Partial Differential Equations: Finite Difference Methods*. Springer.
- [11] Evans, G., Blackledge, J., & Yardley, P. (2000). *Numerical Methods for Partial Differential Equations*. Springer.

- [12] Morton, K. W., & Mayers, D. F. (2005). *Numerical Solution of Partial Differential Equations: An Introduction* (2nd ed.). Cambridge University Press.

Advanced Numerical Methods:

Assignment I (30 Pts)

Instructions:

- Show all steps clearly.
- Use MATLAB or Python for computational parts unless otherwise stated.
- Submit a **PDF report** with code snippets, tables, plots, and analytical work.

Part I: Initial Value Problems (IVPs): 30 Points

1. (5 pts) Consider the IVP:

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5.$$

- (a) Derive the global error bound for Euler's method using Theorem 2.1.1 (page 36). Use this to compute a theoretical error bound at $t = 1.0$ with $h = 0.1$.
 - (b) Compute the actual error at $t = 1.0$ using Euler's method and compare with your bound. Comment on the conservativeness of the bound.
2. (5 pts) Apply the **classical fourth-order Runge–Kutta (RK4)** method with $h = 0.2$ to the same IVP above. Present a table of numerical results at $t = 0, 0.2, \dots, 2.0$, the exact solution, and absolute errors. Discuss why RK4 yields significantly smaller errors than Euler's method.
 3. (5 pts) Analyze the **stability region** of the RK4 method by plotting the boundary

$|R(z)| = 1$ in the complex plane, where

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

Compare it visually with Euler's stability region. Explain why this matters for non-stiff problems.

4. **(5 pts)** For the IVP in Question 1, apply the fourth order Predictor Corrector method. Compare your result with RK4 in Question number 2 result.
5. **(10 pts)** For the third-order IVP

$$t^3 y''' - t^2 y'' + 3ty' + 4y = 5t^3 \ln t + 9t^3, \quad 1 \leq t \leq 2,$$

subject to the initial conditions $y(1) = 0$, $y'(1) = 1$, $y''(1) = 3$, The exact solution is $y(t) = -t^2 + t \cos(\ln t) + t \sin(\ln t) + t^3 \ln t$.

- (a) Use the RK4 method for systems to approximate the solution, and compare the numerical results with the exact solution with step size $h = 0.1$
- (b) Use the 4th-order Adams–Bashforth–Moulton predictor–corrector method and compare the numerical results with the exact solution and the result in RK4 method for step size $h = 0.1$.

Advanced Numerical Methods:

Assignment II (15 Pts)

Instructions:

- Show all steps clearly.
- Use MATLAB or Python for computational parts unless otherwise stated.
- Submit a **PDF report** with code snippets, tables, plots, and analytical work.

Part II: Boundary Value Problems (BVPs): 15 Points

6. (8 pts) Solve the BVP,

$$y'' = y' + 2y + \cos x, \quad 0 < x < \frac{\pi}{2}, \quad y(0) = -0.3, \quad y\left(\frac{\pi}{2}\right) = -0.1,$$

using the linear shooting method with $N = 10$, $N = 20$, $N = 40$, $N = 80$; the exact solution is

$$y(x) = -\frac{1}{10}(\sin x + 3 \cos x).$$

- (a) Use RK4 to solve the two associated IVPs.
- (b) Compute the absolute error and the rate of convergence of the method.

7. (7 pts) Use the **Finite Difference Method** with central differences to discretize the BVP:

$$-u''(x) + e^{u(x)} = x + 1, \quad 0 < x < 1, \quad u(0) = u(1) = 0.$$

- (a) Derive the nonlinear system $G(U) = 0$ and its Jacobian.

- (b) Implement **Newton's method** (up to tolerance 10^{-6}) for $N = 32$.
- (c) Report the number of Newton iterations and plot the numerical solution.

Advanced Numerical Methods:

Assignment III (5 Pts)

Instructions:

- Show all steps clearly.
- Use MATLAB or Python for computational parts unless otherwise stated.
- Submit a **PDF report** with code snippets, tables, plots, and analytical work.

Part III: Partial Differential Equations: 5 Points

8. (5 pts) Consider the heat equation:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < \pi, \quad 0 < t \leq 1,$$

with $u(x, 0) = \sin(2x)$, $u(0, t) = u(\pi, t) = 0$.

- Implement the **explicit FTCS scheme** with $N = 20$ (spatial) and $M = 400$ (temporal).
- Verify the **stability condition** $r = \frac{\Delta t}{h^2} \leq \frac{1}{2}$.
- Compute the maximum absolute error at $t = 1$ and discuss consistency and convergence.